

50 CÂU LỆNH SQL

SẴN BUG THỰC CHIẾN

Dành cho QA · Tester — Dialect MySQL

```
ecommerce_test.sql

-- Tìm bug: email bị trùng (Bug-D)
SELECT email, COUNT(*) AS so_lan
FROM Customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY so_lan DESC;

-- Kết quả phát hiện:
trung_email@email.com | 2
```

50

Câu lệnh SQL

6

Nhóm chủ đề

15

Bug cài sẵn

MySQL

Dialect

- Mỗi câu lệnh: tình huống bug điển hình — SQL — phân tích mệnh đề — kết quả — góc soi lỗi
- Phần chuẩn bị: script tạo DB, sơ đồ ER, dữ liệu mẫu với 15 bug cố ý cài cắm để thực hành

Cách dùng cuốn cẩm nang này

Đây không phải tài liệu dạy SQL từ đầu. Nó là một bộ công cụ thực chiến: 50 câu lệnh để một người làm kiểm thử có thể tự mình đi vào database và tìm ra những lỗi mà giao diện không phơi bày. Mỗi câu lệnh được trình bày theo cùng một khuôn để bạn tra cứu nhanh:

- **Dữ liệu trước query** — bảng mẫu cho thấy trạng thái dữ liệu, dòng lỗi highlight đỏ.
- **Câu lệnh SQL** — sẵn sàng copy, viết theo dialect MySQL.
- **Phân tích từng mệnh đề** — mỗi từ khóa SQL được giải thích riêng.
- **Kết quả sau query** — bảng output sau khi chạy lệnh trên dữ liệu mẫu.
- **Góc soi lỗi của Tester** — cạm bẫy, hiểu lầm và bước tiếp theo.

ĐỌC TRƯỚC KHI CHẠY

Mọi câu lệnh trong sách đều là truy vấn ĐỌC (SELECT) — không sửa, không xóa dữ liệu. Dù vậy, hãy luôn chạy trên bản sao test/staging. Tên bảng/cột dùng schema ecommerce_test ở phần Chuẩn bị môi trường — đổi cho khớp schema thật của bạn.

NGUYÊN TẮC XUYỀN SUỐT

Săn bug bằng SQL thực chất là đi tìm những điều LẪY RA LUÔN ĐÚNG nhưng dữ liệu lại nói ngược lại: tồn kho không thể âm, tổng dòng con phải bằng header, email trong hệ thống phải là duy nhất. Với mỗi bất biến đó, bạn viết một câu lệnh tìm các bản ghi VI PHẠM. Cuốn sách này là 50 ví dụ của đúng một tư duy đó.

Mục lục

CHUẨN BỊ · Môi trường thực hành

PHƯƠNG PHÁP · Dịch yêu cầu thành câu lệnh sẵn bug

PHẦN 1 · Toàn vẹn và trùng lặp dữ liệu

01. Tìm email bị trùng
02. Tìm trùng theo nhiều cột (composite key)
03. Tìm NULL ở cột bắt buộc
04. Phát hiện khoảng trống thừa và ký tự ẩn
05. Kiểm tra bản ghi mồ côi (foreign key orphan)
06. Tìm trùng không phân biệt hoa/thường
07. Đếm giá trị NULL theo từng cột
08. Tìm bản ghi trùng hoàn toàn (full duplicate)
09. Kiểm tra giá trị ngoài danh sách cho phép (ENUM check)
10. Tìm bản ghi xóa mềm vẫn tham gia tính toán

PHẦN 2 · Ràng buộc nghiệp vụ

11. Đối soát tổng tiền đơn hàng với chi tiết items
12. Phát hiện tồn kho âm hoặc NULL
14. Phát hiện giá sản phẩm NULL hoặc bằng 0
15. Tìm đơn hàng không có dòng chi tiết nào
16. Tìm sản phẩm chưa bao giờ được bán
17. Tìm khách hàng chưa có đơn hàng nào
13. Khám phá schema bằng INFORMATION_SCHEMA khi chưa có tài liệu
18. Kiểm tra ràng buộc UNIQUE/FOREIGN KEY có thực sự được enforce
19. Tổng hợp chi tiêu theo từng khách hàng
20. Phát hiện sản phẩm đã bán vượt quá tồn kho

PHẦN 3 · Đối soát và tính toán

21. Tính doanh thu thực theo từng đơn từ Order_Items
22. Xếp hạng sản phẩm bán chạy nhất theo doanh số
23. Kiểm tra giá trị tồn kho (stock × price)
24. So sánh giá bán lịch sử với giá niêm yết hiện tại
25. Tổng doanh thu chỉ tính đơn hàng COMPLETED
26. Phân tích tỷ lệ đơn hàng theo trạng thái
27. Tổng doanh thu theo danh mục sản phẩm
28. Đối soát tồn kho hiện tại với lượng đã bán ra
29. Phát hiện đơn hàng bị tạo trùng (double order)
30. Phát hiện đơn hàng có giá trị bất thường (outlier)

PHẦN 4 · Biên và dữ liệu bất thường

31. Tìm tên khách hàng chứa ký tự bất thường
32. Tìm email không đúng định dạng cơ bản
33. Kiểm tra số lượng sản phẩm bất thường trong Order_Items
34. Kiểm tra ngày đặt hàng bất thường (tương lai hoặc quá xa quá khứ)
35. Tìm tên sản phẩm trùng sau khi chuẩn hóa
36. Phát hiện dữ liệu test/demo còn sót trong dữ liệu thật

- 37. Dò bản ghi gần-trùng bằng SOUNDEX (lỗi gõ chính tả)
- 38. Phát hiện đơn có tổng tiền nhưng không có sản phẩm nào
- 39. Phát hiện tổng items vượt quá 1.5 lần total_amount

PHẦN 5 · Audit, log và dấu vết

- 40. Truy vết item còn sót của đơn đã xóa mềm
- 41. Đối chiếu trạng thái hủy với cờ xóa mềm
- 42. Phát hiện đơn treo (PENDING) tồn đọng quá lâu
- 43. Dựng dòng thời gian đơn hàng — khoảng cách giữa các đơn
- 44. Phát hiện item_id bị nhảy — dấu vết của bản ghi bị xóa
- 45. Phân tích đơn hàng bị hủy — khách nào hay hủy nhất

PHẦN 6 · Truy vấn nâng cao cho QA

- 46. Dùng ROW_NUMBER() phát hiện item trùng trong cùng một đơn
- 47. Dùng CTE + RANK() xếp hạng sản phẩm bán chạy
- 48. Dùng CTE lồng nhau tìm khách chi tiêu trên mức trung bình
- 49. Tính doanh thu tích lũy theo thời gian với SUM() OVER()
- 50. Báo cáo tổng hợp: nhiều loại lỗi trong một câu UNION ALL

CASE STUDY · Một ngày điều tra dữ liệu của QA

KIỂM THỬ GHI DỮ LIỆU · Hành động trên app → xác minh DB

KIỂM THỬ GIAO DỊCH & TRUY CẬP ĐỒNG THỜI

CHẠY SQL AN TOÀN TRÊN PRODUCTION

PHỤ LỤC · Tra cứu nhanh

- A · Gặp triệu chứng → dùng câu nào
- B · Cheat sheet cú pháp lỗi
- C · Bản đồ 15 lỗi mẫu → câu phát hiện
- D · Đáp án bài tập tự luyện
- E · Tra cứu theo module nghiệp vụ

Chuẩn bị môi trường thực hành

Tất cả 50 câu lệnh trong sách đều chạy được ngay trên cơ sở dữ liệu **ecommerce_test** dưới đây. Hệ thống mô phỏng một sàn thương mại điện tử nhỏ gồm 4 bảng, với **15 bug được cố ý cài cắm** để bạn thực hành phát hiện.

Sơ đồ quan hệ 4 bảng (Entity Relationship)

Customers PK customer_id customer_name email membership_tier status	Products PK product_id product_name category price stock
Orders PK order_id FK customer_id total_amount status order_date deleted_at	Order_Items PK item_id FK order_id FK product_id quantity price

Customers 1 : N Orders | Orders 1 : N Order_Items | Products 1 : N Order_Items

PK = Primary Key (khóa chính) **FK** = Foreign Key (khóa ngoại, liên kết bảng khác)

Bảng	Lưu gì	Ví dụ trong sách
Customers	Danh sách khách hàng: họ tên, email, hạng thành viên (Standard / Silver / Gold), trạng thái tài khoản.	C001 = Nguyen Van A, Silver, ACTIVE.
Products	Danh mục sản phẩm: tên, danh mục, giá niêm yết, số lượng tồn kho.	PROD_001 = iPhone 15 Pro Max, giá 30.000.000, tồn 50.
Orders	Mỗi đơn hàng: ai đặt, trạng thái (PENDING / COMPLETED / CANCELLED), ngày đặt, và total_amount — tổng tiền toàn đơn. Giá trị này phải bằng SUM(price × quantity) của tất cả dòng Order_Items cùng order_id.	ORD_001 của C001, tổng 32.000.000, COMPLETED.
Order_Items	Chi tiết từng sản phẩm trong đơn — bảng nối Orders ↔ Products. Mỗi dòng = 1 sản phẩm: mua gì, số lượng, và price (giá 1 đơn vị lúc mua — lưu riêng vì giá sản phẩm có thể thay đổi sau).	ORD_001 có 2 sản phẩm → 2 dòng cùng order_id = ORD_001.

Lưu ý: **total_amount** (Orders) và **price** (Order_Items) là hai cột khác nhau. total_amount = tổng cả đơn; price = giá 1 đơn vị. Nếu hai con số này không khớp là có bug — ví dụ: ORD_002 ghi total_amount = 20.000.000 nhưng SUM(price × quantity) trong Order_Items = 31.000.000 → lệch 11.000.000 (Bug-B trong data mẫu).

Script tạo Database và bơm dữ liệu mẫu

Thay vì copy-paste thủ công, hãy tải file SQL sẵn có và chạy một lần duy nhất — database **ecommerce_test** cùng toàn bộ dữ liệu mẫu sẽ được tạo tự động.

Tải file: maiqai.com/books/ecommerce_test_setup.sql

Chạy trong MySQL Workbench: File → Open SQL Script → chọn file vừa tải → nhấn **Ctrl+Shift+Enter**.

Chạy bằng CLI: `mysql -u root -p < ecommerce_test_setup.sql`

Script tự xóa và tạo lại data mỗi lần chạy — tiện để reset sau khi thực hành.

Lỗi đã cài sẵn trong dữ liệu mẫu

Data mẫu chứa **15 lỗi cố ý** (đánh mã Bug-A → Bug-O) cài sẵn để bạn bắt bằng SQL, gom theo sáu nhóm dưới đây. Nhiều câu lệnh khác trong sách lại là **kiểm tra sạch** — kết quả rỗng nghĩa là dữ liệu đạt, không phải thất bại. Bản đồ đầy đủ từng lỗi ↔ câu phát hiện nằm ở **Phụ lục C** (cuối sách).

Trùng lặp & toàn vẹn dữ liệu

Email trùng (C004/C005 · C001/C009), composite key trùng trong Order_Items (item 1 và 7), email trùng không phân biệt hoa/thường, tên sản phẩm trùng hoàn toàn (PROD_002/PROD_005).

NULL & dữ liệu thiếu

C006: email = NULL. C007: email = chuỗi rỗng. PROD_006: price = NULL. PROD_007: stock = NULL.

Định dạng không hợp lệ

C008: customer_name có khoảng trắng thừa ở đầu và cuối (' Pham Van D '). C010: membership_tier = 'VIP' — ngoài danh sách Standard/Silver/Gold/Platinum.

Mô côi & ràng buộc tham chiếu

ORD_004: customer_id = 'C999' không tồn tại trong bảng Customers — đồng thời là đơn không có dòng item nào.

Giá trị nghiệp vụ sai & lệch số

item_id bỏ trống số 3 — gap trong chuỗi ID liên tục. PROD_003: stock = -5 (tồn kho âm). ORD_001: total_amount = 32.000.000 nhưng Order_Items cộng = 62.000.000 (do item trùng). ORD_002: total_amount = 20.000.000 nhưng Order_Items cộng = 31.000.000 (lệch 11.000.000).

Audit, xóa mềm & dấu vết

ORD_005: đã xóa mềm (deleted_at có giá trị) nhưng item 8, 9 chưa được dọn và đơn vẫn lọt vào tính toán. ORD_003: đã CANCELLED nhưng deleted_at vẫn trống — hai cột dấu vết lệch nhau.

Dữ liệu mẫu sau khi chạy script

Bảng Customers (10 dòng — dòng đỏ: lỗi trùng email, NULL, khoảng trắng, tier sai)

customer_id	customer_name	email	membership_tier	status
C001	Nguyen Van A	a.nguyen@email.com	Silver	ACTIVE
C002	Tran Van B	b.tran@email.com	Standard	ACTIVE
C003	Le Thi C	c.le@email.com	Gold	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	Standard	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	Standard	ACTIVE
C006	Pham Van X	(NULL)	Standard	ACTIVE
C007	Nguyen Thi Y		Standard	ACTIVE
C008	Pham Van D	d.pham@email.com	Gold	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	Silver	ACTIVE
C010	Khach Test VIP	vip@email.com	VIP	ACTIVE

Bảng Products (7 dòng — dòng đỏ: stock âm, NULL, tên trùng)

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

Bảng Orders (5 dòng — dòng đỏ: total_amount sai, orphan customer, ORD_005 đã xóa mềm)

order_id	customer_id	total_amount	status	order_date	deleted_at
ORD_001	C001	32.000.000	COMPLETED	2026-06-20	(NULL)
ORD_002	C002	20.000.000	COMPLETED	2026-06-22	(NULL)
ORD_003	C003	8.000.000	CANCELLED	2026-06-23	(NULL)
ORD_004	C999	5.000.000	PENDING	2026-06-24	(NULL)
ORD_005	C001	15.000.000	CANCELLED	2026-06-25	2026-06-25 10:30:00

Bảng Order_Items (8 dòng — dòng đỏ: composite key trùng; item_id=3 bị bỏ qua)

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000

item_id	order_id	product_id	quantity	price
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

item_id nhảy từ 2 lên 4 (thiếu 3). Item 7 trùng (order_id, product_id) với item 1. ORD_002: tổng items = 31.000.000 nhưng total_amount = 20.000.000 (lệch 11.000.000). Item 8 và 9 thuộc ORD_005 — đơn đã bị xóa mềm nhưng dòng chi tiết vẫn còn.

Phương pháp: Dịch một yêu cầu thành câu lệnh sẵn bug

50 câu trong sách là 50 lời giải có sẵn. Nhưng hệ thống bạn kiểm thử luôn có những quy tắc riêng mà không câu nào đoán trước được. Kỹ năng thật sự không nằm ở việc thuộc lòng câu lệnh, mà ở chỗ **biến một yêu cầu nghiệp vụ thành một truy vấn kiểm chứng**. Dưới đây là quy trình năm bước áp dụng được cho mọi nghiệp vụ.

Bước 1 — Tìm 'bất biến' (điều luôn phải đúng)

Mọi quy tắc nghiệp vụ đều ẩn một điều **LUÔN PHẢI ĐÚNG**. Viết nó ra thành một câu khẳng định, ví dụ:

- 'tổng tiền đơn = tổng tiền các dòng item'
- 'mỗi email chỉ thuộc một khách'
- 'tồn kho không bao giờ âm'

Bước 2 — Đảo thành câu hỏi vi phạm

Bug là nơi bất biến bị phá vỡ. Đổi 'X luôn đúng' thành 'tìm những bản ghi mà X SAI'. Đây là bước biến tư duy kiểm thử thành một mục tiêu truy vấn rõ ràng.

Bước 3 — Khoanh vùng bảng và cột

Bất biến đụng tới bảng và cột nào?

- Một bảng → WHERE đơn giản
- Nhiều bảng → cần JOIN hoặc anti-join
- So sánh hai tổng → cần GROUP BY và hàm tổng hợp

Bước 4 — Chọn khuôn mẫu SQL

Mỗi loại bất biến ứng với một mẫu quen thuộc (tra nhanh ở Phụ lục B):

- 'không được trùng' → GROUP BY ... HAVING COUNT(*) > 1
- 'phải tồn tại ở bảng kia' → NOT EXISTS
- 'hai tổng phải bằng nhau' → JOIN + GROUP BY + HAVING so sánh
- 'phải nằm trong danh sách' → NOT IN

Bước 5 — Chạy và diễn giải

- Kết quả RÕNG → bất biến đang được giữ; đó là tin tốt, không phải thất bại.
- Có dòng trả về → DANH SÁCH CẦN ĐIỀU TRA, không phải bản án: luôn xác minh thủ công trước khi kết luận bug.

Ví dụ áp dụng — từ một câu yêu cầu đến câu lệnh

YÊU CẦU (trích từ tài liệu nghiệp vụ)

“Mỗi khách hàng phải đăng ký bằng một địa chỉ email duy nhất; không hai tài khoản nào được dùng chung một email.”

Bước 1 — Bất biến: email là duy nhất giữa các khách hàng.

Bước 2 — Câu hỏi vi phạm: tìm những email được nhiều hơn một khách dùng.

Bước 3 — Bảng/cột: chỉ bảng Customers, cột email — một bảng nên không cần JOIN; so theo nhóm nên dùng GROUP BY.

Bước 4 — Khuôn mẫu: 'không được trùng' → GROUP BY email HAVING COUNT(*) > 1.

Bước 5 — Diễn giải: kết quả ra trung_email@email.com (C004 và C005) → điều tra hai tài khoản.

```
SELECT email,  
       COUNT(*) AS so_lan  
FROM   Customers  
GROUP BY email  
HAVING COUNT(*) > 1;
```

Câu lệnh vừa tự suy ra chính là Câu 1 trong sách. Khi gặp yêu cầu phức tạp hơn — ví dụ 'mỗi đơn hàng phải trở tới một khách có thật' — vẫn năm bước đó sẽ dẫn bạn tới mẫu anti-join (NOT EXISTS), tức Câu 5. Phương pháp không đổi, chỉ khuôn mẫu ở Bước 4 thay đổi theo loại bất biến.

PHẦN 1 — Toàn vẹn và trùng lặp dữ liệu

Lỗi dữ liệu kinh điển nhất mà QA bắt gặp: bản ghi nhân đôi, ô bắt buộc bị bỏ trống, khóa ngoại trỏ vào nơi không tồn tại. Nhóm câu lệnh này giúp bạn soi nhanh sức khỏe của một bảng trước khi đi sâu kiểm thử nghiệp vụ.

01 Tìm email bị trùng

TÌNH HUỐNG

Hệ thống đăng ký yêu cầu mỗi email chỉ thuộc về một tài khoản. Tuy nhiên do thiếu ràng buộc **UNIQUE** trên cột email, hai bản ghi C004 và C005 đã lọt vào bảng với cùng một địa chỉ. Hậu quả: gửi email nhầm người, đăng nhập nhầm nhằng, chiến dịch marketing tính trùng người dùng.

Bảng Customers — dòng đỏ: email bị trùng (Bug-D):

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT email,
       COUNT(*) AS so_lan
FROM   Customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY so_lan DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL bắt đầu từ đây: tải toàn bộ bảng Customers vào vùng xử lý — 10 dòng trong dữ liệu mẫu.
GROUP BY email	Gom nhóm tất cả dòng có cùng giá trị email lại với nhau. NULL và chuỗi rỗng mỗi loại thành một nhóm riêng.

HAVING COUNT(*) > 1	Lọc nhóm: chỉ giữ nhóm có nhiều hơn 1 bản ghi — tức là email xuất hiện từ 2 lần trở lên. HAVING khác WHERE ở chỗ nó lọc SAU khi gom nhóm.
SELECT email, COUNT(*) AS so_lan	Chỉ đến bước này MySQL mới chiếu kết quả : lấy email và đặt tên cột đếm là so_lan .
ORDER BY so_lan DESC	Sắp xếp kết quả cuối cùng: email trùng nhiều nhất nổi lên đầu.

Giải thích tổng thể

Kỹ thuật dùng ở đây là **GROUP BY + HAVING COUNT**: gom nhóm theo giá trị cần kiểm tra, rồi lọc những nhóm vi phạm tính duy nhất.

Đây là mẫu truy vấn nền tảng để phát hiện mọi loại trùng lặp trong bất kỳ cột nào.

Kết quả sau khi query (minh họa)

email	so_lan
a.nguyen@email.com	2
trung_email@email.com	2

2 cặp trùng: **trung_email@email.com** (C004 + C005 — Bug-D) và **a.nguyen@email.com** (C001 + C009 — trùng case-insensitive). Cần xử lý cả hai và thêm ràng buộc UNIQUE trước khi đưa lên production.

GÓC SOI LỖI CỦA TESTER

Kết quả của câu này phụ thuộc vào **collation** của cột email:

- (1) **Trùng khác hoa/thường**: trên DB mẫu (collation **utf8mb4_0900_ai_ci** — không phân biệt hoa/thường), câu này đã gộp luôn 'A.NGUYEN@EMAIL.COM' với 'a.nguyen@email.com'. Nhưng nếu cột dùng collation phân biệt hoa/thường (vd **utf8mb4_bin**), câu này sẽ BỎ SÓT cặp đó → khi ấy cần Câu 6.
- (2) **Trùng do khoảng trắng thừa** — ' test@mail.com' khác 'test@mail.com' → dùng Câu 4 để phát hiện.

02 Tìm trùng theo nhiều cột (composite key)

TÌNH HUỐNG

Quy tắc nghiệp vụ: mỗi đơn hàng không được chứa cùng một sản phẩm ở hai dòng riêng biệt. Tổ hợp (**order_id, product_id**) phải là duy nhất trong Order_Items. Nếu app bị double-submit hoặc thiếu kiểm tra, một sản phẩm có thể lọt vào hai lần.

Bảng Order_Items — dòng đỏ: cặp (order_id, product_id) xuất hiện lần 2:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT order_id,
       product_id,
       COUNT(*) AS so_lan
FROM   Order_Items
GROUP BY order_id, product_id
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items — 8 dòng trong ví dụ.
GROUP BY order_id, product_id	Gom nhóm theo tổ hợp hai cột. Mỗi nhóm đại diện cho một cặp (đơn hàng, sản phẩm) duy nhất.
HAVING COUNT(*) > 1	Lọc nhóm : chỉ giữ những tổ hợp xuất hiện nhiều hơn 1 lần — vi phạm tính duy nhất nghiệp vụ.
SELECT order_id, product_id, COUNT(*) AS so_lan	Chiều kết quả: tên đơn, tên sản phẩm và số lần trùng.

Giải thích tổng thể

Mẫu **GROUP BY + HAVING COUNT** áp dụng cho khóa tổ hợp.

Thay vì GROUP BY một cột như email, ta GROUP BY nhiều cột để đại diện cho quy tắc duy nhất phức tạp hơn.

Thêm bớt cột trong GROUP BY tùy theo quy tắc nghiệp vụ cần kiểm tra.

Kết quả sau khi query (minh họa)

order_id	product_id	so_lan
ORD_001	PROD_001	2

1 dòng: sản phẩm PROD_001 bị thêm hai lần vào ORD_001. Cần xóa dòng trùng và thêm UNIQUE(order_id, product_id) vào bảng.

GÓC SOI LỖI CỦA TESTER

Trước khi xóa dòng trùng, hãy kiểm tra:

(1) Hai dòng có item_id khác nhau không?

(2) Dòng nào được tạo sau — đó mới là dòng lỗi cần xóa.

Đừng xóa dựa trên giá định — sai item_id sẽ phá vỡ các bảng liên kết.

03 Tìm NULL ở cột bắt buộc

TÌNH HUỐNG

Cột email được quy định là bắt buộc trên giao diện đăng ký, nhưng luồng import dữ liệu cũ hoặc API nội bộ lại không kiểm tra. Kết quả: một số tài khoản không có email — không thể gửi thông báo, không thể reset mật khẩu, chiến dịch CRM bị lỗi.

Bảng Customers — dòng đỏ: email NULL hoặc chuỗi rỗng:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE

customer_id	customer_name	email	status
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       email
FROM   Customers
WHERE  email IS NULL
       OR TRIM(email) = '';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE email IS NULL OR TRIM(email) = ''	Lọc hai kiểu rỗng: IS NULL bắt giá trị chưa có, TRIM(email) = '' bắt chuỗi rỗng hoặc chỉ có khoảng trắng. Thiếu một trong hai điều kiện sẽ bỏ sót một kiểu lỗi.
SELECT customer_id, customer_name, email	Chiếu ra các cột để QA xác minh và báo cáo số lượng bị ảnh hưởng.

Giải thích tổng thể

Có **hai kiểu rỗng** hoàn toàn khác nhau trong SQL:

(1) **NULL** — chưa có giá trị, hệ thống không biết email là gì.

(2) **Chuỗi rỗng ""** — có giá trị nhưng là rỗng, biết email nhưng không có nội dung.

Điều kiện **email != ""** sẽ **KHÔNG** lọc được NULL vì mọi phép so sánh với NULL đều trả về UNKNOWN.

Kết quả sau khi query (minh họa)

customer_id	customer_name	email
C006	Pham Van X	(NULL)
C007	Nguyen Thi Y	

Số dòng trả về = quy mô dữ liệu thiếu. Dùng con số này để viết defect report mức độ ảnh hưởng.

GÓC SOI LỖI CỦA TESTER

Đừng chỉ kiểm tra **IS NULL** — đó chỉ là một nửa bức tranh.

(1) **Chuỗi rỗng**: hệ thống cũ thường lưu "" thay cho NULL → cần thêm điều kiện **TRIM(email) = ""**.

(2) **Khoảng trắng thừa**: ' test@email.com' vẫn được lưu nhưng sau TRIM mới lộ ra là chuỗi rỗng → kết hợp cả hai điều kiện vào một câu lệnh.

04

Phát hiện khoảng trắng thừa và ký tự ẩn

TÌNH HUỐNG

Người dùng copy-paste tên từ Excel hoặc nhập trên điện thoại dễ kéo theo dấu cách thừa ở đầu/cuối. Kết quả: 'Tran Van B' và ' Tran Van B ' bị coi là hai người khác nhau — trùng lặp logic nhưng không bị bắt bởi kiểm tra UNIQUE thông thường.

Bảng Customers — dòng đỏ: tên có khoảng trắng thừa:

customer_id	customer_name	status
C001	Nguyen Van A	ACTIVE
C002	Tran Van B	ACTIVE
C003	Le Thi C	ACTIVE
C004	Khach Hang Ao Bug	ACTIVE
C005	Khach Hang Trung	ACTIVE
C006	Pham Van X	ACTIVE
C007	Nguyen Thi Y	ACTIVE
C008	Pham Van D	ACTIVE
C009	Nguyen Van A (2)	ACTIVE
C010	Khach Test VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       CHAR_LENGTH(customer_name)      AS do_dai_goc,
       CHAR_LENGTH(TRIM(customer_name)) AS do_dai_sau_trim
FROM   Customers
WHERE  customer_name != TRIM(customer_name);
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE customer_name != TRIM(customer_name)	Lọc dòng mà độ dài gốc khác độ dài sau khi cắt trắng — tức là tồn tại khoảng trắng thừa ở đầu hoặc cuối.
SELECT ..., CHAR_LENGTH(customer_name), CHAR_LENGTH(TRIM(...))	Hiển thị độ dài gốc và sau trim để thấy chênh lệch bao nhiêu ký tự .

Giải thích tổng thể

TRIM chỉ cắt khoảng trắng ở hai đầu, không xóa khoảng trắng ở giữa tên.

So sánh CHAR_LENGTH trước và sau TRIM để đo chính xác có bao nhiêu ký tự thừa.

Nếu cần bắt cả tab (t) hay xuống dòng (n), dùng thêm REPLACE hoặc REGEXP.

Ket qua sau khi query (minh hoa)

customer_id	customer_name	do_dai_goc	do_dai_sau_trim
C008	Pham Van D	14	10

Mỗi dòng trả về cần được chuẩn hóa bằng UPDATE ... SET customer_name = TRIM(customer_name).

GÓC SOI LỖI CỦA TESTER

TRIM trong MySQL chỉ cắt dấu cách thường (ASCII 32).

Nếu dữ liệu nhập từ Excel, có thể tồn tại ký tự **non-breaking space** (\u00a0) — TRIM sẽ không bắt được. Xử lý bổ sung bằng: **REPLACE(customer_name, CHAR(160), '')** trước khi so sánh hoặc chuẩn hóa.

05 Kiểm tra bản ghi mồ côi (foreign key orphan)**TÌNH HUỐNG**

Bảng Orders có cột customer_id liên kết sang Customers. Nếu FK constraint bị tắt hoặc dữ liệu được import thủ công, có thể xuất hiện đơn hàng với customer_id không tồn tại — orphan record. Báo cáo doanh thu sẽ sai vì không join được thông tin khách.

Bảng Orders — dòng đỏ: customer_id không có trong Customers:

order_id	customer_id	total_amount	status
ORD_001	C001	32.000.000	COMPLETED
ORD_002	C002	20.000.000	COMPLETED
ORD_003	C003	8.000.000	CANCELLED
ORD_004	C999	5.000.000	PENDING
ORD_005	C001	15.000.000	CANCELLED

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id
FROM   Orders o
LEFT JOIN Customers c ON o.customer_id = c.customer_id
WHERE  c.customer_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o LEFT JOIN Customers c ON o.customer_id = c.customer_id	LEFT JOIN giữ lại TẤT CẢ dòng trong Orders — kể cả dòng không khớp với bất kỳ dòng nào trong Customers. Với INNER JOIN , dòng orphan sẽ biến mất và không phát hiện được.
WHERE c.customer_id IS NULL	Sau LEFT JOIN , những dòng Orders không khớp sẽ có c.customer_id = NULL. Lọc đúng các dòng đó.
SELECT o.order_id, o.customer_id	Chiếu ra order_id và customer_id để xác định đơn hàng nào bị mồ côi.

Giải thích tổng thể

Kỹ thuật **LEFT JOIN + WHERE IS NULL** là cách chuẩn để tìm orphan record.

Nguyên lý: **LEFT JOIN** giữ nguyên tất cả dòng bên trái. Những dòng không có cặp bên phải sẽ có giá trị **NULL** ở mọi cột của bảng phải.

Ta lọc đúng điều kiện đó để xác định bản ghi mồ côi.

Kết quả sau khi query (minh họa)

order_id	customer_id
ORD_004	C999

ORD_004 không có chủ. Cần xác minh: customer C999 có thực tồn tại nhưng bị xóa nhầm, hay đây là đơn hàng được tạo bởi bug?

GÓC SOI LỖI CỦA TESTER

Câu lệnh này kiểm tra chiều **Orders** → **Customers** (đơn hàng không có khách hàng).

Cần kiểm tra thêm các chiều khác:

(1) **Order_Items** → **Products**: item có product_id không tồn tại — phát hiện bằng cùng kỹ thuật LEFT JOIN ... IS NULL (hoặc NOT EXISTS), chỉ đổi cặp bảng.

(2) **Order_Items** → **Orders**: item thuộc order_id không tồn tại.

Cả hai đều là orphan nhưng ở tầng bảng khác nhau.

06 Tìm trùng không phân biệt hoa/thường

TÌNH HUỐNG

Database lưu email theo kiểu người dùng nhập — 'Test@Mail.com' và 'test@mail.com' là cùng một địa chỉ nhưng MySQL mặc định phân biệt hoa/thường trong một số collation. Kiểm tra UNIQUE trên cột gốc sẽ bỏ qua dạng trùng này.

Bảng Customers — dòng đỏ: email trùng khi so sánh không phân biệt hoa/thường:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A. NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT LOWER(email) AS email_chuan,
       COUNT(*)      AS so_lan
FROM   Customers
GROUP BY LOWER(email)
HAVING COUNT(*) > 1
ORDER BY so_lan DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
GROUP BY LOWER(email)	LOWER(email) chuyển về chữ thường trước khi gom nhóm. Nhờ vậy 'A. NGUYEN@EMAIL.COM' và 'a.nguyen@email.com' rơi vào cùng một nhóm.

HAVING COUNT(*) > 1	Chỉ giữ nhóm có nhiều hơn 1 bản ghi — email trùng sau khi chuẩn hóa.
SELECT LOWER(email) AS email_chuan, COUNT(*) AS so_lan	Chiếu email đã chuẩn hóa và số lần trùng.
ORDER BY so_lan DESC	Email trùng nhiều nhất hiện lên đầu.

Giải thích tổng thể

Bọc cột trong hàm **LOWER()** trước GROUP BY là kỹ thuật chuẩn hóa trước khi gom nhóm. Áp dụng tương tự với **UPPER()**, **TRIM()**, hay kết hợp cả hai tùy theo loại dữ liệu cần kiểm tra.

Kết quả sau khi query (minh họa)

email_chuan	so_lan
a.nguyen@email.com	2
trung_email@email.com	2

2 cặp trùng sau khi chuẩn hóa hoa/thường: **a.nguyen@email.com** (C001 + C009) và **trung_email@email.com** (C004 + C005). Trên DB mẫu (collation không phân biệt hoa/thường) Câu 1 cũng ra đúng hai cặp này; LOWER() ở đây đảm bảo phát hiện ĐÚNG bất kể collation — quan trọng khi bạn không chắc cột đang dùng collation nào.

GÓC SOI LỖI CỦA TESTER

Câu lệnh này chỉ cần thiết khi collation của cột là **case-sensitive** (vd: utf8mb4_bin). Nếu bảng dùng **utf8mb4_unicode_ci** (case-insensitive), MySQL tự động so sánh không phân biệt hoa/thường — Câu 1 đã đủ, câu này thừa. Kiểm tra collation trước khi quyết định dùng: **SHOW FULL COLUMNS FROM Customers;**

07 Đếm giá trị NULL theo từng cột

TÌNH HUỐNG

Thay vì kiểm tra NULL từng cột riêng lẻ, câu lệnh này cho ra một bảng tổng hợp — mỗi cột một ô — để QA thấy ngay bức tranh toàn cảnh: cột nào bị thiếu nhiều nhất, ưu tiên xử lý cột nào trước.

Bảng Products — dòng đỏ: có giá trị NULL trong cột số:

product_id	product_name	price	stock
PROD_001	iPhone 15 Pro Max	30.000.000	50
PROD_002	Bàn phím cơ Logitech	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	8.000.000	-5
PROD_004	Sạc du phong Anker	1.000.000	20
PROD_005	Bàn phím cơ Logitech	2.000.000	30
PROD_006	Loa Bluetooth JBL	(NULL)	10
PROD_007	Chuột gaming Razer	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT
  SUM(CASE WHEN product_name IS NULL THEN 1 ELSE 0 END) AS null_ten,
  SUM(CASE WHEN price        IS NULL THEN 1 ELSE 0 END) AS null_gia,
  SUM(CASE WHEN stock        IS NULL THEN 1 ELSE 0 END) AS null_ton
FROM Products;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
SUM(CASE WHEN ... IS NULL THEN 1 ELSE 0 END)	Với mỗi dòng, CASE WHEN trả về 1 nếu cột là NULL, 0 nếu không. SUM cộng lại — kết quả là tổng số NULL của cột đó. Áp dụng lặp lại cho từng cột cần kiểm tra.
SELECT null_ten, null_gia, null_ton	Kết quả là 1 dòng duy nhất chứa số NULL của mỗi cột — dạng báo cáo nhanh cho QA.

Giải thích tổng thể

Kỹ thuật **CASE WHEN + SUM** là cách pivot đơn giản: chuyển điều kiện thành số rồi tổng hợp. Không cần GROUP BY vì toàn bộ bảng là một nhóm duy nhất — kết quả trả về chỉ 1 dòng. Mở rộng bằng cách thêm cột vào SELECT để kiểm tra nhiều trường trong một lần chạy.

Kết quả sau khi query (minh họa)

null_ten	null_gia	null_ton
0	1	1

Cột price và stock đều có 1 NULL — cần điều tra nguồn dữ liệu và quyết định giá trị mặc định.

GÓC SOI LỖI CỦA TESTER

Mở rộng câu lệnh sang bảng **Customers** để tạo báo cáo chất lượng dữ liệu tổng thể:

- (1) Đếm NULL theo từng cột: **null_email, null_tier, null_status**.
- (2) Chạy ngay đầu sprint — phát hiện sớm giúp team thống nhất giá trị mặc định trước khi viết test case.

08

Tìm bản ghi trùng hoàn toàn (full duplicate)

TÌNH HUỐNG

Khác với trùng ID, full duplicate là hai dòng có toàn bộ giá nghiệp vụ giống nhau nhưng ID tự sinh khác nhau — thường xảy ra khi người dùng bấm nút 'Lưu' hai lần hoặc import dữ liệu không kiểm tra trùng trước.

Bảng Products — dòng đỏ: cùng tên và giá với dòng khác:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT product_name,
       price,
       COUNT(*) AS so_lan
FROM   Products
GROUP BY product_name, price
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
GROUP BY product_name, price	Gom nhóm theo tổ hợp các cột nghiệp vụ — ở đây là tên sản phẩm và giá. Thêm category vào GROUP BY nếu muốn kiểm tra chặt hơn.
HAVING COUNT(*) > 1	Lọc nhóm có nhiều hơn 1 dòng — tức là cùng tên và giá xuất hiện lặp lại.
SELECT product_name, price, COUNT(*) AS so_lan	Chiếu tên, giá và số lần trùng để xác định bản ghi cần xóa.

Giải thích tổng thể

Định nghĩa 'trùng hoàn toàn' phụ thuộc vào nghiệp vụ: có thể trùng theo 2, 3 hoặc toàn bộ cột.

Hãy thêm bớt cột trong GROUP BY tùy theo ngữ cảnh — chỉ đưa vào những cột đại diện cho giá trị nghiệp vụ, không phải ID tự sinh.

Kết quả sau khi query (minh họa)

product_name	price	so_lan
Ban phim co Logitech	2.000.000	2

Cần xác định PROD_002 hay PROD_005 là bản gốc trước khi xóa. Kiểm tra xem bản nào có Order_Items liên kết.

GÓC SOI LỖI CỦA TESTER

Trước khi xóa, hãy xem đầy đủ cả hai dòng để quyết định đúng:

SELECT * FROM Products

WHERE product_name = 'Ban phim co Logitech' AND price = 2000000;

Kiểm tra thêm: dòng nào đang có **Order_Items** liên kết — đó là bản gốc, dòng còn lại mới là bản trùng cần xóa.

09

Kiểm tra giá trị ngoài danh sách cho phép (ENUM check)

TÌNH HUỐNG

Cột membership_tier chỉ được nhận một trong bốn giá trị: Standard, Silver, Gold, Platinum. Nếu tầng ứng dụng không validate, hoặc dữ liệu được nhập thẳng vào DB qua script, các giá trị lạ sẽ lọt vào và làm vỡ logic ưu đãi.

Bảng Customers — dòng đỏ: membership_tier ngoài danh sách hợp lệ:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE

customer_id	customer_name	membership_tier	status
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       membership_tier
FROM   Customers
WHERE  membership_tier NOT IN ('Standard','Silver','Gold','Platinum');
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE membership_tier NOT IN ('Standard','Silver','Gold','Platinum')	NOT IN lọc tất cả dòng có giá trị không nằm trong danh sách. Lưu ý: NULL sẽ KHÔNG khớp NOT IN — cần thêm OR membership_tier IS NULL nếu muốn bắt cả NULL.
SELECT customer_id, customer_name, membership_tier	Chiều đủ thông tin để QA xác minh và cập nhật về giá trị đúng.

Giải thích tổng thể

NOT IN + danh sách tường minh là cách nhanh nhất để kiểm tra ENUM không được enforce ở tầng DB. Pattern này áp dụng cho MỌI cột có danh sách giá trị cố định trong hệ thống — chỉ đổi tên cột/bảng và danh sách hợp lệ, cấu trúc câu lệnh giữ nguyên:

(1) **Orders.status: NOT IN ('COMPLETED','PENDING','CANCELLED','PROCESSING')** — chạy trên data mẫu cho kết quả rỗng (4 trạng thái đều hợp lệ, xác nhận sạch).

(2) **Customers.status: NOT IN ('ACTIVE','INACTIVE','SUSPENDED')** — cũng cho kết quả rỗng, nhưng lưu ý data mẫu toàn ACTIVE nên chưa kiểm thử được luồng deactivate/suspend tài khoản — khi viết test case cần bổ sung data có đủ trạng thái.

Kết quả rỗng không phải thất bại — đó là confirmation hệ thống đang đúng, và câu lệnh nên chạy định kỳ hoặc sau mỗi lần migrate dữ liệu để phát hiện sớm giá trị lạ.

Ket qua sau khi query (minh họa)

customer_id	customer_name	membership_tier
C010	Khach Test VIP	VIP

1 bản ghi có tier không hợp lệ. Cần cập nhật về giá trị đúng và bổ sung CHECK constraint hoặc ENUM type trong DB.

GÓC SOI LỖI CỦA TESTER

Cạm bẫy với **NOT IN**: nếu danh sách chứa NULL, toàn bộ điều kiện trả về UNKNOWN.

Ví dụ: **WHERE tier NOT IN ('Standard', NULL)** — không trả về dòng nào, kể cả dòng có tier = 'VIP'.

Quy tắc: **luôn đảm bảo danh sách NOT IN không chứa NULL** — hoặc dùng NOT EXISTS thay thế để an toàn hơn. Bẫy này áp dụng cho mọi cột ENUM, không riêng membership_tier.

10 Tìm bản ghi xóa mềm vẫn tham gia tính toán**TÌNH HUỐNG**

Hệ thống dùng cơ chế **soft-delete** — khi hủy đơn hàng, cột **deleted_at** được ghi thời điểm xóa thay vì xóa hẳn bản ghi. Tuy nhiên nhiều query báo cáo (tổng doanh thu, số đơn trong tháng...) quên điều kiện **WHERE deleted_at IS NULL**, khiến đơn đã hủy vẫn bị tính vào kết quả. Đây là bug rất phổ biến và khó phát hiện bằng mắt thường vì dữ liệu 'trông có vẻ đúng'.

Bảng Orders — dòng đỏ: đơn ORD_005 đã bị xóa mềm (deleted_at có giá trị) nhưng vẫn nằm trong bảng:

order_id	customer_id	total_amount	status	deleted_at
ORD_001	C001	32.000.000	COMPLETED	None
ORD_002	C002	20.000.000	COMPLETED	None
ORD_003	C003	8.000.000	CANCELLED	None
ORD_004	C999	5.000.000	PENDING	None
ORD_005	C001	15.000.000	CANCELLED	2026-06-25 10:30:00

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.total_amount,
       o.status,
       o.deleted_at
FROM   Orders o
WHERE  o.deleted_at IS NOT NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o	Quét toàn bộ bảng Orders — bao gồm cả bản ghi đã bị xóa mềm.
WHERE o.deleted_at IS NOT NULL	deleted_at IS NOT NULL : lọc ra các đơn hàng đã bị đánh dấu xóa. Bản ghi vẫn tồn tại trong bảng nhưng lẽ ra không được tham gia vào bất kỳ phép tính nào.
SELECT o.order_id, o.total_amount, o.status, o.deleted_at	Chiều đủ thông tin để QA đối chiếu: đơn nào bị xóa, số tiền bao nhiêu, trạng thái gì — từ đó kiểm tra xem query báo cáo có đang cộng nhầm những đơn này không.

Giải thích tổng thể

Cơ chế **soft-delete** giữ lại bản ghi để phục vụ audit trail, nhưng yêu cầu **mọi query nghiệp vụ** đều phải thêm điều kiện **WHERE deleted_at IS NULL**.

Bug xảy ra khi developer viết query mới mà quên điều kiện này — đơn đã xóa lặng lẽ cộng vào tổng doanh thu.

QA nên chạy câu này để biết có bao nhiêu bản ghi soft-delete, sau đó đối soát với báo cáo tổng hợp để

phát hiện chênh lệch.

Ket qua sau khi query (minh hoa)

order_id	total_amount	status	deleted_at
ORD_005	15.000.000	CANCELLED	2026-06-25 10:30:00

ORD_005 đã bị xóa mềm nhưng vẫn nằm trong bảng. Nếu query báo cáo không lọc deleted_at, tổng doanh thu sẽ bị thổi phồng thêm 15 triệu.

GÓC SOI LỖI CỦA TESTER

Soft-delete là pattern phổ biến — hầu hết hệ thống e-commerce, CRM, ERP đều dùng cơ chế này thay vì xóa cứng (hard delete).

Khi test, QA cần kiểm tra **mọi màn hình báo cáo** và đảm bảo chúng có điều kiện lọc bản ghi đã xóa.

Mẹo: tìm trong code tất cả câu SELECT trên bảng có cột deleted_at — câu nào thiếu WHERE deleted_at IS NULL là nghi vấn bug.

BÀI TẬP TỰ LUYỆN — PHẦN 1

Bài 1.1. Có những hạng thành viên (membership_tier) nào đang được gán cho từ 2 khách trở lên? Đếm mỗi hạng có bao nhiêu khách.

Gợi ý: GROUP BY membership_tier rồi lọc bằng HAVING COUNT(*) > 1.

Bài 1.2. Tìm những khách hàng có customer_name chứa khoảng trắng thừa ở đầu hoặc cuối.

Gợi ý: So sánh customer_name với TRIM(customer_name); khác nhau là có khoảng trắng thừa.

Bài 1.3. Sản phẩm nào được mua trong nhiều hơn một đơn hàng khác nhau?

Gợi ý: Đếm số order_id PHÂN BIỆT cho mỗi product_id: COUNT(DISTINCT order_id).

→ Lời giải đầy đủ ở **Phụ lục D** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 2 — Ràng buộc nghiệp vụ

Mỗi hệ thống đều có những quy tắc bất thành văn: tổng tiền không thể âm, tồn kho không thể dưới không, trạng thái phải nằm trong danh sách cho phép. Khi tăng ứng dụng quên kiểm tra, dữ liệu sai sẽ lặng lẽ trôi vào database. Nhóm này cũng trang bị hai công cụ nền tảng: đọc schema một hệ thống lạ khi chưa có tài liệu, và xác minh rằng buộc đã khai báo có thực sự tồn tại không.

11

Đối soát tổng tiền đơn hàng với chi tiết items

TÌNH HUỐNG

Hệ thống lưu **total_amount** trong Orders nhưng không tự đối soát với tổng tính từ Order_Items. Khi có dữ liệu nhân đôi hoặc bug logic, hai con số lặng lẽ lệch nhau mà không có cảnh báo nào.

Bảng Orders — dòng đồ: total_amount lệch với tổng Order_Items:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.total_amount           AS ghi_trong_don,
       SUM(i.quantity * i.price) AS tinh_tu_items,
       o.total_amount
       - SUM(i.quantity * i.price) AS chenh_lech
FROM   Orders o
JOIN   Order_Items i ON o.order_id = i.order_id
GROUP BY o.order_id, o.total_amount
HAVING SUM(i.quantity * i.price) != o.total_amount;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Order_Items i ON o.order_id = i.order_id	INNER JOIN ghép mỗi đơn hàng với tất cả items của nó. Đơn không có item sẽ bị bỏ qua — dùng LEFT JOIN + Câu 15 để bắt thêm.
GROUP BY o.order_id, o.total_amount	Gom toàn bộ items của cùng một đơn thành một nhóm. total_amount cần có trong GROUP BY vì được dùng trong HAVING.
HAVING SUM(i.quantity * i.price) != o.total_amount	HAVING lọc sau khi đã tính aggregate — chỉ giữ đơn có tổng items khác với total_amount đã ghi.

<pre>SELECT ..., chenh_lech</pre>	Hiển thị cả ba con số: ghi bao nhiêu, tính ra bao nhiêu, chênh lệch bao nhiêu — đủ thông tin để QA viết defect report.
-----------------------------------	--

Giải thích tổng thể

Kỹ thuật **JOIN + GROUP BY + HAVING** để đối soát header-detail: total_amount trong Orders phải bằng SUM(quantity × price) từ Order_Items.

Kết quả phát hiện **hai nguyên nhân khác nhau**:

- (1) ORD_001 lệch vì item bị nhân đôi (item 1 và item 7 cùng order + product).
 - (2) ORD_002 lệch vì total_amount bị ghi sai thủ công (Bug-B).
- Cùng triệu chứng nhưng cách xử lý khác nhau — cần tra log để phân biệt.

Ket qua sau khi query (minh hoa)

order_id	ghi_trong_don	tinh_tu_items	chenh_lech
ORD_001	32.000.000	62.000.000	-30.000.000
ORD_002	20.000.000	31.000.000	-11.000.000
ORD_005	15.000.000	3.000.000	12.000.000

3 đơn lệch: ORD_001 do item trùng (62M vs 32M); ORD_002 do total_amount ghi sai (31M vs 20M); ORD_005 đã bị xóa mềm nhưng vẫn tham gia đối soát — minh họa bug Câu 10 (soft-delete leak).

GÓC SOI LỖI CỦA TESTER

INNER JOIN làm ẩn các đơn không có item — những đơn này có thể có total_amount > 0 nhưng không bao giờ xuất hiện trong kết quả câu này.

Kết hợp với **Câu 15** để kiểm tra toàn diện:

- (1) Câu 11: đơn có item nhưng tổng lệch → bug tính toán hoặc dữ liệu nhân đôi.
- (2) Câu 15: đơn không có item nào → trường hợp nghiêm trọng hơn.

Cảnh báo 'fan-out': mẫu JOIN + SUM này chỉ đúng khi đối soát với MỘT bảng con (Order_Items). Nếu JOIN thêm bảng con thứ hai (vd Order_Discounts, Shipments), mỗi dòng đơn bị nhân bản và SUM sẽ phồng sai toàn bộ. Khi có nhiều bảng con, hãy gộp (aggregate) từng bảng trong subquery riêng rồi mới JOIN các kết quả đã gộp.

12

Phát hiện tồn kho âm hoặc NULL

TÌNH HUỐNG

Cột **stock** có hai kiểu lỗi cần soi cùng lúc: giá trị **âm** (bán quá số lượng thực có) và giá trị **NULL** (chưa từng được nhập kho). Hai lỗi này có nguyên nhân và cách xử lý khác hẳn nhau, nhưng đều khiến trang sản phẩm hiển thị sai tình trạng còn hàng.

Bảng Products — dòng đỏ: stock âm (Bug-C) và stock NULL:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10

product_id	product_name	category	price	stock
PROD_007	Chuot gaming Razer	Phu kiện	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT product_id,
       product_name,
       stock
FROM   Products
WHERE  stock < 0
       OR stock IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
WHERE stock < 0 OR stock IS NULL	Hai điều kiện gộp bằng OR: stock < 0 bắt tồn kho âm; stock IS NULL bắt dữ liệu chưa nhập. Không thể dùng = NULL vì NULL = NULL trả về UNKNOWN, không phải TRUE — đây là lỗi phổ biến nhất khi mới học SQL.
SELECT product_id, product_name, stock	Chiếu đủ thông tin để QA xác định sản phẩm và mức độ lệch.

Giải thích tổng thể

Câu lệnh đơn giản nhưng **cực kỳ quan trọng** trong kiểm thử e-commerce — gộp hai loại lỗi cùng một cột để có báo cáo đầy đủ trong một lần chạy.

Tồn kho âm thường xảy ra ở hai tình huống: (1) **Thiếu CHECK constraint** — DB chấp nhận ngay UPDATE stock = stock - 1 dù stock đang = 0, không báo lỗi gì; (2) **Race condition** — hai người cùng đặt hàng lúc còn 1 sản phẩm, cả hai cùng đọc stock = 1, cùng kiểm tra 'còn hàng', cùng trừ 1 → kết quả cuối stock = -1.

Tồn kho NULL nghĩa là **không có thông tin** — khác hẳn với stock = 0 (hết hàng nhưng đã biết rõ số lượng). Mọi phép so sánh với NULL đều trả về UNKNOWN nên bắt buộc dùng IS NULL/IS NOT NULL, không thể dùng = hay !=.

Ket qua sau khi query (minh họa)

product_id	product_name	stock
PROD_003	Tai nghe Sony WH-1000XM5	-5
PROD_007	Chuot gaming Razer	(NULL)

2 sản phẩm lỗi, hai nguyên nhân khác nhau: PROD_003 tồn kho = -5 (bán quá số lượng — điều tra race condition hoặc logic trừ kho); PROD_007 tồn kho = NULL (chưa nhập kho — bổ sung dữ liệu hoặc thêm DEFAULT 0 vào schema).

GÓC SOI LỖI CỦA TESTER

Thêm **CHECK constraint** vào DB để ngăn giá trị âm từ đầu:

ALTER TABLE Products ADD CONSTRAINT chk_stock CHECK (stock >= 0);

Constraint này chỉ chặn INSERT/UPDATE mới — dữ liệu âm đã có vẫn giữ nguyên, vẫn cần chạy câu lệnh trên để dọn dữ liệu cũ.

CHECK không chặn được NULL (NULL luôn vượt qua mọi CHECK trừ khi thêm NOT NULL) — muốn bắt buộc phải nhập, thêm riêng **NOT NULL** cho cột stock.

14

Phát hiện giá sản phẩm NULL hoặc bằng 0

TÌNH HUỐNG

Sản phẩm có **price = NULL** hoặc **price = 0** là dữ liệu chưa hoàn chỉnh. Nếu trang web tính tổng giỏ hàng bằng SUM(price × quantity), NULL sẽ làm toàn bộ tổng trả về NULL; price = 0 cho phép mua miễn phí ngoài ý muốn.

Bảng Products — dòng đỏ: price = NULL (chưa nhập giá):

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT product_id,
       product_name,
       price
FROM   Products
WHERE  price IS NULL
       OR price <= 0;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
WHERE price IS NULL OR price <= 0	IS NULL bắt giá chưa nhập; <= 0 bắt giá âm hoặc bằng 0. Dùng OR để kiểm tra cả hai trường hợp trong một câu lệnh.
SELECT product_id, product_name, price	Chiếu thông tin để QA xác định sản phẩm cần bổ sung hoặc sửa giá.

Giải thích tổng thể
Giá là trường nghiệp vụ quan trọng nhất trong e-commerce.
Câu lệnh bắt hai loại lỗi:
(1) **NULL** — chưa nhập giá, thường xảy ra khi sản phẩm mới được thêm vào nhưng chưa định giá.
(2) **<= 0** — giá không hợp lệ: âm hoặc bằng 0 ngoài ý muốn.
Trong data mẫu, PROD_006 chưa có giá — nếu người dùng thêm vào giỏ hàng, tổng đơn sẽ trả về NULL.

Ket qua sau khi query (minh hoa)

product_id	product_name	price
PROD_006	Loa Bluetooth JBL	(NULL)

PROD_006 chưa có giá. Cần bổ sung trước khi cho phép bán — hoặc ẩn sản phẩm này khỏi giao diện người dùng.

GÓC SOI LỖI CỦA TESTER

Để ngăn dữ liệu lỗi ngay từ đầu, schema nên có hai ràng buộc:

(1) **NOT NULL** — bắt buộc nhập giá, không cho để trống.

(2) **CHECK (price > 0)** — giá phải lớn hơn 0, không cho nhập âm hoặc bằng 0.

Nếu hệ thống có sản phẩm miễn phí hợp lệ, đổi thành **CHECK (price >= 0)** để cho phép price = 0 có chủ đích.

Tránh đặt **DEFAULT 0** cho cột price: khi tạo sản phẩm mới mà quên nhập giá, DB sẽ tự điền 0 thay vì báo lỗi — sản phẩm lạng lế lên sàn với giá miễn phí.

Cùng kỹ thuật này áp dụng được cho **Order_Items.price** (giá tại thời điểm bán) — chỉ đổi **WHERE price <= 0** (cột này NOT NULL theo schema nên không cần nhánh IS NULL). Hai cột price mang ý nghĩa khác nhau: Products.price là **giá niêm yết hiện tại**, Order_Items.price là **giá đã chốt lúc khách mua** — chúng có thể lệch nhau hợp lệ khi giá niêm yết đổi sau khi đơn đã tạo (xem Câu 24). Vì vậy cần kiểm tra độc lập trên cả hai bảng, đặc biệt sau khi deploy tính năng coupon, flash sale, hoặc discount. Thêm CHECK tương ứng cho Order_Items: **ALTER TABLE Order_Items ADD CONSTRAINT chk_item_price CHECK (price > 0);**

15 Tìm đơn hàng không có dòng chi tiết nào**TÌNH HUỐNG**

Mỗi đơn hàng phải có ít nhất một sản phẩm. Đơn rỗng — tồn tại trong Orders nhưng không có dòng nào trong Order_Items — là dấu hiệu bug luồng xử lý: đơn được tạo nhưng bước thêm sản phẩm bị lỗi hoặc bị bỏ qua.

Bảng Orders — dòng đỏ: ORD_004 không có item nào trong Order_Items:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id,
       o.total_amount,
       o.status
FROM   Orders o
LEFT JOIN Order_Items i
      ON o.order_id = i.order_id
WHERE  i.order_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

```
FROM Orders o
LEFT JOIN Order_Items i
      ON o.order_id = i.order_id
```

LEFT JOIN giữ TẤT CẢ đơn hàng, kể cả đơn không có dòng nào trong Order_Items. Với INNER JOIN, đơn rỗng sẽ biến mất.

```
WHERE i.order_id IS NULL
```

Sau LEFT JOIN, đơn không khớp có toàn bộ cột của Order_Items = NULL. Lọc những dòng đó là cách nhận diện đơn rỗng — đây là mẫu anti-join.

<pre>SELECT o.order_id, o.customer_id, o.total_amount, o.status</pre>	Hiển thị đủ thông tin để QA điều tra: ai đặt, số tiền bao nhiêu, trạng thái gì.
--	---

Giải thích tổng thể

Kỹ thuật **LEFT JOIN + WHERE IS NULL** (anti-join) là cách chuẩn để tìm bản ghi không có con. Nguyên lý: sau LEFT JOIN, mọi cột từ bảng bên phải sẽ là NULL với những dòng không khớp. Trong data mẫu, ORD_004 (khách C999 không tồn tại) cũng không có item nào — đây là đơn có **hai lỗi chồng nhau**: orphan customer và rỗng items.

Ket qua sau khi query (minh hoa)

order_id	customer_id	total_amount	status
ORD_004	C999	5.000.000	PENDING

ORD_004 không có item nào và customer_id C999 không tồn tại. Hai lỗi chồng nhau — cần điều tra lịch sử tạo đơn.

GÓC SOI LỖI CỦA TESTER

Anti-join LEFT JOIN + WHERE IS NULL là pattern dùng lại ở nhiều câu:

- (1) Câu 5: Orders không có Customers — đơn hàng mò côi.
- (2) Câu 15: Orders không có Order_Items — đơn rỗng.
- (3) Câu 16: Products không có Order_Items — sản phẩm chưa bán.

Ba câu cùng kỹ thuật nhưng kiểm tra ở ba tầng quan hệ khác nhau.

16

Tìm sản phẩm chưa bao giờ được bán

TÌNH HUỐNG

Sản phẩm có trong danh mục nhưng không xuất hiện trong bất kỳ đơn hàng nào. Có thể là sản phẩm mới chưa ra mắt, sản phẩm bị ẩn nhưng dữ liệu chưa dọn, hoặc sản phẩm bị lỗi khiến không ai thêm được vào giỏ hàng.

Bảng Products — dòng đỏ: chưa có trong Order_Items:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       p.price,
       p.stock
FROM   Products p
LEFT JOIN Order_Items oi
      ON p.product_id = oi.product_id
WHERE  oi.product_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p LEFT JOIN Order_Items oi ON p.product_id = oi.product_id	LEFT JOIN giữ TẤT CẢ sản phẩm, kể cả sản phẩm không có dòng nào trong Order_Items.
WHERE oi.product_id IS NULL	Sau LEFT JOIN, sản phẩm không khớp có oi.product_id = NULL. Đây chính xác là sản phẩm chưa bao giờ được bán.
SELECT p.product_id, p.product_name, p.price, p.stock	Chiếu thêm price và stock để QA đánh giá: chưa bán vì lỗi, chưa có giá, hay chưa có hàng?

Giải thích tổng thể

Cùng kỹ thuật anti-join với Câu 5 và Câu 15, áp dụng cho chiều Products → Order_Items.

Trong data mẫu, 3 sản phẩm chưa bán — mỗi cái có lý do khác nhau:

(1) PROD_005: trùng tên PROD_002 — sản phẩm bị nhân đôi.

(2) PROD_006: price = NULL — chưa định giá.

(3) PROD_007: stock = NULL — chưa nhập kho.

Kết hợp với Câu 12 và Câu 14 để phân tích từng trường hợp.

Kết quả sau khi query (minh họa)

product_id	product_name	price	stock
PROD_005	Bàn phím cơ Logitech	2.000.000	30
PROD_006	Loa Bluetooth JBL	(NULL)	10
PROD_007	Chuột gaming Razer	1.500.000	(NULL)

3 sản phẩm chưa bán: PROD_005 trùng tên PROD_002, PROD_006 chưa có giá, PROD_007 chưa có tồn kho.

GÓC SOI LỖI CỦA TESTER

Sản phẩm chưa bán không nhất thiết là lỗi — có thể là hàng mới chưa ra mắt.

Để phân biệt, kết hợp thêm điều kiện từ Câu 12 và Câu 14:

(1) **PROD_005**: trùng với PROD_002 → xác nhận bằng Câu 8 rồi xóa bản thừa.

(2) **PROD_006**: price = NULL → cần nhập giá (Câu 14).

(3) **PROD_007**: stock = NULL → cần nhập tồn kho (Câu 12).

17 Tìm khách hàng chưa có đơn hàng nào

TÌNH HUỐNG

Khách hàng đã đăng ký tài khoản nhưng chưa đặt đơn nào. Có thể là dữ liệu test, tài khoản tạo nhầm, hoặc khách thật nhưng gặp lỗi khi checkout. Nhận diện nhóm này giúp làm sạch dữ liệu trước khi kiểm thử tính năng phân tích người dùng.

Bảng Customers — dòng đỏ: chưa có đơn hàng nào trong Orders:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name,
       c.membership_tier,
       c.status
FROM   Customers c
LEFT JOIN Orders o
      ON c.customer_id = o.customer_id
WHERE  o.customer_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers c LEFT JOIN Orders o ON c.customer_id = o.customer_id	LEFT JOIN giữ TẤT CẢ khách hàng, kể cả người chưa có đơn hàng nào.
WHERE o.customer_id IS NULL	Sau LEFT JOIN, khách không có đơn sẽ có cột o.customer_id = NULL. Lọc đúng những khách đó.
SELECT c.customer_id, c.customer_name, c.membership_tier, c.status	Chiếu thêm tier và status để QA phân loại: tài khoản test, tài khoản lỗi, hay khách thật chưa mua?

Giải thích tổng thể

Cùng kỹ thuật anti-join, chiều Customers → Orders.
Trong data mẫu, 7 trong 10 khách chưa có đơn — đây là tài khoản test được thêm để phục vụ các câu lệnh từ Câu 1 đến Câu 10.
Khi làm sạch dữ liệu trước sprint, QA nên chạy câu này và đánh dấu tài khoản nào là test — tránh đưa vào báo cáo làm sai số liệu thật.

Kết quả sau khi query (minh họa)

customer_id	customer_name	membership_tier	status
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE

customer_id	customer_name	membership_tier	status
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

7 khách chưa có đơn — phần lớn là tài khoản test tạo cho Phần 1. Cần đánh dấu để loại khỏi báo cáo sản xuất.

GÓC SOI LỖI CỦA TESTER

Câu này hay bị nhầm với Câu 5 (đơn hàng mờ côi). Điểm khác nhau:

(1) **Câu 5:** Orders không có Customers — đơn hàng mờ côi, thiếu chủ.

(2) **Câu 17:** Customers không có Orders — khách hàng chưa mua gì.

Hai hướng bổ sung cho nhau: Câu 5 bắt lỗi tăng Orders, Câu 17 bắt lỗi tăng Customers.

13

Khám phá schema bằng INFORMATION_SCHEMA khi chưa có tài liệu

TÌNH HUỐNG

QA mới nhận một hệ thống không có tài liệu database — chỉ có quyền SELECT, không biết có bao nhiêu bảng, cột nào kiểu gì, cột nào bắt buộc. Trước khi viết được bất kỳ câu lệnh soi lỗi nào, cần một cách tra cứu schema ngay trong chính SQL, không phụ thuộc tài liệu hay sơ đồ ER có thể đã lỗi thời.

Những gì QA nhìn thấy qua SELECT * — không biết cấu trúc đứng sau:

customer_id	customer_name	email	membership_tier	status
C001	Nguyen Van A	a.nguyen@email.com	Silver	ACTIVE
C002	Tran Van B	b.tran@email.com	Standard	ACTIVE
C003	Le Thi C	c.le@email.com	Gold	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	Standard	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	Standard	ACTIVE
C006	Pham Van X	(NULL)	Standard	ACTIVE
C007	Nguyen Thi Y		Standard	ACTIVE
C008	Pham Van D	d.pham@email.com	Gold	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	Silver	ACTIVE
C010	Khach Test VIP	vip@email.com	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT table_name,
       COUNT(*) AS so_cot,
       SUM(CASE WHEN is_nullable='YES'
                 THEN 1 ELSE 0 END) AS so_cot_nullable,
       SUM(CASE WHEN column_key='PRI'
                 THEN 1 ELSE 0 END) AS so_khoa_chinh
FROM   information_schema.columns
WHERE  table_schema = 'ecommerce_test'
GROUP BY table_name
ORDER BY table_name;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM information_schema.columns	information_schema là schema hệ thống có sẵn trong mọi DB MySQL — chứa metadata mô tả chính các bảng/cột trong DB, không cần quyền đặc biệt ngoài SELECT.
WHERE table_schema = 'ecommerce_test'	Lọc đúng database đang khảo sát — information_schema chứa metadata của TẤT CẢ database trên server, không lọc sẽ trả về lẫn lộn.
SUM(CASE WHEN ... THEN 1 ELSE 0 END)	Đếm có điều kiện: đếm số cột cho phép NULL và số cột là khóa chính, cho cái nhìn nhanh về độ "chặt" của từng bảng.
GROUP BY table_name	Gom theo từng bảng để có một dòng tổng quan mỗi bảng.

Giải thích tổng thể

information_schema.columns là bản đồ schema luôn cập nhật — không như tài liệu viết tay có thể đã lỗi thời từ lâu.

Mỗi DB có 4 bảng (Customers, Order_Items, Orders, Products), mỗi bảng có 1 khóa chính. Orders nhiều cột nullable nhất (3/6) — gợi ý nhiều trường tùy chọn, đáng kiểm tra kỹ.

Lưu ý: **table_name có thể trả về chữ thường** dù tên gốc viết hoa (tùy hệ điều hành server MySQL chạy) — luôn kiểm tra case thật trước khi lọc WHERE table_name = '...'.

Kết quả sau khi query (minh họa)

table_name	so_cot	so_cot_nullable	so_khoa_chinh
customers	5	3	1
order_items	5	2	1
orders	6	3	1
products	5	3	1

4 bảng, mỗi bảng 1 khóa chính. Orders có 6 cột (nhiều nhất, vì có thêm order_date và deleted_at) với 3 cột nullable — nên soi kỹ khi tìm dữ liệu thiếu.

GÓC SOI LỖI CỦA TESTER

Khi cần đào sâu một bảng cụ thể (kiểu dữ liệu từng cột, độ dài chuỗi tối đa...), bỏ GROUP BY và lọc thêm theo table_name:

```
SELECT column_name, column_type, is_nullable, column_key
FROM information_schema.columns
WHERE table_schema='ecommerce_test' AND table_name='Customers'
ORDER BY ordinal_position;
```

Đây là bước đầu tiên nên làm trước khi viết bất kỳ câu lệnh nào trên một hệ thống lạ — biến "đoán mò cấu trúc" thành "đọc trực tiếp từ nguồn đáng tin nhất".

18

Kiểm tra ràng buộc UNIQUE/FOREIGN KEY có thực sự được enforce

TÌNH HUỐNG

Sách liên tục khuyên "thêm UNIQUE cho email", "thêm FOREIGN KEY cho customer_id" — nhưng làm sao biết những ràng buộc đó **đã tồn tại hay chưa** trên một hệ thống cụ thể? Đừng tin lời đồn hay tài liệu cũ — tra trực tiếp metadata để biết DB đang thực sự bảo vệ những gì.

Bảng Customers — dòng đỏ: 2 email trùng (C004/C005) lẽ ra phải bị UNIQUE chặn:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT tc.table_name,
       tc.constraint_type,
       kcu.column_name
FROM   information_schema.table_constraints tc
LEFT JOIN information_schema.key_column_usage kcu
      ON tc.constraint_name = kcu.constraint_name
      AND tc.table_schema = kcu.table_schema
      AND tc.table_name = kcu.table_name
WHERE  tc.table_schema = 'ecommerce_test'
ORDER BY tc.table_name, tc.constraint_type;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM information_schema .table_constraints tc	Bảng hệ thống liệt kê MỌI ràng buộc đã khai báo: PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK.
LEFT JOIN information_schema .key_column_usage kcu	Nói thêm để biết ràng buộc đó áp dụng trên cột nào . Bắt buộc dùng LEFT JOIN thay vì JOIN thường: ràng buộc CHECK không có dòng tương ứng trong key_column_usage (CHECK không gắn với một cột theo kiểu khóa), nên INNER JOIN sẽ ÂM THÂM loại CHECK constraint khỏi kết quả — đúng loại lỗi khó nhận ra vì không báo gì cả.
WHERE tc.table_schema = 'ecommerce_test'	Giới hạn đúng database đang kiểm tra.

Giải thích tổng thể

Kết quả chỉ trả về **4 PRIMARY KEY** — đúng 1 cho mỗi bảng — và **không có một UNIQUE hay FOREIGN KEY** nào khác.

Đây chính là lời giải thích kỹ thuật cho lý do Câu 1 bắt được email trùng (C004/C005): không hề có UNIQUE constraint trên cột email để chặn từ đầu.

Tương tự, Câu 5 bắt được ORD_004 trở tới customer_id không tồn tại vì **không có FOREIGN KEY** nào ràng buộc Orders.customer_id phải khớp Customers.customer_id — DB chấp nhận mọi giá trị, đúng hay sai.

Ket qua sau khi query (minh hoa)

table_name	constraint_type	column_name
customers	PRIMARY KEY	customer_id
order_items	PRIMARY KEY	item_id
orders	PRIMARY KEY	order_id
products	PRIMARY KEY	product_id

Chỉ có khóa chính. Không một ràng buộc tham chiếu hay duy nhất nào tồn tại — toàn bộ việc giữ "sạch" dữ liệu trong hệ thống này hiện đang dựa hoàn toàn vào tầng ứng dụng, không có lưới an toàn ở tầng DB.

GÓC SOI LỖI CỦA TESTER

Khi báo cáo phát hiện thiếu ràng buộc, đừng chỉ nói "nên thêm UNIQUE" — hãy nói rõ **ĐANG THIẾU** kèm bằng chứng truy vấn này, để team không tranh cãi "chắc đã có rồi".

Thêm ràng buộc thực tế:

```
ALTER TABLE Customers ADD CONSTRAINT uq_email UNIQUE (email);
ALTER TABLE Orders ADD CONSTRAINT fk_customer
FOREIGN KEY (customer_id) REFERENCES Customers(customer_id);
```

Lưu ý: thêm UNIQUE sẽ **THẤT BẠI** nếu dữ liệu trùng (C004/C005) chưa được dọn trước — phải chạy Câu 1 xử lý dữ liệu cũ rồi mới ALTER TABLE được.

Nếu hệ thống ĐÃ có CHECK constraint, kết quả sẽ hiện dòng **constraint_type = CHECK** với **column_name = NULL** (do CHECK không gắn cột theo kiểu khóa) — để xem chính xác điều kiện CHECK kiểm tra gì, tra thêm bảng riêng:

```
SELECT constraint_name, check_clause FROM information_schema.check_constraints
WHERE constraint_schema = 'ecommerce_test';
```

19

Tổng hợp chi tiêu theo từng khách hàng

TÌNH HUỐNG

QA cần bảng tổng hợp để kiểm tra: khách nào đã mua, bao nhiêu đơn, tổng tiền bao nhiêu. Bảng này là nền để phát hiện bất thường — khách chi tiêu quá cao/thấp, hoặc số đơn không khớp với dữ liệu loyalty.

Bảng Orders — dữ liệu đầu vào để tổng hợp chi tiêu:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name,
       COUNT(o.order_id) AS so_don,
       SUM(o.total_amount) AS tong_chi_tieu
FROM   Customers c
JOIN   Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY tong_chi_tieu DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id	INNER JOIN: chỉ lấy khách đã có ít nhất một đơn. Khách chưa mua (C004–C010) sẽ không xuất hiện — dùng LEFT JOIN nếu muốn hiện cả họ với so_don = 0. ORD_004 (C999) cũng bị loại vì C999 không có trong Customers.
GROUP BY c.customer_id, c.customer_name	Gom tất cả đơn hàng của cùng một khách thành một nhóm để tính tổng.
SELECT ..., COUNT, SUM	COUNT(o.order_id) đếm số đơn; SUM(o.total_amount) cộng tổng tiền. Lưu ý: SUM dùng total_amount từ Orders — chưa chắc khớp với tổng items (xem Câu 11).
ORDER BY tong_chi_tieu DESC	Khách chi tiêu nhiều nhất lên đầu — để phát hiện outlier bất thường. Trên production, thêm LIMIT 10–20 để chỉ lấy nhóm khách chi tiêu cao nhất.

Giải thích tổng thể

Câu lệnh không tìm bug trực tiếp mà tạo **bảng nền để phát hiện bất thường**.

QA so sánh kết quả với dữ liệu hệ thống loyalty, CRM hoặc báo cáo tài chính — nếu con số lệch là có vấn đề.

Lưu ý: vì dùng total_amount từ Orders, kết quả bao gồm Bug-B (ORD_002 ghi sai).

Cần chạy Câu 11 trước để phát hiện và sửa dữ liệu lệch, sau đó Câu 19 mới phản ánh đúng thực tế.

Ket qua sau khi query (minh hoa)

customer_id	customer_name	so_don	tong_chi_tieu
C001	Nguyen Van A	2	47.000.000
C002	Tran Van B	1	20.000.000
C003	Le Thi C	1	8.000.000

Chỉ 3/10 khách có đơn (C001, C002, C003); 7 khách C004–C010 chưa mua nên không xuất hiện. C001 dẫn đầu 47M (2 đơn, bao gồm cả đơn ORD_005 đã bị xóa mềm). Con số C002 (20M) là Bug-B — thực tế items cộng lại 31M. C003 vẫn được tính dù đơn đã CANCELLED (câu này không lọc status).

GÓC SOI LỖI CỦA TESTER

Câu này dùng total_amount từ Orders — con số ghi sẵn, chưa chắc đúng.

Để tính chính xác từ Order_Items:

Thay **SUM(o.total_amount)** bằng **SUM(oi.quantity * oi.price)** sau khi JOIN thêm bảng Order_Items.

Chạy Câu 11 trước để phát hiện và sửa dữ liệu lệch, rồi mới dùng Câu 19 báo cáo.

20

Phát hiện sản phẩm đã bán vượt quá tồn kho

TÌNH HUỐNG

Tổng số lượng đã bán (từ Order_Items) lớn hơn tồn kho hiện tại (từ Products). Đây là dấu hiệu của race condition, thiếu kiểm tra stock trước khi trừ, hoặc dữ liệu kho bị cập nhật sai sau khi xác nhận đơn.

Bảng Products — dòng đỏ: tồn kho có thể bị vượt quá:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       p.stock,
       SUM(oi.quantity) AS tong_da_ban
FROM   Products p
JOIN   Order_Items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.stock
HAVING SUM(oi.quantity) > p.stock;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p JOIN Order_Items oi ON p.product_id = oi.product_id	INNER JOIN ghép mỗi sản phẩm với tất cả dòng trong Order_Items. Sản phẩm chưa bán sẽ không xuất hiện — dùng LEFT JOIN + Câu 16 để kiểm tra.
GROUP BY p.product_id, p.product_name, p.stock	Gom tất cả dòng Order_Items của cùng một sản phẩm để tính tổng số đã bán.
HAVING SUM(oi.quantity) > p.stock	HAVING so sánh sau aggregate: tổng đã bán > tồn kho hiện tại là vi phạm ràng buộc nghiệp vụ.
SELECT ..., tong_da_ban	Hiển thị cả stock hiện tại và tổng đã bán để QA thấy ngay mức độ vượt quá.

Giải thích tổng thể

Câu lệnh phát hiện vi phạm ràng buộc: **tổng bán ra không được vượt tồn kho**.
PROD_003 bị phát hiện vì stock = -5 và tong_da_ban = 1 (1 > -5).
Điều này nghĩa là hệ thống đã bán sản phẩm khi tồn kho đã âm — không có check trước khi giảm kho.
Với hệ thống thực, câu này giúp phát hiện overselling trước khi khách hàng phàn nàn vì không nhận được hàng.

Ket qua sau khi query (minh hoa)

product_id	product_name	stock	tong_da_ban
PROD_003	Tai nghe Sony WH-1000XM5	-5	1

PROD_003: tồn kho = -5 nhưng tổng đã bán = 1. Hệ thống đã cho phép bán khi kho đã âm — thiếu validation trước khi trừ kho.

GÓC SOI LỖI CỦA TESTER

INNER JOIN làm câu này bỏ qua sản phẩm chưa có đơn nào.

Ba câu cùng nhau tạo bức tranh đầy đủ về tình trạng kho:

- (1) **Câu 12:** stock âm — phát hiện trực tiếp không cần tính tổng bán.
- (2) **Câu 16:** sản phẩm chưa bán — góc nhìn ngược lại.
- (3) **Câu 20:** tổng bán > tồn kho — phát hiện overselling.

BÀI TẬP TỰ LUYỆN — PHẦN 2

Bài 2.1. Liệt kê các sản phẩm có giá cao hơn giá trung bình của toàn bộ sản phẩm.

Gợi ý: Dùng subquery (`SELECT AVG(price) FROM Products`) ngay trong mệnh đề `WHERE`.

Bài 2.2. Trong các đơn COMPLETED, đơn nào có total_amount KHÁC tổng tiền tính từ Order_Items?

Gợi ý: Lọc `WHERE status='COMPLETED'` trước, `JOIN + GROUP BY`, rồi `HAVING` so sánh hai tổng.

Bài 2.3. Sản phẩm nào CHƯA từng được bán nhưng vẫn còn tồn kho lớn hơn 0?

Gợi ý: `NOT EXISTS` để tìm sản phẩm chưa bán, kết hợp điều kiện `stock > 0`.

→ Lời giải đầy đủ ở **Phụ lục D** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 3 — Đối soát và tính toán

Phần lớn bug tài chính không nằm ở một bản ghi đơn lẻ mà ở chỗ hai con số đáng lẽ bằng nhau lại lệch nhau. Nhóm này tập trung vào kỹ thuật đối soát: tổng header so với tổng detail, tồn kho so với lượng đã bán.

21 Tính doanh thu thực theo từng đơn từ Order_Items

TÌNH HUỐNG

Câu 19 tổng hợp chỉ tiêu dùng **total_amount** — con số ghi sẵn trong Orders, có thể sai như Bug-B ở ORD_002. Doanh thu thật phải tính trực tiếp từ Order_Items: **SUM(quantity × price)**. Câu này là điểm khởi đầu để QA xác minh độ tin cậy của dữ liệu trước khi đưa vào báo cáo.

Bảng Order_Items — dòng đỏ: item_id=7 nhân đôi PROD_001 trong ORD_001:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT order_id,
       COUNT(*)           AS so_dong_items,
       SUM(quantity)       AS tong_so_luong,
       SUM(quantity * price) AS doanh_thu_thuc
FROM   Order_Items
GROUP BY order_id
ORDER BY order_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items — 8 dòng trong dữ liệu mẫu.
GROUP BY order_id	Gom tất cả dòng có cùng order_id thành một nhóm. Mỗi nhóm đại diện cho một đơn hàng.
COUNT(*), SUM(quantity), SUM(quantity * price)	COUNT(*) đếm số dòng items của đơn. SUM(quantity) tổng số lượng sản phẩm. SUM(quantity × price) tính tiền thực từ items.

ORDER BY order_id

Sắp xếp kết quả theo mã đơn để so sánh dễ hơn với bảng Orders.

Giải thích tổng thể

Kỹ thuật **GROUP BY + aggregate** tổng hợp từ bảng chi tiết lên level đơn hàng.

COUNT(*) là chỉ số cảnh báo sớm: nếu con số lớn bất thường so với dự kiến, rất có thể có item bị trùng.

Trong data mẫu, ORD_001 có 3 dòng thay vì 2 — COUNT(*) = 3 là tín hiệu đầu tiên trước khi đối soát doanh_thu_thuc với total_amount.

Kết quả sau khi query (minh họa)

order_id	so_dong_items	tong_so_luong	doanh_thu_thuc
ORD_001	3	3	62.000.000
ORD_002	2	2	31.000.000
ORD_003	1	1	8.000.000
ORD_005	2	2	3.000.000

ORD_001: 3 dòng items → COUNT bất thường, doanh thu = 62M (lẽ ra 32M). ORD_002: tổng = 31M, khác total_amount 20M (Bug-B). ORD_004 vắng mặt vì không có item nào.

GÓC SOI LỖI CỦA TESTER

Kết hợp câu này với Câu 11 để phân tích sâu hơn:

(1) **Câu 21**: tính doanh thu thực từ items theo từng đơn — con số tuyệt đối.

(2) **Câu 11**: chỉ lọc ra đơn có doanh_thu_thuc ≠ total_amount — xác định đúng đơn lỗi.

Câu 21 cho bức tranh tổng thể; Câu 11 chỉ đích danh đơn cần điều tra.

22**Xếp hạng sản phẩm bán chạy nhất theo doanh số****TÌNH HUỐNG**

Báo cáo sản phẩm bán chạy là cơ sở để ra quyết định nhập hàng và marketing. Nhưng nếu dữ liệu Order_Items có item trùng, bảng xếp hạng sẽ sai — sản phẩm bình thường đột ngột lên top vì được tính nhiều lần.

Bảng Order_Items — dòng đỏ: item_id=7 làm PROD_001 bị tính 3 lần:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       SUM(oi.quantity)           AS tong_so_luong,
       SUM(oi.quantity * oi.price) AS tong_doanh_so
FROM   Products p
JOIN   Order_Items oi
       ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY tong_doanh_so DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p JOIN Order_Items oi ON p.product_id = oi.product_id	INNER JOIN ghép mỗi sản phẩm với tất cả items bán ra. Sản phẩm chưa bán (PROD_005, 006, 007) bị loại — dùng LEFT JOIN nếu muốn hiện cả chúng với doanh_so = NULL.
GROUP BY p.product_id, p.product_name	Gom tất cả dòng Order_Items của cùng một sản phẩm thành một nhóm để tính tổng số lượng và doanh số.
ORDER BY tong_doanh_so DESC	Sản phẩm doanh số cao nhất nổi lên đầu — dễ phát hiện outlier nếu chênh lệch bất thường.

Giải thích tổng thể

Kỹ thuật **JOIN + GROUP BY + ORDER BY** là nền của mọi báo cáo xếp hạng.
Kết quả hiện tại bị méo vì dữ liệu đầu vào có lỗi: PROD_001 xuất hiện 3 lần trong items (item 1, 4, 7) thay vì 2.
Doanh số thực của PROD_001 chỉ là 60M — con số 90M là ảo do item trùng.
Bảng xếp hạng chính xác phải bắt đầu bằng việc xác nhận dữ liệu sạch (Câu 2).

Kết quả sau khi query (minh họa)

product_id	product_name	tong_so_luong	tong_doanh_so
PROD_001	iPhone 15 Pro Max	3	90.000.000
PROD_003	Tai nghe Sony WH-1000XM5	1	8.000.000
PROD_002	Bàn phím cơ Logitech	2	4.000.000
PROD_004	Sạc du phong Anker	2	2.000.000

PROD_001 dẫn đầu với 90M — thực ra chỉ 60M nếu không có item 7 trùng. Chạy Câu 2 để xác nhận trùng, sửa dữ liệu rồi mới tin vào kết quả xếp hạng này.

GÓC SOI LỖI CỦA TESTER

Trước khi dùng kết quả xếp hạng để ra quyết định kinh doanh, QA cần kiểm tra:

- (1) **Câu 2:** có item nào bị trùng (order_id + product_id) không?
- (2) **Câu 21:** COUNT(*) theo order có dòng nào bất thường không?

Dữ liệu bản ở Order_Items lan trực tiếp lên báo cáo cấp quản lý — đây là loại bug dễ bị bỏ qua nhất vì không gây lỗi hệ thống.

23

Kiểm tra giá trị tồn kho (stock × price)

TÌNH HUỐNG

Giá trị tồn kho = **stock × price** là chỉ số tài chính quan trọng trong quản lý kho. Nếu stock âm (Bug-C) hoặc price = NULL, giá trị tính ra sẽ bị âm hoặc NULL — báo cáo kế toán sẽ sai lệch.

Bảng Products — dòng đỏ: stock âm hoặc NULL sẽ cho gia_tri_kho sai:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT product_id,
       product_name,
       price,
       stock,
       price * stock AS gia_tri_kho
FROM   Products
ORDER BY gia_tri_kho;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products — 7 sản phẩm trong dữ liệu mẫu.
price * stock AS gia_tri_kho	MySQL tính tích hai cột và đặt tên kết quả là gia_tri_kho . Quy tắc: NULL × bất kỳ = NULL; âm × dương = âm.
ORDER BY gia_tri_kho	Sắp xếp tăng dần: MySQL đặt NULL trước tiên, rồi đến âm, rồi dương. Cách hiển thị này đưa dòng lỗi lên đầu bảng kết quả.

Giải thích tổng thể

Phép nhân đơn giản nhưng phản ánh ngay **hai loại lỗi dữ liệu** từ Câu 12 và Câu 14:

(1) **stock âm**: PROD_003 cho gia_tri_kho = -40.000.000 — không thể có kho giá trị âm.

(2) **NULL**: PROD_006 (price NULL) và PROD_007 (stock NULL) cho gia_tri_kho = NULL — không thể tính được giá trị kho, ảnh hưởng trực tiếp đến báo cáo tổng tài sản.

Ket qua sau khi query (minh hoa)

product_id	product_name	price	stock	gia_tri_kho
PROD_006	Loa Bluetooth JBL	(NULL)	10	(NULL)
PROD_007	Chuot gaming Razer	1.500.000	(NULL)	(NULL)
PROD_003	Tai nghe Sony WH-1000XM5	8.000.000	-5	-40.000.000

product_id	product_name	price	stock	gia_tri_kho
PROD_004	Sac du phong Anker	1.000.000	20	20.000.000
PROD_005	Ban phim co Logitech	2.000.000	30	60.000.000
PROD_002	Ban phim co Logitech	2.000.000	100	200.000.000
PROD_001	iPhone 15 Pro Max	30.000.000	50	1.500.000.000

3 dòng bất thường ở đầu: 2 NULL (không tính được) và 1 âm -40M (tồn kho âm do Bug-C). ORDER BY gia_tri_kho tự động đưa các dòng lỗi lên đầu — không cần WHERE để lọc riêng.

GÓC SOI LỖI CỦA TESTER

Để chỉ lấy dòng lỗi, thêm điều kiện lọc vào câu lệnh:

(1) Kho giá trị âm: **WHERE price * stock < 0**

(2) Kho không tính được: **WHERE price IS NULL OR stock IS NULL**

Câu 23 (kho âm) kết hợp với Câu 12 (stock âm) và Câu 14 (price NULL) tạo thành bộ 3 kiểm tra tồn kho đầy đủ.

24 So sánh giá bán lịch sử với giá niêm yết hiện tại

TÌNH HUỐNG

Cột **price** trong Order_Items là giá tại thời điểm mua — snapshot của giá khi giao dịch xảy ra. Cột **price** trong Products là giá niêm yết hiện tại. Nếu hai con số khác nhau, có thể bình thường (giá thay đổi) hoặc là bug (ghi nhầm giá lúc tạo đơn). QA cần kiểm tra để phân biệt hai trường hợp.

Bảng Order_Items — so sánh oi.price (lúc mua) với Products.price (hiện tại):

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT oi.order_id,
       oi.product_id,
       oi.price      AS gia_luc_ban,
       p.price       AS gia_hien_tai,
       oi.price - p.price AS chenh_lech
FROM   Order_Items oi
JOIN   Products p
      ON oi.product_id = p.product_id
WHERE  oi.price <> p.price;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items oi JOIN Products p ON oi.product_id = p.product_id	INNER JOIN ghép mỗi item với thông tin sản phẩm tương ứng. Kết quả có cả oi.price (lúc mua) và p.price (hiện tại) trên cùng một dòng.
WHERE oi.price <> p.price	Lọc chỉ những dòng có giá bán khác giá hiện tại. Kết quả rỗng = tất cả items được ghi đúng giá.
SELECT ..., oi.price - p.price AS chenh_lech	Tính độ lệch: dương = bán đắt hơn giá hiện tại; âm = bán rẻ hơn giá hiện tại.

Giải thích tổng thể

Order_Items.price là **giá snapshot** — lưu lại giá đúng lúc khách mua, không đổi kể cả khi Products.price thay đổi sau đó.

Kết quả rỗng trong data mẫu là bình thường — giá chưa thay đổi.

Thực tế, câu này quan trọng khi team sửa giá sản phẩm: cần xác nhận đơn cũ đã dùng đúng giá cũ, không phải giá mới bị gán ngược.

Kết quả sau khi query (minh họa)

order_id	product_id	gia_luc_ban	gia_hien_tai	chenh_lech
----------	------------	-------------	--------------	------------

Kết quả rỗng — tất cả items được ghi đúng giá so với Products hiện tại. Câu này nên chạy lại sau mỗi lần cập nhật bảng giá sản phẩm.

GÓC SOI LỖI CỦA TESTER

Kết quả rỗng ở đây là tốt, nhưng cần phân biệt hai tình huống:

(1) **Chênh lệch hợp lý**: giá sản phẩm tăng/giảm sau khi đơn đã đặt — oi.price đúng, p.price là giá mới. Không phải bug.

(2) **Chênh lệch bất thường**: item vừa tạo đã khác giá niêm yết — có thể bug khi tạo đơn hoặc có discount không được ghi nhận đúng cách.

25**Tổng doanh thu chỉ tính đơn hàng COMPLETED****TÌNH HUỐNG**

Doanh thu thật chỉ đến từ đơn đã hoàn tất. Nếu tổng hợp cả đơn CANCELLED hay PENDING, con số sẽ bị thổi phồng. QA cần xác minh hệ thống lọc đúng trạng thái khi tính doanh thu trước khi chốt số báo cáo cuối kỳ.

Bảng Orders — dòng đỏ: ORD_002 có total_amount sai (Bug-B):

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT c.customer_name,
       o.order_id,
       o.total_amount,
       o.order_date
FROM   Orders o
JOIN   Customers c
      ON o.customer_id = c.customer_id
WHERE  o.status = 'COMPLETED'
ORDER BY o.total_amount DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id	INNER JOIN lấy tên khách hàng cho mỗi đơn. ORD_004 (customer_id = C999 không tồn tại) tự động bị loại bởi JOIN này.
WHERE o.status = 'COMPLETED'	Lọc chỉ đơn đã hoàn tất. ORD_003 (CANCELLED) và ORD_004 (PENDING) không được tính vào doanh thu.
ORDER BY o.total_amount DESC	Đơn có giá trị lớn nhất lên đầu — để phát hiện đơn bất thường hoặc Bug-B khi đối chiếu với Câu 11.

Giải thích tổng thể

Thêm **WHERE status = 'COMPLETED'** là điều kiện bắt buộc trong mọi câu báo cáo doanh thu.

Trong data mẫu, tổng total_amount hai đơn COMPLETED = 32M + 20M = 52M.

Nhưng ORD_002 ghi total_amount = 20M trong khi tổng items thực = 31M (Bug-B) — tức 52M này đã thấp hơn thực tế. Đừng vội chốt một con số 'doanh thu thực' ở đây: cần làm sạch dữ liệu lệch (Câu 11) trước khi đưa vào báo cáo.

Kết quả sau khi query (minh họa)

customer_name	order_id	total_amount	order_date
Nguyen Van A	ORD_001	32.000.000	2026-06-20
Tran Van B	ORD_002	20.000.000	2026-06-22

2 đơn COMPLETED, tổng 52M. Con số của ORD_002 (20M) bị sai do Bug-B — chạy Câu 11 để xác nhận trước khi chốt báo cáo.

GÓC SOI LỖI CỦA TESTER

Câu này chỉ lọc theo status — không kiểm tra total_amount có đúng không.

Để có doanh thu chính xác nhất, dùng tổ hợp ba câu:

- (1) **Câu 11**: phát hiện đơn có total_amount lệch với tổng items.
- (2) **Câu 25**: lọc chỉ đơn COMPLETED.
- (3) **Câu 21**: tính lại doanh thu từ items cho những đơn đã xác nhận sạch.

26 Phân tích tỷ lệ đơn hàng theo trạng thái

TÌNH HUỐNG

Tỷ lệ CANCELLED cao là dấu hiệu vấn đề UX, luồng thanh toán, hoặc bug nghiệp vụ. QA cần bảng thống kê trạng thái đơn để theo dõi xu hướng theo sprint và phát hiện khi tỷ lệ một trạng thái thay đổi bất thường.

Bảng Orders — dữ liệu đầu vào để phân tích trạng thái:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT status,
      COUNT(*) AS so_don,
      ROUND(COUNT(*) * 100.0 /
            (SELECT COUNT(*) FROM Orders), 1)
            AS phan_tram
FROM   Orders
GROUP BY status
ORDER BY so_don DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
GROUP BY status	Gom tất cả đơn có cùng trạng thái thành một nhóm. Kết quả là 1 dòng cho mỗi giá trị status tồn tại trong bảng.
COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Orders)	Subquery đếm tổng số đơn. Chia COUNT nhóm cho tổng rồi nhân 100 để ra phần trăm. ROUND(..., 1) làm tròn 1 chữ số thập phân.
ORDER BY so_don DESC	Trạng thái phổ biến nhất lên đầu.

Giải thích tổng thể

Subquery (**SELECT COUNT(*) FROM Orders**) trong SELECT là scalar subquery — trả về một giá trị dùng làm mẫu số cho tất cả các dòng.

Nhân với **100.0** (không phải 100) để ép MySQL dùng phép chia số thực — nếu dùng 100 (số nguyên), kết quả sẽ bị làm tròn xuống 0.

Với data mẫu: COMPLETED = 40%, CANCELLED = 40%, PENDING = 20%.

Ket qua sau khi query (minh hoa)

status	so_don	phan_tram
COMPLETED	2	40.0
CANCELLED	2	40.0
PENDING	1	20.0

40% đơn hoàn tất, 40% bị hủy, 20% chờ xử lý. Với dữ liệu test ít, tỷ lệ này không phản ánh thực tế. Câu này có giá trị thật khi chạy trên dữ liệu production đủ lớn.

GÓC SOI LỖI CỦA TESTER

Hai lưu ý khi dùng câu này trên production:

(1) **Trạng thái lạ**: nếu xuất hiện status không trong danh sách chuẩn, đây là dòng bất thường cần điều tra — kết hợp với Câu 9 (kỹ thuật ENUM check).

(2) **Tỷ lệ thay đổi đột ngột**: CANCELLED tăng từ 10% lên 30% sau một sprint là tín hiệu cần xem lại luồng checkout hoặc thanh toán.

27 Tổng doanh thu theo danh mục sản phẩm**TÌNH HUỐNG**

Quản lý sản phẩm cần biết danh mục nào đóng góp bao nhiêu vào doanh thu để ra quyết định nhập hàng và phân bổ ngân sách. Câu lệnh này tổng hợp từ Order_Items lên level danh mục qua bảng Products.

Bảng Order_Items — dòng đỏ: item_id=7 làm danh mục Diện thoại bị thối phồng:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT p.category,
       COUNT(DISTINCT oi.order_id) AS so_don_co_sp,
       SUM(oi.quantity)           AS tong_so_luong,
       SUM(oi.quantity * oi.price) AS tong_doanh_so
FROM   Order_Items oi
JOIN   Products p
      ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY tong_doanh_so DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items oi JOIN Products p ON oi.product_id = p.product_id	INNER JOIN lấy thông tin category từ Products cho mỗi dòng Order_Items. Sản phẩm chưa bán không xuất hiện — danh mục chỉ tồn tại nếu có đơn.
GROUP BY p.category	Gom tất cả items về cùng danh mục thành một nhóm. Mỗi dòng kết quả = một danh mục.
COUNT(DISTINCT oi.order_id), SUM(oi.quantity * oi.price)	COUNT(DISTINCT order_id) đếm số đơn có ít nhất 1 sản phẩm thuộc danh mục này (không tính trùng). SUM(quantity × price) tính tổng doanh số.
ORDER BY tong_doanh_so DESC	Danh mục doanh số cao nhất lên đầu.

Giải thích tổng thể

Kỹ thuật **JOIN + GROUP BY category** tổng hợp dữ liệu qua hai bảng lên một level cao hơn.
Kết quả bị ảnh hưởng bởi item trùng: 'Dien thoai' cho 90M thay vì 60M thực tế.
COUNT(DISTINCT order_id) thay vì **COUNT(*)**: nếu một đơn có 2 item cùng danh mục, đơn đó chỉ được đếm 1 lần — phản ánh đúng số đơn có mua sản phẩm danh mục này.

Ket qua sau khi query (minh hoa)

category	so_don_co_sp	tong_so_luong	tong_doanh_so
Dien thoai	2	3	90.000.000
Phu kien	4	5	14.000.000

'Dien thoai' dẫn đầu 90M — bị thổi phồng do item 7 trùng. Thực tế là 60M. 'Phu kien' 14M từ 4 đơn (3 sản phẩm PROD_002, 004, 003).

GÓC SOI LỖI CỦA TESTER

Kết quả chỉ có 2 danh mục vì INNER JOIN loại sản phẩm chưa bán.
Nếu muốn xem đầy đủ tất cả danh mục (kể cả chưa có doanh số):
Đổi sang **RIGHT JOIN** Products rồi LEFT JOIN Order_Items — danh mục chưa bán sẽ hiện với tong_doanh_so = NULL.
Cũng cần chạy Câu 2 để làm sạch item trùng trước khi dùng câu này báo cáo.

28

Đối soát tồn kho hiện tại với lượng đã bán ra

TÌNH HUỐNG

Tồn kho hiện tại (stock) + lượng đã bán (SUM items) phải khớp với tồn kho ban đầu. Nếu con số ước tính không hợp lý — âm, quá thấp hoặc quá cao — có thể có giao dịch bị bỏ sót, dữ liệu kho bị sửa thủ công, hoặc bug trong logic trừ kho.

Bảng Products — dòng đỏ: PROD_003 stock âm sẽ cho uoc_tinh_ban_dau bất thường:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       p.stock AS ton_kho_hien_tai,
       COALESCE(SUM(oi.quantity), 0) AS tong_da_ban,
       p.stock
       + COALESCE(SUM(oi.quantity), 0)
       AS uoc_tinh_ban_dau
FROM   Products p
LEFT JOIN Order_Items oi
      ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.stock
ORDER BY p.product_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p LEFT JOIN Order_Items oi ON p.product_id = oi.product_id	LEFT JOIN giữ TẤT CẢ sản phẩm, kể cả chưa bán. Sản phẩm chưa bán sẽ có SUM(oi.quantity) = NULL từ phía Order_Items.
COALESCE(SUM(oi.quantity), 0) AS tong_da_ban	COALESCE chuyển NULL thành 0 cho sản phẩm chưa bán. Không dùng COALESCE thì tong_da_ban = NULL — phép cộng tiếp theo sẽ bị NULL.
p.stock + COALESCE(SUM(oi.quantity), 0) AS uoc_tinh_ban_dau	Ước tính tồn kho ban đầu = tồn hiện tại + đã bán. Nếu con số này âm → có giao dịch khi kho đã âm.
GROUP BY p.product_id, p.product_name, p.stock	p.stock phải có trong GROUP BY vì được dùng trong SELECT nhưng không phải aggregate.

Giải thích tổng thể

Kỹ thuật **LEFT JOIN + COALESCE + aggregate** tạo bảng đối soát kho đầy đủ.
PROD_003: uoc_tinh_ban_dau = -5 + 1 = **-4** — tồn kho ước tính ban đầu âm, vật lý không thể xảy ra → xác nhận dữ liệu tồn kho đã sai ở đâu đó, cần điều tra.
PROD_001: 50 + 3 = 53 — lẽ ra phải là 52 nếu chỉ bán 2 unit, chênh 1 đơn vị chính xác bằng item 7 trùng.

Kết quả sau khi query (minh họa)

product_id	product_name	ton_kho_hien_tai	tong_da_ban	uoc_tinh_ban_dau
PROD_001	iPhone 15 Pro Max	50	3	53
PROD_002	Bàn phím cơ Logitech	100	2	102
PROD_003	Tai nghe Sony WH-1000XM5	-5	1	-4
PROD_004	Sạc du phong Anker	20	2	22
PROD_005	Bàn phím cơ Logitech	30	0	30
PROD_006	Loa Bluetooth JBL	10	0	10
PROD_007	Chuột gaming Razer	(NULL)	0	(NULL)

PROD_003: uoc_tinh_ban_dau = -4 → bất khả thi, xác nhận dữ liệu tồn kho không nhất quán, cần điều tra. PROD_001: 53 thay vì 52 → item trùng thổi phồng thêm 1 đơn vị.

GÓC SOI LỖI CỦA TESTER

COALESCE là hàm quan trọng khi LEFT JOIN: không dùng sẽ cho NULL không tính được.

Hai trường hợp kết quả bất thường cần điều tra thêm:

(1) **uoc_tinh_ban_dau âm**: tồn kho âm là bất khả thi về mặt vật lý — bug dữ liệu gốc cần điều tra.

(2) **uoc_tinh_ban_dau quá cao**: có thể items bị trùng hoặc kho không trừ đúng.

PROD_007 trả về NULL vì stock = NULL — COALESCE chỉ xử lý phía tong_da_ban, không xử lý p.stock.

29**Phát hiện đơn hàng bị tạo trùng (double order)****TÌNH HUỐNG**

Khi người dùng bấm nút Đặt hàng nhiều lần hoặc xảy ra lỗi mạng khiến request gửi hai lần, hệ thống có thể tạo hai đơn giống nhau trong tích tắc. Đây là lỗi double-charge nghiêm trọng — khách bị trừ tiền hai lần cho cùng một đơn mà không hay biết.

Bảng Orders — kiểm tra có cặp đơn nào trùng customer + tổng tiền + ngày không:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT customer_id,
       total_amount,
       order_date,
       COUNT(*) AS so_lan
FROM   Orders
GROUP BY customer_id, total_amount, order_date
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
GROUP BY customer_id, total_amount, order_date	Gom theo bộ ba: cùng khách, cùng tổng tiền, cùng ngày. Nếu một nhóm có nhiều hơn 1 đơn — rất có thể là double order.
HAVING COUNT(*) > 1	Chỉ giữ nhóm xuất hiện từ 2 lần trở lên — dấu hiệu của đơn bị tạo trùng.

Giải thích tổng thể

Kỹ thuật **GROUP BY nhiều cột + HAVING COUNT** — cùng mẫu với Câu 1 và Câu 2, áp dụng cho bảng Orders để phát hiện double submission.

Kết quả rỗng trong data mẫu là bình thường — chưa có đơn trùng.

Trong thực tế, câu này nên chạy sau mỗi đợt stress test hoặc khi khách phản ánh bị trừ tiền hai lần.

Kết quả sau khi query (minh họa)

customer_id	total_amount	order_date	so_lan
-------------	--------------	------------	--------

Kết quả rỗng — không có đơn hàng bị tạo trùng. Chạy lại câu này sau stress test để phát hiện sớm double-charge bug.

GÓC SOI LỖI CỦA TESTER

Gộp theo `order_date` (ngày) có thể bỏ sót double order xảy ra trong cùng ngày nhưng cách nhau vài phút — vẫn là hai đơn hợp lệ.

Nếu DB lưu cả giờ phút, dùng điều kiện chặt hơn:

`TIMESTAMPDIFF(MINUTE, o1.order_date, o2.order_date) < 5` — chỉ bắt hai đơn cách nhau dưới 5 phút mới là double order thật sự.

30

Phát hiện đơn hàng có giá trị bất thường (outlier)

TÌNH HUỐNG

Đơn hàng có `total_amount` quá cao so với mức bình thường có thể do: nhập nhầm số lượng, giá sản phẩm bị set sai, hoặc bug logic tính tiền. Câu lệnh này tìm đơn vượt ngưỡng 1.5× trung bình như một bước kiểm tra nhanh.

Bảng Orders — dòng đỏ: ORD_001 vượt ngưỡng 1.5× trung bình:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       total_amount
FROM   Orders
WHERE  total_amount >
       (SELECT AVG(total_amount) * 1.5
        FROM Orders)
ORDER BY total_amount DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
WHERE total_amount > (SELECT AVG(total_amount) * 1.5 FROM Orders)	Subquery tính trung bình total_amount rồi nhân 1.5 làm ngưỡng. Trong data mẫu: AVG = 16.000.000 → ngưỡng = 24.000.000. Chỉ ORD_001 (32M) vượt ngưỡng này.
ORDER BY total_amount DESC	Đơn có giá trị cao nhất lên đầu để QA xem xét trước.

Giải thích tổng thể

Dùng **subquery trong WHERE** để so sánh từng dòng với một ngưỡng động tính từ chính bảng đó — không cần biết trước ngưỡng là bao nhiêu.

AVG trong data mẫu = 16.000.000 → ngưỡng 1.5× = 24.000.000.

ORD_001 (32M) vượt ngưỡng và được flag. Điều này không có nghĩa là lỗi — đây là điểm khởi đầu để QA

điều tra thêm bằng Câu 11.

Kết quả sau khi query (minh họa)

order_id	customer_id	total_amount
ORD_001	C001	32.000.000

ORD_001 (32M) vượt ngưỡng $1.5 \times$ trung bình (24.000.000). Tổng tiền 32M là hợp lệ (30M + 2M), nhưng doanh thu từ items = 62M do item trùng — đây mới là bất thường thật sự, cần Câu 11 để phát hiện.

GÓC SOI LỖI CỦA TESTER

Ngưỡng $1.5 \times$ trung bình là điểm khởi đầu — điều chỉnh tùy nghiệp vụ:

- (1) Đơn B2B thường có giá trị lớn hơn B2C nhiều lần — nên tách hai nhóm riêng.
- (2) Nếu muốn chặt hơn, dùng $2 \times$ hoặc $3 \times$ trung bình.
- (3) Phương pháp thống kê tốt hơn là dùng **độ lệch chuẩn**: flag đơn vượt trung bình + $2 \times \text{STDDEV}$. MySQL có sẵn **STDDEV()** / **STDDEV_SAMP()** / **STDDEV_POP()** — chỉ là không lồng trực tiếp với AVG trong cùng một HAVING được, nên tính ngưỡng (trung bình + $2 \times$ độ lệch chuẩn) bằng subquery.

BÀI TẬP TỰ LUYỆN — PHẦN 3

Bài 3.1. Tính tổng doanh thu thực (số lượng $\times$ giá) của từng đơn hàng, sắp xếp giảm dần.

Gợi ý: `SUM(quantity * price)` gom theo `GROUP BY order_id`.

Bài 3.2. Mỗi danh mục (category) có bao nhiêu sản phẩm, kể cả sản phẩm chưa bán?

Gợi ý: Đếm trực tiếp trên bảng `Products` (không `JOIN Order_Items`), `GROUP BY category`.

Bài 3.3. Khách hàng nào có tổng chi tiêu cao hơn mức trung bình của các khách đã từng mua?

Gợi ý: Tính tổng mỗi khách, rồi so với AVG của chính các tổng đó (subquery lồng).

→ Lời giải đầy đủ ở **Phụ lục D** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 4 — Biên và dữ liệu bất thường

Người dùng thật luôn nhập những thứ ngoài dự đoán: chuỗi quá dài, ký tự lạ, số âm, ngày tháng vô lý. Đây là nhóm câu lệnh để tìm những outlier mà form nhập liệu lẽ ra phải chặn từ đầu.

31 Tìm tên khách hàng chứa ký tự bất thường

TÌNH HUỐNG
Form nhập liệu thường chỉ chặn ký tự đặc biệt ở frontend. Nếu backend API không validate, tên khách có thể chứa ký tự số, ngoặc, ký tự lạ — gây lỗi khi render, export PDF, hoặc tích hợp hệ thống bên ngoài vốn kỳ vọng dữ liệu sạch.

Bảng Customers — dòng đỏ: tên chứa ký tự ngoài chữ cái thường:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name
FROM   Customers
WHERE  customer_name REGEXP '[0-9()]';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE customer_name REGEXP '[0-9()]'	REGEXP kiểm tra chuỗi theo mẫu biểu thức chính quy. Pattern [0-9()] khớp với bất kỳ ký tự nào là chữ số (0–9) hoặc dấu ngoặc tròn.
SELECT customer_id, customer_name	Chiếu hai cột để QA xác minh và quyết định chuẩn hóa.

Giải thích tổng thể

REGEXP (Regular Expression) là công cụ mạnh để kiểm tra định dạng dữ liệu mà LIKE không làm được. Pattern **[0-9()]** khớp với CHỮ SỐ hoặc DẤU NGOẶC — hai loại ký tự không nên xuất hiện trong tên người thật.

C009 'Nguyen Van A (2)' bị bắt vì có cả '(' ')' và chữ số '2' — đây là tên được nhập để đối phó với ràng buộc UNIQUE, không phải tên thật.

Kết quả sau khi query (minh họa)

customer_id	customer_name
C009	Nguyen Van A (2)

1 bản ghi: C009 có tên chứa ngoặc và số — dấu hiệu dữ liệu test hoặc người dùng cố gắng bypass ràng buộc unique tên.

GÓC SOI LỖI CỦA TESTER

Mở rộng pattern để bắt thêm ký tự đặc biệt khác:

(1) **[!@#\$\$%^&*]**: bắt ký tự đặc biệt thường gặp trong SQL injection.

(2) **[^a-zA-Z]**: chỉ cho phép chữ cái Latin và dấu cách — cẩn thận vì sẽ bắt cả tên tiếng Việt có dấu.

Với dữ liệu tiếng Việt, cách an toàn hơn là **whitelist**: kiểm tra từng dòng thủ công thay vì dùng REGEXP.

32 Tìm email không đúng định dạng cơ bản

TÌNH HUỐNG

Câu 3 đã bắt email NULL và chuỗi rỗng. Câu này đi thêm một bước: kiểm tra email **có nội dung** nhưng thiếu ký tự @ hoặc dấu chấm domain — dấu hiệu nhập sai hoặc bypass validation bằng cách nhập chữ bừa.

Bảng Customers — dòng đỏ: email NULL, rỗng hoặc không có @/dấu chấm:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       email
FROM   Customers
WHERE  email IS NULL
       OR TRIM(email) = ''
       OR email NOT LIKE '%@%'
       OR email NOT LIKE '%.%';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE email IS NULL OR TRIM(email) = ''	Hai điều kiện này trùng với Câu 3 — nhắc lại ở đây để câu lệnh hoạt động độc lập, không cần chạy thêm câu khác.
OR email NOT LIKE '%@%' OR email NOT LIKE '%.%'	NOT LIKE '%@%' : email không chứa ký tự @ — không thể hợp lệ. NOT LIKE '%.%' : không có dấu chấm — thiếu phần domain (vd .com).
SELECT customer_id, customer_name, email	Chiếu đủ thông tin để QA liên hệ xác nhận email thật.

Giải thích tổng thể

Câu này kiểm tra email theo **ba tầng**:

- (1) Tầng tồn tại: NULL hoặc chuỗi rỗng → không có email.
- (2) Tầng cấu trúc: thiếu @ → không thể là email hợp lệ.
- (3) Tầng domain: thiếu dấu chấm → không có phần .com/.vn.

LIKE '%@%.%' không bắt được email như 'abc@' hay '@domain.com' — với dữ liệu nghiêm ngặt, dùng REGEXP thay thế.

Kết quả sau khi query (minh họa)

customer_id	customer_name	email
C006	Pham Van X	(NULL)
C007	Nguyen Thi Y	

C006: email NULL — chưa có. C007: email rỗng — nhập bỏ qua. Trên data mẫu, hai tầng kiểm tra @ và dấu chấm CHƯA bắt thêm dòng nào ngoài NULL/rỗng (các email còn lại đều đúng cấu trúc) — nhưng đó là tuyến phòng thủ quan trọng trên dữ liệu thật khi người dùng nhập email sai cấu trúc.

GÓC SOI LỖI CỦA TESTER

LIKE là kiểm tra nhanh nhưng lỏng lẻo — không bắt được các trường hợp như:

- (1) 'abc@': có @ nhưng không có domain → LIKE '%@%' vẫn pass.
- (2) '@domain.com': thiếu phần local → LIKE cũng pass.

Để kiểm tra chặt hơn, dùng REGEXP:

WHERE email NOT REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$'

33 Kiểm tra số lượng sản phẩm bất thường trong Order_Items**TÌNH HUỐNG**

Cột quantity phải luôn ≥ 1 . Quantity = 0 là đơn trống về số lượng nhưng vẫn tồn tại trong DB — lỗi logic. Quantity âm có thể là bug trừ kho ngược chiều hoặc dữ liệu hoàn trả bị ghi sai bảng. Quantity quá lớn (> 1000) cũng cần xem xét.

Bảng Order_Items — kiểm tra quantity có trong ngưỡng hợp lệ không:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000

item_id	order_id	product_id	quantity	price
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT item_id,
       order_id,
       product_id,
       quantity
FROM   Order_Items
WHERE  quantity <= 0
       OR quantity > 1000;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items .
WHERE quantity <= 0 OR quantity > 1000	Hai điều kiện biên: <= 0 bắt lỗi âm và zero; > 1000 bắt số lượng bất thường lớn (ngưỡng tùy ngành hàng).
SELECT item_id, order_id, product_id, quantity	Chiếu đủ thông tin để QA xác định item nào vi phạm và thuộc đơn nào.

Giải thích tổng thể

Kiểm tra biên dưới (**quantity <= 0**) và biên trên (**quantity > 1000**) là cặp kiểm tra chuẩn cho mọi trường số lượng.

Kết quả rỗng trong data mẫu — tất cả items đều có quantity = 1, hoàn toàn hợp lệ.

Kết quả rỗng ở đây là confirmation tốt: hệ thống chưa có bug quantity trong dữ liệu hiện tại.

Kết quả sau khi query (minh họa)

item_id	order_id	product_id	quantity
---------	----------	------------	----------

Kết quả rỗng — tất cả quantity đều hợp lệ (= 1). Chạy lại sau mỗi lần import hàng loạt hoặc sau sprint có tính năng 'hoàn hàng'.

GÓC SOI LỖI CỦA TESTER

Ngưỡng > 1000 chỉ là ví dụ — điều chỉnh tùy nghiệp vụ:

(1) Bán lẻ (B2C): quantity > 10 đã đáng ngờ.

(2) Bán buôn (B2B): quantity > 10.000 mới bất thường.

Để thêm CHECK constraint vào DB ngăn từ đầu:

ALTER TABLE Order_Items ADD CONSTRAINT chk_qty CHECK (quantity >= 1);

34

Kiểm tra ngày đặt hàng bất thường (tương lai hoặc quá xa quá khứ)

TÌNH HUỐNG

Ngày đặt hàng trong tương lai là dấu hiệu đồng hồ server sai, dữ liệu được nhập thủ công với ngày sai, hoặc bug timezone. Ngày quá xa quá khứ (trước 2020) gợi ý dữ liệu migration bị lỗi hoặc giá trị placeholder chưa được cập nhật.

Bảng Orders — kiểm tra order_date có trong khoảng hợp lý không:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       order_date
FROM   Orders
WHERE  order_date > CURDATE()
       OR order_date < '2020-01-01';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
WHERE order_date > CURDATE() OR order_date < '2020-01-01'	CURDATE() trả về ngày hiện tại của server — không cần hardcode ngày vào câu lệnh. Ngày < 2020-01-01 là ngưỡng tự chọn; điều chỉnh tùy thời điểm hệ thống bắt đầu hoạt động.
SELECT order_id, customer_id, order_date	Chiếu ngày để QA xác minh nguồn gốc dữ liệu.

Giải thích tổng thể

Dùng **CURDATE()** thay vì hardcode ngày giúp câu lệnh luôn đúng mà không cần sửa theo thời gian.

Kết quả rỗng trong data mẫu — tất cả đơn đều có ngày hợp lệ trong tháng 6/2026.

Trên production, câu này nên chạy định kỳ — đặc biệt sau khi deploy tính năng mới có liên quan đến xử lý thời gian hoặc timezone.

Ket qua sau khi query (minh hoa)

order_id	customer_id	order_date
----------	-------------	------------

Kết quả rỗng — tất cả ngày đặt hàng đều hợp lệ. Điều chỉnh ngưỡng '2020-01-01' theo ngày go-live thực tế của hệ thống.

GÓC SOI LỖI CỦA TESTER

Cạm bẫy timezone: CURDATE() trả về ngày theo timezone của MySQL server.

Nếu app chạy ở timezone khác (vd UTC+7), ngày 2026-06-24 23:30 UTC+7 = 2026-06-24 16:30 UTC — cùng ngày nhưng CURDATE() của server UTC trả về 2026-06-24.

Khi nghi ngờ timezone gây lỗi, thêm điều kiện: **TIMESTAMPDIFF(HOUR, order_date, NOW()) < -8** để bắt đơn 'tương lai' trong vòng 8 giờ.

35 Tìm tên sản phẩm trùng sau khi chuẩn hóa**TÌNH HUỐNG**

Câu 8 đã bắt trùng chính xác theo tên và giá. Câu này đi sâu hơn: chuẩn hóa tên về chữ thường và cắt khoảng trắng trước khi so sánh. Bắt được cả các kiểu trùng tinh vi hơn mà UNIQUE constraint case-insensitive chưa chặn được.

Bảng Products — dòng đỏ: tên trùng sau khi LOWER + TRIM:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuat gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT LOWER(TRIM(product_name)) AS ten_chuan,
       COUNT(*)                  AS so_ban_ghi
FROM   Products
GROUP BY LOWER(TRIM(product_name))
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
GROUP BY LOWER(TRIM(product_name))	LOWER chuyển về chữ thường, TRIM cắt khoảng trắng hai đầu. Nhờ vậy 'Ban phim co Logitech' và ' ban phim co logitech ' rơi vào cùng một nhóm.
HAVING COUNT(*) > 1	Chỉ giữ nhóm có nhiều hơn 1 bản ghi — tên bị trùng sau chuẩn hóa.

Giải thích tổng thể

Kỹ thuật **LOWER + TRIM + GROUP BY** là cách chuẩn hóa trước khi so sánh — đã dùng ở Câu 6 (email), áp dụng tương tự cho product_name.

PROD_002 và PROD_005 đều là 'Ban phim co Logitech' → sau LOWER+TRIM cho 'ban phim co logitech' → COUNT = 2.

Câu 8 đã bắt cặp này theo (tên + giá), câu này bắt lại chỉ theo tên — phát hiện trùng ngay cả khi giá đã bị

sửa khác nhau.

Kết quả sau khi query (minh họa)

ten_chuan	so_ban_ghi
ban phim co logitech	2

'ban phim co logitech' xuất hiện 2 lần (PROD_002 + PROD_005). Chạy thêm `SELECT * FROM Products WHERE LOWER(TRIM(product_name)) = 'ban phim co logitech'` để xem chi tiết cả hai dòng.

GÓC SOI LỖI CỦA TESTER

Kết quả câu này chỉ cho tên chuẩn hóa và số lần — không biết product_id nào.

Để xem đầy đủ thông tin hai sản phẩm trùng, kết hợp với subquery:

```
SELECT * FROM Products
WHERE LOWER(TRIM(product_name)) IN
(SELECT LOWER(TRIM(product_name)) FROM Products
GROUP BY LOWER(TRIM(product_name)) HAVING COUNT(*) > 1)
```

36

Phát hiện dữ liệu test/demo còn sót trong dữ liệu thật

TÌNH HUỐNG

Trong lúc dựng môi trường hoặc thử nghiệm tính năng, dev/QA hay tạo tài khoản và bản ghi có tên gợi ý rõ ràng là test ('Test', 'Demo', 'Fake'...). Vấn đề là những bản ghi này đôi khi **quên dọn** trước khi lên production — làm sai lệch số liệu báo cáo, hoặc tệ hơn, biến thành lỗ hổng nếu tài khoản test có quyền cao.

Bảng Customers — dòng đỏ: tên gợi ý dữ liệu test/demo:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       email
FROM   Customers
WHERE  LOWER(customer_name) LIKE '%test%'
       OR LOWER(customer_name) LIKE '%demo%'
       OR LOWER(customer_name) LIKE '%fake%'
       OR LOWER(customer_name) LIKE '%ao bug%'
       OR LOWER(IFNULL(email, '')) LIKE '%test%'
       OR LOWER(IFNULL(email, '')) LIKE '%demo%';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

LOWER(customer_name) LIKE '%test%'	LOWER chuẩn hóa trước khi so khớp — bắt được cả 'Test', 'TEST', 'test'. '%...%' tìm chuỗi con ở bất kỳ vị trí nào trong tên.
LOWER(IFNULL(email, '')) LIKE '%test%'	IFNULL(email, '') thay NULL bằng chuỗi rỗng trước khi LOWER — tránh LOWER(NULL) trả về NULL khiến LIKE luôn UNKNOWN và bỏ sót dòng có email NULL.
OR ... OR ...	Mỗi từ khóa nghi vấn ('test', 'demo', 'fake'...) là một nhánh OR riêng — danh sách từ khóa nên được team thống nhất và cập nhật theo quy ước đặt tên test data thực tế.

Giải thích tổng thể

Kỹ thuật **LIKE '%từ_khóa%'** trên nhiều cột — khác hẳn các câu trùng lặp/ENUM trước đó vì đây là tìm kiếm theo **ngữ nghĩa từ khóa**, không phải so khớp giá trị chính xác.

Bắt được 2 bản ghi: C004 'Khach Hang Ao Bug' (tên tự đặt là dữ liệu giả) và C010 'Khach Test VIP' (có chữ 'Test' rõ ràng) — cả hai đều là tài khoản dựng cho mục đích minh họa trong data mẫu của chính cuốn sách này.

Trên hệ thống thật, danh sách từ khóa cần mở rộng theo quy ước nội bộ: 'qa_', 'sample_', tên miền nội bộ dùng để test (vd '@company-test.com')...

Kết quả sau khi query (minh họa)

customer_id	customer_name	email
C004	Khach Hang Ao Bug	trung_email@email.com
C010	Khach Test VIP	vip@email.com

2 bản ghi nghi là dữ liệu test/demo. Trước khi xóa: xác nhận với team xem có phải seed data cố ý giữ lại cho mục đích nào đó không — đừng xóa chỉ vì tên 'nghe giống' test.

GÓC SOI LỖI CỦA TESTER

LIKE theo từ khóa có hai rủi ro cần lường trước:

(1) **Dương tính giả (false positive)**: khách hàng thật tên trùng từ khóa (vd công ty tên 'Testco') sẽ bị bắt nhầm — luôn xác nhận thủ công trước khi xóa hàng loạt.

(2) **Âm tính giả (false negative)**: dữ liệu test đặt tên khéo léo không chứa từ khóa nào sẽ lọt qua — đây là giới hạn cố hữu của tìm kiếm theo từ khóa, không phải lỗi kỹ thuật.

Phòng ngừa tận gốc: thêm cột **is_test_data BOOLEAN** để đánh dấu tường minh ngay từ lúc tạo, thay vì suy luận ngược từ tên gọi.

37

Dò bản ghi gần-trùng bằng SOUNDEX (lỗi gõ chính tả)

TÌNH HUỐNG

Câu 6 và Câu 35 bắt trùng nhờ chuẩn hóa hoa/thường và khoảng trắng — nhưng cả hai đều cần chuỗi gốc **giống hệt nhau** sau khi chuẩn hóa. Lỗi gõ chính tả thật ('Nguyen Van A' gõ thành 'Nguyen Van Ah' hay đọc sai dấu) tạo ra chuỗi khác hẳn về mặt ký tự dù phát âm gần như nhau — LOWER/TRIM không bắt được loại trùng này.

Bảng Customers — dòng đồ: hai tên phát âm giống nhau (SOUNDEX trùng):

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT a.customer_id AS id_a,
       a.customer_name AS ten_a,
       b.customer_id AS id_b,
       b.customer_name AS ten_b
FROM   Customers a
JOIN   Customers b
       ON SOUNDEX(a.customer_name) = SOUNDEX(b.customer_name)
      AND a.customer_id < b.customer_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

SOUNDEX(a.customer_name) = SOUNDEX(b.customer_name)	SOUNDEX chuyển chuỗi thành mã 4 ký tự đại diện cho cách phát âm — hai chuỗi viết khác nhau nhưng đọc gần giống sẽ cho cùng mã.
FROM Customers a JOIN Customers b	Self-join : bảng tự ghép với chính nó để so sánh từng cặp bản ghi. Vì điều kiện JOIN bọc trong hàm SOUNDEX(), MySQL không dùng được index — đây là phép so khớp toàn bảng × toàn bảng, xem cảnh báo hiệu năng ở phần Ghi chú.
AND a.customer_id < b.customer_id	Điều kiện chống trùng cặp: nếu không có dòng này, mỗi cặp sẽ xuất hiện 2 lần (A-B và B-A) và mỗi dòng còn tự khớp với chính nó.

Giải thích tổng thể

SOUNDEX là kỹ thuật **fuzzy matching** — khác hẳn so khớp chính xác (Câu 1, 8) hay chuẩn hóa hoa/thường-khoảng trắng (Câu 6, 35): nó so khớp theo **âm đọc**, không theo ký tự.

C001 'Nguyen Van A' và C009 'Nguyen Van A (2)' cho cùng mã SOUNDEX vì SOUNDEX bỏ qua khoảng trắng và ký tự không phải chữ cái khi mã hóa — cùng một cặp trùng mà Câu 31 (regex ký tự lạ) đã phát hiện, nay được xác nhận bằng một kỹ thuật hoàn toàn khác.

SOUNDEX được thiết kế gốc cho tiếng Anh nên kém chính xác với tên có dấu tiếng Việt — hữu ích nhất khi dữ liệu đã được chuyển về không dấu hoặc cho các trường mã/tên tiếng Anh.

Ket qua sau khi query (minh hoa)

id_a	ten_a	id_b	ten_b
C001	Nguyen Van A	C009	Nguyen Van A (2)

1 cặp phát âm giống nhau. Đây là cùng cặp Câu 31 đã phát hiện qua dấu hiệu khác (ký tự số/ngoặc trong tên) — SOUNDEX hữu ích nhất khi lỗi gõ không để lại dấu hiệu ký tự rõ ràng như vậy, ví dụ 'Nguyen Van A' bị gõ nhầm thành 'Nguyenn Van A'.

GÓC SOI LỖI CỦA TESTER

SOUNDEX chỉ là một trong các thuật toán fuzzy matching — hạn chế cần biết trước khi dùng:

(1) Độ nhạy thấp với tiếng Việt có dấu — cân nhắc bỏ dấu (UNACCENT/REPLACE thủ công) trước khi SOUNDEX để tăng độ chính xác.

(2) Dễ cho **dương tính giả** với tên ngắn — luôn xác nhận thủ công, không tự động xóa.

(3) Hệ quản trị khác có hàm tương đương: PostgreSQL có **SOUNDEX** qua extension fuzzystrmatch (kèm LEVENSHTEIN đo khoảng cách chỉnh sửa), SQL Server có **DIFFERENCE()**.

(4) **CẢNH BÁO HIỆU NĂNG — không chạy nguyên văn câu này trên bảng lớn**: đây là self-join với điều kiện bọc hàm, không index nào hỗ trợ được — EXPLAIN cho thấy MySQL phải quét toàn bảng cho từng dòng (chi phí xấp xỉ N^2). Với 10 dòng demo là tức thì; với bảng khách hàng thật hàng trăm nghìn dòng, câu này có thể chạy hàng giờ hoặc làm nghẽn DB đang phục vụ giao dịch. Chạy an toàn theo một trong các cách:

— Lọc trước bằng **WHERE** để thu hẹp tập so sánh (vd theo khu vực, theo ngày tạo gần đây) thay vì so toàn bảng với toàn bảng.

— Nếu cần chạy định kỳ, thêm cột tính sẵn **soundex_name** (cập nhật lúc ghi dữ liệu) và đánh index trên cột đó — khi ấy JOIN so sánh giá trị thường, không còn bọc hàm.

— Luôn chạy trên **replica/read-only** (xem chương "Chạy SQL an toàn trên production"), không chạy trực tiếp trên DB giao dịch chính.

38 Phát hiện đơn có tổng tiền nhưng không có sản phẩm nào

TÌNH HUỐNG

Câu 15 đã tìm đơn rỗng (không có items). Câu này thêm điều kiện: đơn có **total_amount > 0** nhưng lại không có item nào — nghĩa là hệ thống đã thu tiền (hoặc ghi nhận số tiền) cho một đơn không có sản phẩm cụ thể. Đây là mâu thuẫn dữ liệu nghiêm trọng.

Bảng Orders — dòng đỏ: ORD_004 ghi 5M nhưng Order_Items không có dòng nào:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id,
       o.total_amount,
       o.status
FROM   Orders o
LEFT JOIN Order_Items oi
      ON o.order_id = oi.order_id
WHERE  o.total_amount > 0
AND    oi.order_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o LEFT JOIN Order_Items oi ON o.order_id = oi.order_id	LEFT JOIN giữ tất cả đơn hàng. Đơn không có item sẽ có oi.order_id = NULL sau LEFT JOIN.
WHERE o.total_amount > 0 AND oi.order_id IS NULL	Kết hợp hai điều kiện: total_amount > 0 (có ghi nhận tiền) VÀ oi.order_id IS NULL (không có items). Mâu thuẫn này là bug nghiêm trọng.
SELECT o.order_id, o.customer_id, o.total_amount, o.status	Chiều đủ thông tin để QA điều tra: bao nhiêu tiền, trạng thái gì.

Giải thích tổng thể

Câu 15 bắt **mọi đơn rỗng** (kể cả total_amount = 0). Câu 38 chỉ bắt đơn rỗng **nhưng có tiền** — trường hợp nghiêm trọng hơn.

ORD_004 bị phát hiện: total_amount = 5M nhưng không có item nào trong Order_Items.

ORD_004 còn có thêm bug customer_id = C999 không tồn tại (Câu 5) — một đơn tích lũy nhiều lỗi chồng nhau.

Ket qua sau khi query (minh hoa)

order_id	customer_id	total_amount	status
ORD_004	C999	5.000.000	PENDING

ORD_004: 5M nhưng không có sản phẩm nào. Thêm vào đó: C999 không tồn tại. Đây là đơn hàng 'ma' — cần điều tra log xem được tạo như thế nào.

GÓC SOI LỖI CỦA TESTER

ORD_004 là ví dụ điển hình của **bug chồng bug**:

(1) Câu 5: customer_id = C999 không có trong Customers.

(2) Câu 15: không có item nào trong Order_Items.

(3) Câu 38: có total_amount = 5M nhưng không có gì để tính.

Khi tìm thấy loại đơn này, ưu tiên kiểm tra access log và payment log để xác định có transaction thật không.

39

Phát hiện tổng items vượt quá 1.5 lần total_amount

TÌNH HUỐNG

Câu 11 bắt mọi đơn có tổng items khác total_amount dù chỉ 1 đồng. Câu này dùng ngưỡng 1.5× để bắt những trường hợp **lệch bất thường lớn**: tổng items cao hơn total_amount gấp rưỡi là dấu hiệu dữ liệu sai ở mức độ không thể do làm tròn hay phí vận chuyển.

Bảng Orders — dòng đỏ: total_amount bất thường so với tổng items:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.total_amount,
       SUM(oi.quantity * oi.price) AS tinh_tu_items
FROM   Orders o
JOIN   Order_Items oi
      ON o.order_id = oi.order_id
GROUP BY o.order_id, o.total_amount
HAVING SUM(oi.quantity * oi.price)
       > o.total_amount * 1.5;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Order_Items oi ON o.order_id = oi.order_id	INNER JOIN ghép đơn hàng với items. Đơn không có item (ORD_004) bị loại — dùng Câu 38 để phát hiện riêng trường hợp đó.
GROUP BY o.order_id, o.total_amount	Gom tất cả items của cùng một đơn để tính tổng.
HAVING SUM(oi.quantity * oi.price) > o.total_amount * 1.5	HAVING so sánh sau aggregate: tổng items > 1.5 lần total_amount là bất thường. Ngưỡng 1.5× để bỏ qua chênh lệch nhỏ do phí vận chuyển, discount.

Giải thích tổng thể

Câu 11 (= ANY difference) vs Câu 39 (> 1.5×): hai mức độ kiểm tra khác nhau.
ORD_001: tinh_tu_items = 62M, total_amount = 32M > 48M → bị bắt.
ORD_002: tinh_tu_items = 31M, total_amount = 20M > 30M → bị bắt.
Cả hai đều bị bắt: ORD_001 do item trùng, ORD_002 do Bug-B (total_amount bị ghi thấp).

Kết quả sau khi query (minh họa)

order_id	total_amount	tinh_tu_items
ORD_001	32.000.000	62.000.000
ORD_002	20.000.000	31.000.000

2 đơn vượt ngưỡng 1.5×. ORD_001: 62M vs 32M (do item trùng). ORD_002: 31M vs 20M (do Bug-B). Hai nguyên nhân khác nhau, cùng câu phát hiện.

GÓC SOI LỖI CỦA TESTER

Điều chỉnh ngưỡng tùy ngữ cảnh nghiệp vụ:

- (1) > 1.5×: bắt lệch lớn, bỏ qua discount/phí nhỏ (câu này).
- (2) != (bất kỳ lệch): bắt tất cả sai lệch dù nhỏ (Câu 11).
- (3) > 2×: chỉ bắt lệch nghiêm trọng nhất.

Với hệ thống có discount, nên kết hợp thêm điều kiện loại trừ đơn có coupon trước khi áp ngưỡng.

BÀI TẬP TỰ LUYỆN — PHẦN 4

Bài 4.1. Tìm các sản phẩm bị thiếu dữ liệu giá HOẶC tồn kho (giá trị NULL).

Gợi ý: *WHERE price IS NULL OR stock IS NULL* — KHÔNG dùng *'= NULL'*.

Bài 4.2. Tìm khách hàng có tên chứa chữ số.

Gợi ý: *REGEXP '[0-9]'* bắt mọi ký tự là chữ số trong tên.

Bài 4.3. Tìm các tên sản phẩm bị trùng sau khi chuẩn hóa (bỏ khoảng trắng + đưa về chữ thường).

Gợi ý: *GROUP BY LOWER(TRIM(product_name))* rồi *HAVING COUNT(*) > 1*.

→ Lời giải đầy đủ ở **Phụ lục D** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 5 — Audit, log và dấu vết

Cột thời gian và mối quan hệ giữa các bảng là nơi kể lại lịch sử dữ liệu. Khi chúng mâu thuẫn — sản phẩm bị xóa vẫn còn trong đơn, khách hàng không tồn tại vẫn có giao dịch — đó là dấu hiệu của bug logic hoặc lỗi hỏng dữ liệu.

40 Truy vết item còn sót của đơn đã xóa mềm

TÌNH HUỐNG

Câu 10 đã phát hiện đơn ORD_005 bị **xóa mềm** (cột deleted_at có giá trị) nhưng vẫn nằm trong bảng Orders. Vấn đề chưa dừng ở đó: các dòng chi tiết của đơn này trong Order_Items (item 8, 9) **không hề bị dọn**. Khi báo cáo doanh thu hoặc tồn kho quên lọc đơn đã xóa mềm, những item này vẫn bị cộng vào.

Bảng Order_Items — dòng đỏ: item 8, 9 thuộc ORD_005 (đơn đã xóa mềm):

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT oi.item_id,
       oi.order_id,
       oi.product_id,
       oi.quantity,
       oi.price
FROM   Order_Items oi
JOIN   Orders o
      ON oi.order_id = o.order_id
WHERE  o.deleted_at IS NOT NULL
ORDER BY oi.item_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items oi JOIN Orders o ON oi.order_id = o.order_id	INNER JOIN ghép mỗi dòng item với đơn cha của nó để đọc được trạng thái xóa mềm từ bảng Orders.
WHERE o.deleted_at IS NOT NULL	Chỉ giữ item thuộc đơn đã bị xóa mềm — deleted_at có giá trị nghĩa là đơn lẽ ra không còn hiệu lực.

ORDER BY oi.item_id

Sắp xếp theo item_id để dễ đối chiếu với bảng gốc.

Giải thích tổng thể

Câu 10 soi **bảng cha** (Orders) để tìm đơn xóa mềm; câu này đi tiếp xuống **bảng con** (Order_Items) để tìm những dòng bị bỏ lại.

Đây là mẫu kiểm tra **tính nhất quán khi xóa mềm**: xóa cha thì con phải được đánh dấu hoặc loại khỏi mọi phép tính.

item 8 và 9 thuộc ORD_005 — vẫn tồn tại dù đơn đã bị hủy và xóa mềm.

Ket qua sau khi query (minh họa)

item_id	order_id	product_id	quantity	price
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

2 item thuộc đơn đã xóa mềm vẫn nằm trong Order_Items. Mọi báo cáo tính từ Order_Items phải thêm điều kiện loại đơn có deleted_at, nếu không doanh thu và lượng bán sẽ bị thổi phồng.

GÓC SOI LỖI CỦA TESTER

Cách phòng và soát lỗi này:

- (1) **Lọc nhất quán**: mọi truy vấn báo cáo nên JOIN sang Orders và thêm **WHERE deleted_at IS NULL**.
- (2) **Dọn theo tầng**: khi xóa mềm đơn, cần nhắc đánh dấu cả item con.
- (3) **Đối soát định kỳ**: chạy câu này theo lịch để phát hiện sớm item mồ côi sau mỗi đợt xóa.

41**Đối chiếu trạng thái hủy với cờ xóa mềm****TÌNH HUỐNG**

Hệ thống dùng hai dấu vết để đánh dấu một đơn không còn hiệu lực: cột **status = 'CANCELLED'** và cột **deleted_at** (thời điểm xóa mềm). Theo quy ước, hai cột này phải đi cùng nhau. Khi chúng lệch nhau — hủy nhưng chưa xóa mềm, hoặc xóa mềm nhưng status chưa đổi — quy trình đã bỏ sót một bước.

Bảng Orders — dòng đỏ: ORD_003 hủy nhưng deleted_at vẫn trống:

order_id	status	deleted_at
ORD_001	COMPLETED	(NULL)
ORD_002	COMPLETED	(NULL)
ORD_003	CANCELLED	(NULL)
ORD_004	PENDING	(NULL)
ORD_005	CANCELLED	2026-06-25 10:30:00

CÂU LỆNH SQL

```
SELECT order_id,
       status,
       deleted_at,
       CASE
         WHEN status = 'CANCELLED'
           AND deleted_at IS NULL
         THEN 'Hủy nhưng chưa xóa mềm'
         WHEN status <> 'CANCELLED'
           AND deleted_at IS NOT NULL
         THEN 'Xóa mềm nhưng status chưa CANCELLED'
       END AS van_de
FROM   Orders
WHERE  (status = 'CANCELLED' AND deleted_at IS NULL)
       OR (status <> 'CANCELLED' AND deleted_at IS NOT NULL)
ORDER BY order_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

CASE WHEN ... THEN ... END AS van_de	CASE gán nhãn mô tả loại lệch cho từng dòng — giúp QA đọc kết quả là biết ngay vi phạm kiểu nào.
WHERE (status = 'CANCELLED' AND deleted_at IS NULL) OR (status <> 'CANCELLED' AND deleted_at IS NOT NULL)	Hai vế OR bắt hai chiều lệch: hủy mà chưa xóa mềm, và xóa mềm mà status chưa chuyển CANCELLED.
ORDER BY order_id	Sắp xếp theo mã đơn cho dễ tra.

Giải thích tổng thể

Khi một sự kiện nghiệp vụ được ghi ở **nhiều cột dấu vết**, các cột đó phải luôn đồng bộ. Câu này kiểm tra sự đồng bộ giữa status và deleted_at.

ORD_003 đã CANCELLED nhưng deleted_at vẫn trống — bước xóa mềm bị bỏ quên.

ORD_005 thì chuẩn: vừa CANCELLED vừa có deleted_at, nên không bị bắt.

Kết quả sau khi query (minh họa)

order_id	status	deleted_at	van_de
ORD_003	CANCELLED	(NULL)	Hủy nhưng chưa xóa mềm

ORD_003 hủy nhưng chưa được xóa mềm — nếu báo cáo lọc theo deleted_at, đơn này vẫn lọt vào như đơn còn hiệu lực. Cần đồng bộ lại hai cột.

GÓC SOI LỖI CỦA TESTER

Mở rộng kiểm tra đồng bộ trạng thái:

- Thêm cặp cột khác: **paid_at** với status = 'PAID', **shipped_at** với status = 'SHIPPED'.
- Đưa câu này vào bộ kiểm tra định kỳ sau mỗi lần đổi luồng trạng thái.
- Gốc rễ thường nằm ở code: nên cập nhật status và deleted_at trong cùng một transaction.

42

Phát hiện đơn treo (PENDING) tồn đọng quá lâu

TÌNH HUỐNG

Một đơn ở trạng thái **PENDING** vài giờ là bình thường. Nhưng nếu nó treo nhiều ngày, rất có thể luồng thanh toán bị kẹt, job xử lý đã chết, hoặc đơn bị bỏ quên. Câu này đo số ngày tồn đọng tính đến một **mốc chốt sổ cố định** (2026-06-30) để kết quả ổn định, không đổi theo ngày chạy.

Bảng Orders — dòng đỏ: ORD_004 vẫn PENDING từ 2026-06-24:

order_id	customer_id	status	order_date
ORD_001	C001	COMPLETED	2026-06-20
ORD_002	C002	COMPLETED	2026-06-22
ORD_003	C003	CANCELLED	2026-06-23
ORD_004	C999	PENDING	2026-06-24
ORD_005	C001	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       status,
       order_date,
       DATEDIFF('2026-06-30', order_date)
         AS so_ngay_ton_dong
FROM   Orders
WHERE  status = 'PENDING'
AND    DATEDIFF('2026-06-30', order_date) > 3
ORDER BY so_ngay_ton_dong DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

DATEDIFF('2026-06-30', order_date) AS so_ngay_ton_dong	DATEDIFF tính số ngày giữa mốc chốt sổ và ngày đặt đơn — chính là tuổi của đơn. Dùng ngày cố định để kết quả không trôi theo thời gian.
WHERE status = 'PENDING' AND DATEDIFF('2026-06-30', order_date) > 3	Lọc đơn còn PENDING và đã treo quá 3 ngày . Ngưỡng 3 ngày tùy SLA của hệ thống. Lưu ý: DATEDIFF() bọc quanh order_date khiến điều kiện này non-sargable — xem cách viết thay thế tận dụng được index ở phần Ghi chú.
ORDER BY so_ngay_ton_dong DESC	Đơn treo lâu nhất lên đầu — ưu tiên xử lý trước.

Giải thích tổng thể

Câu 34 bắt ngày **vô lý** (tương lai hoặc quá xa quá khứ); câu này bắt đơn **tồn đọng** — ngày hợp lệ nhưng trạng thái mắc kẹt quá lâu.

ORD_004 đặt 2026-06-24, tính đến mốc 2026-06-30 là 6 ngày vẫn PENDING.

Trên production thường thay 2026-06-30 bằng **CURDATE()** để đo theo thời gian thực.

Kết quả sau khi query (minh họa)

order_id	customer_id	status	order_date	so_ngay_ton_dong
ORD_004	C999	PENDING	2026-06-24	6

ORD_004 treo 6 ngày chưa xử lý — lại đúng đơn có customer_id C999 không tồn tại (Câu 5). Đơn treo lâu là nơi nên soi kỹ vì thường đi kèm lỗi khác.

GÓC SOI LỖI CỦA TESTER

Tinh chỉnh câu này theo nghiệp vụ:

- (1) Đổi mốc cố định thành **CURDATE()** khi chạy giám sát thực tế.
- (2) Điều chỉnh ngưỡng ngày theo SLA: thanh toán có thể là vài giờ, giao hàng có thể là vài ngày.
- (3) Áp dụng cho các trạng thái 'mắc kẹt' khác: PROCESSING, AWAITING_PAYMENT.
- (4) **Hiệu năng trên bảng lớn**: khác với Câu 34 (so sánh trực tiếp order_date, tận dụng được index nếu có), **DATEDIFF(...) > 3** bọc hàm quanh cột nên non-sargable — dù order_date có được đánh index, MySQL vẫn phải tính DATEDIFF cho từng dòng thay vì dùng index range scan. Trên bảng lớn, viết lại dạng sargable cho cùng kết quả:

WHERE status = 'PENDING'

AND order_date < DATE_SUB('2026-06-30', INTERVAL 3 DAY);

43 Dựng dòng thời gian đơn hàng — khoảng cách giữa các đơn

TÌNH HUỐNG

Một chuỗi đơn hàng đều đặn là dấu hiệu hệ thống chạy bình thường. Khi xuất hiện **khoảng lặng dài** (nhiều ngày không có đơn) hoặc **cụm dày đặc** (nhiều đơn trong tích tắc), đó là dấu vết đáng soi: job tạo đơn chết, hoặc bot đặt hàng hàng loạt. Câu này đo số ngày giữa mỗi đơn và đơn liền trước nó.

Bảng Orders — dòng thời gian 5 đơn theo order_date:

order_id	order_date
ORD_001	2026-06-20
ORD_002	2026-06-22
ORD_003	2026-06-23
ORD_004	2026-06-24
ORD_005	2026-06-25

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.order_date,
       DATEDIFF(
         o.order_date,
         (SELECT MAX(o2.order_date)
          FROM Orders o2
          WHERE o2.order_date < o.order_date)
       ) AS ngay_ke_tu_don_truoc
FROM   Orders o
ORDER BY o.order_date;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

```
(SELECT MAX(o2.order_date)
 FROM Orders o2
 WHERE o2.order_date
       < o.order_date)
```

Subquery tương quan: với mỗi đơn, tìm ngày đặt lớn nhất trong số các đơn **trước nó** — chính là đơn liền trước. Đây là correlated subquery (EXPLAIN gắn nhãn **dependent**) — MySQL chạy lại subquery này cho **TỪNG** dòng outer, không phải 1 lần — xem cảnh báo hiệu năng ở phần Ghi chú.

DATEDIFF(o.order_date, ...) AS ngay_ke_tu_don_truoc	Lấy hiệu số ngày giữa đơn hiện tại và đơn liền trước. Đơn đầu tiên không có đơn nào trước nên trả về NULL.
ORDER BY o.order_date	Sắp theo thời gian để đọc như một dòng sự kiện.

Giải thích tổng thể

Đây là cách dựng dòng thời gian **không dùng window function** — phù hợp cả MySQL phiên bản cũ. Subquery tương quan đóng vai trò như LAG() nhưng viết bằng subquery.

Câu 49 ở PHẦN 6 sẽ làm việc tương tự gọn hơn bằng window function.

Khoảng cách đều 1–2 ngày ở data mẫu là bình thường; trên thực tế, một con số đột biến mới là dấu hiệu cần điều tra.

Kết quả sau khi query (minh họa)

order_id	order_date	ngay_ke_tu_don_truoc
ORD_001	2026-06-20	(NULL)
ORD_002	2026-06-22	2
ORD_003	2026-06-23	1
ORD_004	2026-06-24	1
ORD_005	2026-06-25	1

Dòng thời gian liền mạch: cách nhau 1–2 ngày. Đơn đầu tiên ORD_001 có khoảng cách NULL vì không có đơn nào trước. Nếu thấy khoảng cách bất thường (vd 30 ngày, hay nhiều đơn cách 0 ngày), đó mới là điểm cần soi.

GÓC SOI LỖI CỦA TESTER

Ứng dụng dòng thời gian trong kiểm thử:

- (1) **Khoảng lặng dài**: hệ thống ngừng nhận đơn — job hoặc API có thể đã chết.
- (2) **Cụm 0 ngày dày đặc**: nhiều đơn cùng lúc — nghi vấn bot hoặc double-submit.
- (3) Thêm điều kiện trên khoảng cách để chỉ liệt kê các điểm vượt ngưỡng.
- (4) **CẢNH BÁO HIỆU NĂNG**: subquery tương quan ở đây chạy lại có table scan cho mỗi dòng outer — chi phí xấp xỉ N^2 giống hết mức rủi ro ở Câu 37 (SOUNDEX self-join). Với vài nghìn đơn vẫn ổn; với bảng Orders production hàng triệu dòng, cách viết này có thể chạy rất lâu hoặc nghẽn DB. Khi cần chạy thật trên dữ liệu lớn, dùng **LAG(order_date) OVER (ORDER BY order_date)** (window function, MySQL 8.0+, xem Câu 49) — cùng kết quả nhưng chỉ quét bảng một lần. Cách viết subquery ở Câu 43 này chỉ nên dùng để minh họa logic hoặc khi chạy trên MySQL cũ chưa có window function.

44

Phát hiện item_id bị nhảy — dấu vết của bản ghi bị xóa

TÌNH HUỐNG

item_id là khóa chính tự tăng (AUTO_INCREMENT). Nếu dãy số bị nhảy — ví dụ tồn tại 1, 2, 4 nhưng không có 3 — có nghĩa là một bản ghi đã từng tồn tại và bị xóa. Đây là kỹ thuật audit đơn giản để phát hiện dữ liệu bị xóa không có log.

Bảng Order_Items — item_id=3 bị thiếu trong dãy 1→9:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000

item_id	order_id	product_id	quantity	price
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT a.item_id + 1      AS id_bi_mat
FROM   Order_Items a
LEFT JOIN Order_Items b
      ON b.item_id = a.item_id + 1
WHERE  b.item_id IS NULL
AND    a.item_id <
      (SELECT MAX(item_id)
       FROM Order_Items);
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items a LEFT JOIN Order_Items b ON b.item_id = a.item_id + 1	Self-join: bảng tự join với chính nó. Alias a là bản ghi gốc, b là bản ghi tiếp theo liền kề. Nếu b.item_id = NULL → không có bản ghi nào có id = a.item_id + 1.
WHERE b.item_id IS NULL AND a.item_id < (SELECT MAX(item_id) FROM Order_Items)	Hai điều kiện kết hợp: tiếp theo bị thiếu (IS NULL) VÀ chưa phải bản ghi cuối (< MAX). Bỏ điều kiện MAX sẽ trả thêm id = MAX+1, không phải gap.
SELECT a.item_id + 1 AS id_bi_mat	Kết quả là id bị thiếu — tức là a.item_id + 1.

Giải thích tổng thể

Self-join là kỹ thuật dùng một bảng join với chính nó để phát hiện mối quan hệ giữa các dòng trong cùng bảng.

Logic: với mỗi item_id=n, kiểm tra xem n+1 có tồn tại không. Nếu không → n+1 là gap.

item_id=2 tồn tại nhưng item_id=3 không → trả về id_bi_mat=3.

item_id=9 là MAX nên không kiểm tra tiếp — 10 không phải gap, chỉ là chưa có.

Kết quả sau khi query (minh họa)

id_bi_mat
3

item_id=3 bị thiếu trong dãy 1→7. Một item đã từng tồn tại và bị xóa, hoặc INSERT thất bại và AUTO_INCREMENT vẫn tiếp tục tăng.

GÓC SOI LỖI CỦA TESTER

Gap trong AUTO_INCREMENT không phải lúc nào cũng là bug:

- (1) **INSERT thất bại:** transaction rollback, nhưng AUTO_INCREMENT không rollback lại — tạo gap bình thường.
- (2) **Xóa dữ liệu:** hard DELETE không để lại dấu vết gì khác.
- (3) **Bulk insert lỗi giữa chừng:** một số ID đã được cấp nhưng bản ghi tương ứng không được lưu. Để biết nguyên nhân chính xác, cần có bảng audit log riêng.

45

Phân tích đơn hàng bị hủy — khách nào hay hủy nhất

TÌNH HUỐNG

Đơn bị hủy không chỉ là vấn đề kinh doanh — còn là tín hiệu kỹ thuật. Nếu một khách hàng cụ thể có tỷ lệ hủy cao bất thường, có thể luồng thanh toán gặp lỗi chỉ với profile đó, hoặc dữ liệu test chưa được dọn sạch sau sprint.

Bảng Orders — dòng đỏ: ORD_003 có status = CANCELLED:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name,
       COUNT(o.order_id) AS so_don_huy
FROM   Orders o
JOIN   Customers c
       ON o.customer_id = c.customer_id
WHERE  o.status = 'CANCELLED'
GROUP BY c.customer_id, c.customer_name
ORDER BY so_don_huy DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id	INNER JOIN: chỉ ghép đơn có customer_id hợp lệ. ORD_004 (C999) bị loại khỏi kết quả vì C999 không có trong Customers.
WHERE o.status = 'CANCELLED'	Lọc chỉ giữ đơn bị hủy trước khi group.
GROUP BY c.customer_id, c.customer_name ORDER BY so_don_huy DESC	Gom theo khách hàng và sắp xếp giảm dần — khách hủy nhiều nhất lên đầu.

Giải thích tổng thể

Kỹ thuật **WHERE** → **GROUP BY** → **ORDER BY** là chuỗi chuẩn để phân tích theo nhóm:

- (1) WHERE lọc loại đơn cần phân tích trước.
- (2) GROUP BY gom về từng khách hàng.
- (3) ORDER BY DESC đặt trường hợp đáng ngờ nhất lên đầu.

Data mẫu nhỏ nên chỉ có C001 và C003 có đơn bị hủy — trên production, câu này hiệu quả nhất khi chạy kèm HAVING COUNT(*) > N để lọc noise.

Ket qua sau khi query (minh họa)

customer_id	customer_name	so_don_huy
C003	Le Thi C	1
C001	Nguyen Van A	1

C001 và C003 mỗi khách hủy 1 đơn (C001 hủy ORD_005, C003 hủy ORD_003). Câu này giúp theo dõi và cảnh báo sớm nếu tỷ lệ hủy của khách tăng đột biến.

GÓC SOI LỖI CỦA TESTER

Mở rộng câu này để phân tích sâu hơn:

- (1) Thêm **HAVING COUNT(*) > 2**: chỉ hiện khách hủy nhiều bất thường.
- (2) Thêm **tỷ lệ**: số đơn hủy / tổng số đơn của khách — khách có 1 đơn và hủy 1 đơn = 100% hủy, đáng ngờ hơn khách có 10 đơn hủy 1.
- (3) Kết hợp **order_date**: xem các đơn hủy có tập trung trong một khoảng thời gian cụ thể không — có thể liên quan đến release lỗi.

BÀI TẬP TỰ LUYỆN — PHẦN 5

Bài 5.1. Dùng NOT EXISTS để liệt kê sản phẩm chưa bao giờ xuất hiện trong Order_Items.

Gợi ý: *WHERE NOT EXISTS (SELECT 1 FROM Order_Items WHERE product_id khớp).*

Bài 5.2. Dùng NOT EXISTS tìm đơn hàng trở tới khách hàng không tồn tại trong Customers.

Gợi ý: *Orders mà NOT EXISTS một khách có customer_id tương ứng.*

→ Lời giải đầy đủ ở **Phụ lục D** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 6 — Truy vấn nâng cao cho QA

Năm câu lệnh cuối dùng tới window function, CTE và UNION. Chúng giải quyết những bài toán kiểm thử khó: lấy bản ghi mới nhất mỗi nhóm, phân hạng khách hàng, tổng hợp nhiều loại lỗi trong một báo cáo duy nhất.

46

Dùng ROW_NUMBER() phát hiện item trùng trong cùng một đơn

TÌNH HUỐNG

Câu 2 đã phát hiện item trùng bằng GROUP BY + HAVING COUNT(*) > 1 — cho biết nhóm nào bị trùng nhưng không chỉ ra dòng nào là bản gốc, dòng nào là bản thừa. Câu này dùng **window function ROW_NUMBER()** để đánh số thứ tự từng item trong cùng nhóm (order_id + product_id). Bất kỳ item nào có số thứ tự > 1 là bản ghi trùng — và ta có thể thấy chính xác item nào là lần xuất hiện đầu tiên, lần nào là trùng.

Bảng Order_Items — dòng đỏ: item 1 và item 7 cùng ORD_001/PROD_001:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT item_id,
       order_id,
       product_id,
       ROW_NUMBER() OVER (
         PARTITION BY order_id, product_id
         ORDER BY item_id
       ) AS so_lan_trong_don
FROM   Order_Items
ORDER BY order_id, product_id, item_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items

MySQL tải toàn bộ bảng Order_Items.

<pre>ROW_NUMBER() OVER (PARTITION BY order_id, product_id ORDER BY item_id) AS so_lan_trong_don</pre>	ROW_NUMBER() là window function — tính toán trên một 'cửa sổ' dữ liệu mà không gộp các dòng lại. PARTITION BY chia dữ liệu thành nhóm (như GROUP BY nhưng giữ nguyên từng dòng). ORDER BY item_id xác định thứ tự đánh số trong mỗi nhóm.
<pre>ORDER BY order_id, product_id, item_id</pre>	Sắp xếp kết quả để dễ quan sát — các item cùng nhóm (order+product) xếp liền nhau.

Giải thích tổng thể

Window function khác GROUP BY ở chỗ: GROUP BY gộp nhiều dòng thành 1, còn window function giữ nguyên tất cả dòng và thêm một cột tính toán.

PARTITION BY order_id, product_id tạo nhóm cho mỗi cặp (đơn hàng + sản phẩm).

Trong nhóm ORD_001/PROD_001: item_id=1 được đánh số 1, item_id=7 được đánh số 2.

Bất kỳ dòng nào có **so_lan_trong_don > 1** là bản ghi trùng trong cùng đơn.

Kết quả sau khi query (minh họa)

item_id	order_id	product_id	so_lan_trong_don
1	ORD_001	PROD_001	1
7	ORD_001	PROD_001	2
2	ORD_001	PROD_002	1
4	ORD_002	PROD_001	1
5	ORD_002	PROD_004	1
6	ORD_003	PROD_003	1
9	ORD_005	PROD_002	1
8	ORD_005	PROD_004	1

Item_id=7 có so_lan_trong_don=2 — đây là bản ghi trùng của item_id=1 trong cùng ORD_001/PROD_001. Lọc WHERE so_lan_trong_don > 1 để chỉ xem bản trùng.

GÓC SOI LỖI CỦA TESTER

Ứng dụng thực tế của ROW_NUMBER() trong QA:

(1) **Phát hiện trùng**: WHERE so_lan > 1 → xem chính xác dòng nào là trùng.

(2) **Lấy bản ghi mới nhất**: PARTITION BY customer_id ORDER BY order_date DESC → lấy dòng số 1 của mỗi khách = đơn mới nhất.

(3) **Phân trang logic**: đánh số thứ tự trong nhóm để lấy top-N mỗi nhóm.

ROW_NUMBER yêu cầu MySQL 8.0+. Kiểm tra phiên bản: **SELECT VERSION();**

47

Dùng CTE + RANK() xếp hạng sản phẩm bán chạy

TÌNH HUỐNG

Kết hợp hai kỹ thuật nâng cao: **CTE** (Common Table Expression) để tính doanh số trung gian, sau đó **RANK()** để xếp hạng. Đây là cách viết SQL rõ ràng hơn nhiều so với subquery lồng nhau — đặc biệt hữu ích khi kiểm thử hệ thống báo cáo.

Bảng Products — PROD_001 xếp hạng 1 nhưng doanh số bị thổi phồng do item trùng:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
WITH doanh_so AS (  
  SELECT p.product_id,  
         p.product_name,  
         SUM(oi.quantity * oi.price)  
           AS tong_doanh_so  
  FROM   Products p  
  JOIN   Order_Items oi  
        ON p.product_id = oi.product_id  
  GROUP BY p.product_id, p.product_name  
)  
SELECT product_id,  
       product_name,  
       tong_doanh_so,  
       RANK() OVER  
         (ORDER BY tong_doanh_so DESC)  
         AS hang  
FROM   doanh_so;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

WITH doanh_so AS (SELECT ..., SUM(...) FROM Products JOIN Order_Items GROUP BY ...)	CTE (WITH ... AS): đặt tên cho một kết quả trung gian. Câu SELECT bên trong tính tổng doanh số — giống Câu 22 nhưng được đặt tên 'doanh_so' để tái sử dụng bên dưới.
RANK() OVER (ORDER BY tong_doanh_so DESC) AS hang	RANK(): gán số thứ hạng theo thứ tự. Khác ROW_NUMBER: nếu hai sản phẩm bằng doanh số, cả hai cùng nhận hạng 1 và hạng 2 bị bỏ qua (1, 1, 3...).
FROM doanh_so	Tham chiếu tới CTE như một bảng thông thường — đây là điểm mạnh của CTE: viết một lần, dùng nhiều chỗ.

Giải thích tổng thể

CTE giúp viết SQL theo kiểu 'từng bước' thay vì lồng subquery:
Bước 1 (WITH): tính tổng doanh số từng sản phẩm → lưu vào 'doanh_so'.
Bước 2 (SELECT chính): lấy từ 'doanh_so', thêm cột RANK().
PROD_001 xếp hạng 1 với 90M nhưng con số này bị thổi phồng do item trùng (Câu 2/46) — doanh số thực chỉ khoảng 60M.

Kết quả sau khi query (minh họa)

product_id	product_name	tong_doanh_so	hang
PROD_001	iPhone 15 Pro Max	90.000.000	1

product_id	product_name	tong_doanh_so	hang
PROD_003	Tai nghe Sony WH-1000XM5	8.000.000	2
PROD_002	Ban phim co Logitech	4.000.000	3
PROD_004	Sac du phong Anker	2.000.000	4

4 sản phẩm có doanh số (PROD_005, 006, 007 chưa bán). PROD_001 dẫn đầu với 90M nhưng bị phóng đại do item_id=7 trùng lặp.

GÓC SOI LỖI CỦA TESTER

Ba loại xếp hạng window function — chọn đúng theo nghiệp vụ:

- (1) **ROW_NUMBER()**: luôn số liên tiếp, không cùng hạng — 1, 2, 3, 4.
 - (2) **RANK()**: cùng điểm = cùng hạng, bỏ hạng tiếp theo — 1, 1, 3, 4.
 - (3) **DENSE_RANK()**: cùng điểm = cùng hạng, không bỏ hạng — 1, 1, 2, 3.
- Với báo cáo doanh số, RANK() thường phù hợp hơn ROW_NUMBER().

48

Dùng CTE lồng nhau tìm khách chi tiêu trên mức trung bình

TÌNH HUỐNG

Câu 47 dùng CTE một lớp. Câu này đi thêm một bước: **tái sử dụng CTE trong subquery** ngay bên trong câu SELECT chính — tính tổng chi tiêu từng khách, rồi lọc những người vượt trung bình của chính tập đó. Không dùng CTE, câu này phải viết subquery lồng 3 cấp.

Bảng Customers — C001 là khách duy nhất chi tiêu trên mức trung bình COMPLETED:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
WITH tong_chi AS (
  SELECT c.customer_id,
         c.customer_name,
         SUM(o.total_amount) AS tong_da_mua
  FROM   Customers c
  JOIN   Orders o
        ON c.customer_id = o.customer_id
  WHERE  o.status = 'COMPLETED'
  GROUP BY c.customer_id, c.customer_name
)
SELECT customer_id,
       customer_name,
       tong_da_mua
FROM   tong_chi
WHERE  tong_da_mua >
      (SELECT AVG(tong_da_mua)
       FROM   tong_chi)
ORDER BY tong_da_mua DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

WITH tong_chi AS (... WHERE status = 'COMPLETED' GROUP BY customer_id ...)	CTE tính tổng chi tiêu từng khách, chỉ tính đơn COMPLETED. Lọc WHERE bên trong CTE — không tính đơn CANCELLED hay PENDING.
WHERE tong_da_mua > (SELECT AVG(tong_da_mua) FROM tong_chi)	Điểm mạnh của CTE: tái sử dụng 'tong_chi' trong subquery ngay bên trong mệnh đề WHERE. Không CTE, ta phải tính lại toàn bộ GROUP BY một lần nữa trong subquery.
ORDER BY tong_da_mua DESC	Sắp xếp giảm dần — khách chi nhiều nhất lên đầu.

Giải thích tổng thể

CTE 'tong_chi' được dùng **hai lần** trong câu lệnh — đây là lý do chính để dùng CTE:

(1) FROM tong_chi: lấy danh sách từng khách và tổng chi tiêu.

(2) SELECT AVG(tong_da_mua) FROM tong_chi: tính trung bình từ cùng tập đó.

Tổng COMPLETED: C001=32M, C002=20M → trung bình = 26M → chỉ C001 (32M) vượt qua.

Kết quả sau khi query (minh họa)

customer_id	customer_name	tong_da_mua
C001	Nguyen Van A	32.000.000

Chỉ C001 (32M) vượt mức trung bình 26M. C002 có 20M < 26M nên bị loại. Đây là khách VIP tiềm năng — dữ liệu test nên cover luồng upgrade membership.

GÓC SOI LỖI CỦA TESTER

CTE vs Subquery — khi nào dùng cái nào:

- (1) **Dùng CTE** khi cần tái sử dụng kết quả trung gian nhiều lần, hoặc khi logic chia thành nhiều bước rõ ràng.
- (2) **Dùng subquery** khi logic đơn giản và chỉ dùng một lần.
- (3) **CTE đệ quy** (WITH RECURSIVE): dùng cho cấu trúc cây (category cha/con, cây tổ chức) — MySQL 8.0+ hỗ trợ.

49

Tính doanh thu tích lũy theo thời gian với SUM() OVER()

TÌNH HUỐNG

Báo cáo tài chính thường yêu cầu 'running total' — tổng cộng dồn theo thời gian. Không có window function, ta phải dùng self-join hoặc subquery tương quan rất phức tạp. **SUM() OVER()** giải quyết trong một câu lệnh đơn giản, và QA dùng nó để kiểm tra xem hệ thống báo cáo có tính đúng không.

Bảng Orders — doanh thu tích lũy tính theo order_date tăng dần:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       order_date,
       total_amount,
       SUM(total_amount) OVER (
         ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING
           AND CURRENT ROW
       ) AS luy_ke
FROM   Orders
ORDER BY order_date;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders — tất cả 5 đơn, kể cả CANCELLED và PENDING.
SUM(total_amount) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS luy_ke	SUM() OVER(): cộng dồn total_amount theo thứ tự order_date. ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW: cửa sổ tính từ dòng đầu tiên đến dòng hiện tại. Bản rút gọn SUM(...) OVER (ORDER BY order_date) chỉ cho CÙNG kết quả khi cột ORDER BY là duy nhất. Nếu có dòng trùng giá trị (vd nhiều đơn cùng order_date), frame mặc định là RANGE — gộp mọi dòng trùng vào cùng một mốc tích lũy, KHÁC với ROWS (cộng từng dòng). Viết ROWS tường minh khi cần cộng dồn theo từng dòng.
ORDER BY order_date	Sắp xếp kết quả cuối cùng theo ngày — đảm bảo luy_ke tăng dần theo thời gian khi đọc từ trên xuống.

Giải thích tổng thể

SUM() OVER() tính tổng tích lũy mà không gộp dòng — giữ nguyên từng đơn hàng.
ORD_001 (32M) → luy_ke = 32M. ORD_002 (20M) → luy_ke = 52M. ORD_003 (8M, CANCELLED) → luy_ke = 60M. ORD_004 (5M, C999) → luy_ke = 65M.
Lưu ý: câu này cộng tất cả đơn kể cả CANCELLED và PENDING — luy_ke thực tế (chỉ COMPLETED) là

52M, không phải 65M.

Kết quả sau khi query (minh họa)

order_id	customer_id	order_date	total_amount	luy_ke
ORD_001	C001	2026-06-20	32.000.000	32.000.000
ORD_002	C002	2026-06-22	20.000.000	52.000.000
ORD_003	C003	2026-06-23	8.000.000	60.000.000
ORD_004	C999	2026-06-24	5.000.000	65.000.000
ORD_005	C001	2026-06-25	15.000.000	80.000.000

luy_ke = 80M nhưng đây là tổng thô gồm cả đơn CANCELLED (ORD_003/8M, ORD_005/15M) và đơn lỗi (ORD_004/5M).
Doanh thu thực COMPLETED: 52M.

GÓC SOI LỖI CỦA TESTER

Để tính running total chỉ của đơn COMPLETED:

SUM(CASE WHEN status = 'COMPLETED'

THEN total_amount ELSE 0 END)

OVER (ORDER BY order_date) AS luy_ke_thuc

Kỹ thuật CASE WHEN bên trong window function cho phép tính tích lũy có điều kiện mà không cần WHERE
lọc trước (vì WHERE loại bỏ dòng → mất ORDER BY liên tục theo thời gian).

50 Báo cáo tổng hợp: nhiều loại lỗi trong một câu UNION ALL

TÌNH HUỐNG

Câu cuối tổng hợp nhiều kiểm tra thành **một báo cáo duy nhất**: email trùng (Câu 6), tồn kho âm (Câu 12), khách hàng ma (Câu 5). QA dùng câu này như một 'health check' nhanh — chạy một lần, thấy ngay hệ thống đang có bao nhiêu loại lỗi tồn đọng.

Bảng Orders (1 trong 3 nguồn lỗi minh họa) — câu lệnh quét cả Customers, Products và Orders:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```

SELECT 'Email trùng' AS loại_lỗi,
       customer_id AS doi_tuong,
       'Customers' AS bang
FROM Customers
WHERE email IN (
  SELECT email FROM Customers
  WHERE email IS NOT NULL
  AND email != ''
  GROUP BY email
  HAVING COUNT(*) > 1)
UNION ALL
SELECT 'Ton kho am', product_id, 'Products'
FROM Products WHERE stock < 0
UNION ALL
SELECT 'Khach hang ma', order_id, 'Orders'
FROM Orders
WHERE customer_id NOT IN
  (SELECT customer_id FROM Customers)
ORDER BY loại_lỗi;

```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

<pre> SELECT 'Email trùng' AS loại_lỗi, customer_id, 'Customers' AS bang FROM Customers WHERE email IN (...) </pre>	Khối 1: tìm email trùng bằng subquery (tương tự Câu 6 nhưng nhúng vào đây). Cột 'Email trùng' là chuỗi cố định — giúp phân biệt loại lỗi trong kết quả gộp.
<pre> UNION ALL SELECT 'Ton kho am', ... UNION ALL SELECT 'Khach hang ma', ... </pre>	UNION ALL ghép ba khối SELECT thành một bảng. Điều kiện: cùng số cột, kiểu dữ liệu tương thích. Dùng UNION ALL thay UNION để không mất dòng trùng — các lỗi ở bảng khác nhau không bao giờ trùng. Khối 'Khach hang ma' dùng NOT IN — an toàn ở đây vì customer_id là PRIMARY KEY của Customers (không thể NULL), nhưng xem lại bẫy NOT IN + NULL đã học ở Câu 9 trước khi tái sử dụng pattern này cho cột khác.
<pre> ORDER BY loại_lỗi </pre>	Sắp xếp theo loại lỗi — nhóm các lỗi cùng loại lại gần nhau để dễ đọc báo cáo.

Giải thích tổng thể

Kết quả có 6 dòng — nhiều hơn dự kiến do một phát hiện bất ngờ:

Email trùng: C001, C004, C005, C009 — không chỉ C004 và C005.

Nguyên nhân: MySQL 8.0+ mặc định so sánh VARCHAR **không phân biệt hoa/thường** (collation utf8mb4_0900_ai_ci). 'a.nguyen@email.com' (C001) và 'A.NGUYEN@EMAIL.COM' (C009) được coi là cùng email → cả hai bị bắt.

Đây là bug ẩn mà Câu 6 cũng đã bắt — cần xác nhận lại policy: hệ thống có phân biệt hoa thường trong email không?

Kết quả sau khi query (minh họa)

loại_lỗi	doi_tuong	bang
Email trùng	C001	Customers
Email trùng	C004	Customers
Email trùng	C005	Customers
Email trùng	C009	Customers

loai_loi	doi_tuong	bang
Khách hàng ma	ORD_004	Orders
Tồn kho âm	PROD_003	Products

6 lỗi từ 3 loại: 4 email trùng (C001+C009 vì MySQL case-insensitive), 1 đơn mờ côi (ORD_004), 1 tồn kho âm (PROD_003). Chạy câu này mỗi ngày để theo dõi dữ liệu sạch hay không.

GÓC SOI LỖI CỦA TESTER

Mở rộng health check: thêm loại lỗi mới bằng cách thêm UNION ALL:

UNION ALL

SELECT 'Gia NULL', product_id, 'Products'
FROM Products WHERE price IS NULL

Để báo cáo có thêm cột 'so_luong' đếm bao nhiêu lỗi mỗi loại, bọc toàn bộ UNION ALL vào một CTE rồi GROUP BY loại_loi bên ngoài.

Khi thêm khối mới dùng **NOT IN (subquery)** như khối 'Khách hàng ma': nếu cột trong subquery CÓ thể NULL, đổi sang **NOT EXISTS** để an toàn tuyệt đối (xem cách viết tương đương ở Bài tập 5.2) — đừng dựa vào giả định 'cột này chắc không NULL', vì giả định đó có thể đúng hôm nay nhưng sai sau khi schema đổi.

BÀI TẬP TỰ LUYỆN — PHẦN 6

Bài 6.1. Dùng RANK() xếp hạng sản phẩm theo tổng doanh số bán ra (tính từ Order_Items).

Gợi ý: `RANK() OVER (ORDER BY SUM(quantity*price) DESC) kết hợp GROUP BY product_id.`

Bài 6.2. Dùng UNION ALL tạo báo cáo đếm nhanh: bao nhiêu khách lỗi email, bao nhiêu sản phẩm giá NULL, bao nhiêu đơn mờ côi.

Gợi ý: Mỗi dòng là một `SELECT COUNT(*)` kèm nhãn chuỗi cố định, nối bằng `UNION ALL`.

→ Lời giải đầy đủ ở **Phụ lục D** (cuối sách). Hãy tự viết trước khi xem.

Case study: Một ngày điều tra dữ liệu của QA

BỐI CẢNH

Sáng thứ Hai, dashboard báo doanh số iPhone 15 Pro Max (PROD_001) tháng này là **90.000.000đ** — tương đương 3 máy. Nhưng bộ phận kho khẳng định chỉ xuất **2 máy** (60.000.000đ). Lẽch 30 triệu. QA được nhờ soi xem dữ liệu sai ở đâu.

Thay vì đoán, QA lần theo dấu vết bằng chính các câu lệnh trong sách — mỗi bước thu hẹp phạm vi nghi ngờ cho tới khi chạm nguyên nhân gốc.

Nhịp 1 · Chụp toàn cảnh (Câu 21)

Đếm số dòng items theo từng đơn: ORD_001 hiện 3 dòng cho một đơn lẽ ra chỉ 2 sản phẩm. Con số bất thường này khiến QA nghi có item bị nhân đôi.

Nhịp 2 · Định vị nghi phạm (Câu 22)

Xếp hạng doanh số: PROD_001 = 90.000.000đ = đúng 3×30 triệu. Nhưng nghiệp vụ chỉ ghi nhận 2 lần bán — vậy có một lần bán 'ảo' đã lọt vào dữ liệu.

Nhịp 3 · Truy tới gốc (Câu 46)

Dùng ROW_NUMBER() PARTITION BY (order_id, product_id): cặp ORD_001/PROD_001 có so_lan = 2 — item_id 1 và item_id 7 là cùng một dòng bị lặp. Đây chính là chiếc iPhone thứ ba 'ảo'.

Nhịp 4 · Đo đúng phần lệch (Câu 11)

Đối soát ORD_001: tổng từ items = 62 triệu nhưng total_amount ghi 32 triệu — lệch đúng 30 triệu, khớp chính xác giá một chiếc iPhone bị đếm thừa.

Nhịp 5 · Đánh giá phạm vi ảnh hưởng (Câu 27)

Tổng theo danh mục: 'Điện thoại' hiện 90 triệu thay vì 60 triệu thực. Báo cáo cấp danh mục và toàn hệ thống đều bị thổi phồng từ đúng một dòng lỗi này.

Nhịp 6 · Quét lỗi đi kèm (Câu 50)

Trong lúc mở DB, chạy health-check tổng hợp: ngoài item trùng còn lộ email trùng (C004/C005) và đơn mờ côi ORD_004/C999 — không liên quan vụ 30 triệu nhưng ghi nhận để xử lý riêng.

KẾT LUẬN & KIẾN NGHỊ

Nguyên nhân gốc: bảng Order_Items thiếu ràng buộc duy nhất trên (order_id, product_id), khiến cùng một dòng bị ghi hai lần.

- Báo dev thêm **UNIQUE(order_id, product_id)** (hoặc kiểm tra ở tầng ứng dụng) để chặn lặp.
- Dọn dữ liệu: xóa item_id = 7 bị trùng, tính lại total_amount và các báo cáo liên quan.
- Báo kế toán con số đúng: doanh số iPhone là **60 triệu**, không phải 90 triệu.

Điều đáng nhớ không phải sáu câu lệnh, mà là **nhịp điều tra**: chụp toàn cảnh → định vị nghi phạm → truy tới gốc → đo đúng phần lệch → đánh giá phạm vi → quét lỗi đi kèm. SQL là đèn pin; tư duy điều tra mới là hướng soi.

Kiểm thử ghi dữ liệu: hành động trên ứng dụng → xác minh database

50 câu phía trước soi **dữ liệu đã có sẵn**. Nhưng phần lớn công việc kiểm thử chạm tới database lại là một câu hỏi khác: “**Tôi vừa thao tác trên ứng dụng — database có ghi lại đúng không?**” Tạo một đơn hàng, sửa hồ sơ, hủy giao dịch — mỗi hành động phải để lại đúng dấu vết trong DB. Đây là kỹ năng QA chức năng dùng hằng ngày.

Khuôn ba bước: Trước → Hành động → Sau (Arrange – Act – Assert)

TRƯỚC (Arrange) — chụp trạng thái nền

Ghi lại giá trị DB sẽ bị thay đổi **trước** khi thao tác — nhất là số sẽ biến động (tồn kho, số dư, số dòng). Không có mốc 'trước', bạn không thể khẳng định 'sau' đúng hay sai.

HÀNH ĐỘNG (Act) — thao tác qua app/API

Thực hiện đúng một thao tác trên giao diện hoặc API như người dùng thật — **không** sửa DB bằng tay. Mục tiêu là kiểm xem hệ thống ghi đúng không, chứ không phải tự ghi hộ nó.

SAU (Assert) — soi DB và so với kỳ vọng

Chạy SELECT kiểm từng điều phải đúng và so với kỳ vọng: bản ghi chính, các bảng liên quan, giá trị dẫn xuất, và quan trọng — **không có bản ghi thừa hay thiếu**.

Ví dụ — đặt một đơn hàng mới rồi xác minh

HÀNH ĐỘNG

Trên giao diện, khách **C001** đặt mua **2** chiếc iPhone 15 Pro Max (PROD_001, giá 30.000.000). Hệ thống sinh đơn mới **ORD_006**. Tồn kho PROD_001 trước khi đặt là **50**. Sau thao tác, DB phải thể hiện đúng bốn điều dưới đây.

1. Bản ghi đơn được tạo đúng — đúng khách, tổng tiền, trạng thái, ngày.

```
SELECT order_id, customer_id, total_amount, status, order_date
FROM Orders
WHERE order_id = 'ORD_006';
```

Kỳ vọng: customer_id = C001, total_amount = 60.000.000, status hợp lệ (vd PENDING), order_date = hôm nay.

2. Dòng chi tiết đúng và KHÔNG thừa — số lượng dòng phải khớp.

```
SELECT product_id, quantity, price
FROM Order_Items
WHERE order_id = 'ORD_006';
```

Kỳ vọng đúng 1 dòng: PROD_001, quantity = 2, price = 30.000.000 (giá tại thời điểm mua). Nếu ra 2 dòng → double-submit (Câu 2, 29); nếu rỗng → đơn tạo mà thiếu chi tiết (Câu 15, 38).

3. Tồn kho bị trừ đúng số lượng — so với mốc 'trước'.

```
SELECT stock
FROM Products
WHERE product_id = 'PROD_001';
```

Trước khi đặt stock = 50, bán 2 → kỳ vọng 48. Vẫn 50 → quên trừ kho; còn 46 → trừ hai lần (xem Câu 20, 28).

4. Giá trị dẫn xuất nhất quán: total_amount = SUM(quantity × price).

```
SELECT o.total_amount,
       SUM(oi.quantity * oi.price) AS tinh_tu_items
FROM Orders o
JOIN Order_Items oi ON o.order_id = oi.order_id
WHERE o.order_id = 'ORD_006'
GROUP BY o.total_amount;
```

Hai con số phải bằng nhau (60.000.000 = 2 × 30.000.000). Lỗi là lỗi tính tiền — đúng mẫu Câu 11, nay áp cho một đơn vừa tạo.

CHECKLIST — SAU MỖI THAO TÁC GHI, KIỂM GÌ?

- Bản ghi chính được tạo / sửa / xóa đúng như mong đợi?
- Mọi bảng **liên quan** cập nhật đúng (tồn kho, audit log, bảng đối soát)?
- Số dòng đúng — không thừa (double-submit), không thiếu?
- Giá trị dẫn xuất nhất quán (total = SUM items, số dư, điểm thưởng)?
- Cột tự động đúng (timestamp, status mặc định, khóa ngoại)?
- Nếu thao tác **thất bại**: DB có rollback sạch, không để lại bản ghi rác?

GÓC SOI LỖI CỦA TESTER

Những bug 'ghi dữ liệu' hay gặp nhất:

- App báo 'thành công' nhưng DB không ghi — hoặc ngược lại, DB ghi mà app báo lỗi (để lại rác).
- Cập nhật **một phần**: đơn được tạo nhưng quên trừ kho hoặc quên ghi audit log.
- Trừ kho hai lần / tạo đơn trùng do double-submit (Câu 2, 29).
- **Soft-delete**: 'xóa' trên app nhưng DB chỉ đánh dấu cờ — phải kiểm đúng cơ chế, đừng tưởng mất là mất.
- Sai múi giờ hoặc timestamp khi ghi (Câu 34).

Luôn kiểm trên môi trường test/staging, và chạy lại file **ecommerce_test_setup.sql** để đưa dữ liệu về mốc gốc trước mỗi lượt kiểm — nhờ vậy con số 'trước' luôn xác định và kết quả tái lập được.

Kiểm thử giao dịch và truy cập đồng thời

Các câu trước phát hiện **hậu quả**: tồn kho âm (Câu 12), đơn trùng (Câu 29). Chương này đi tới **nguyên nhân**: phần lớn các bug đó sinh ra từ giao dịch không nguyên tử hoặc nhiều người thao tác cùng lúc. QA cần biết cách chủ động **kiểm và tái hiện** chúng — bằng chính SQL.

Tính nguyên tử: cả cụm cùng sống hoặc cùng chết

Một thao tác nghiệp vụ thường đụng nhiều bảng. 'Đặt đơn' = thêm Orders + thêm Order_Items + trừ Products.stock. Cả ba phải nằm trong MỘT **transaction**: lỗi một bước thì huỷ tất cả (ROLLBACK), không để lại cập nhật nửa vời.

```
START TRANSACTION;
INSERT INTO Orders      VALUES ('ORD_006','C001',60000000,'PENDING',CURDATE());
INSERT INTO Order_Items VALUES (10,'ORD_006','PROD_001',2,30000000);
UPDATE Products SET stock = stock - 2 WHERE product_id = 'PROD_001';
ROLLBACK;    -- giả lập một bước lỗi → huỷ TẤT CẢ

-- Kiểm: không được còn dấu vết nào của ORD_006
SELECT * FROM Orders      WHERE order_id = 'ORD_006';    -- phải rỗng
SELECT * FROM Order_Items WHERE order_id = 'ORD_006';    -- phải rỗng
SELECT stock FROM Products WHERE product_id = 'PROD_001'; -- phải vẫn = 50
```

Nếu sau ROLLBACK vẫn còn ORD_006, hoặc kho đã bị trừ → ứng dụng **không bọc transaction đúng**: lỗi nửa chừng sẽ để lại dữ liệu rác. Bài học cho tester: **luôn test cả luồng lỗi**, đừng chỉ test luồng thành công.

Race condition: tái hiện 'bán vượt kho' bằng hai phiên

Giả sử một sản phẩm chỉ còn 1 chiếc. Hai khách bấm Mua gần như cùng lúc. Mở hai phiên MySQL và xen kẽ các bước theo thời gian:

Phiên A (khách 1)	Phiên B (khách 2)
1) START TRANSACTION; SELECT stock; -- còn 1	
	2) START TRANSACTION; SELECT stock; -- cũng còn 1
3) UPDATE stock=stock-1; COMMIT; -- còn 0	
	4) UPDATE stock=stock-1; COMMIT; -- thành -1 !

Cả hai cùng đọc thấy 'còn 1' rồi cùng trừ → bán ra 2 chiếc, kho thành -1. Đây chính là nguồn gốc của tồn kho âm (Câu 12) và bán vượt kho (Câu 20). Hai cách chặn:

```
-- Cách 1: UPDATE có điều kiện (nguyên tử) – chỉ trừ khi còn hàng
UPDATE Products SET stock = stock - 1
WHERE product_id = 'PROD_004' AND stock >= 1;
-- Nếu ROW_COUNT() = 0 → đã hết hàng, từ chối đơn

-- Cách 2: khoá bi quan – đọc kèm khoá, phiên kia phải CHỜ
SELECT stock FROM Products WHERE product_id = 'PROD_004' FOR UPDATE;
```

Cách test: mở hai phiên và xen kẽ như bảng trên. Nếu hệ thống cho bán nhiều hơn tồn → bug. Lưu ý: race chỉ lộ khi chạy đồng thời — thao tác tuần tự một người gần như không bao giờ thấy.

Mức cô lập (isolation level) — phiên này thấy gì của phiên kia

Mức cô lập quyết định một transaction 'nhìn thấy' dữ liệu của transaction khác ra sao. Ba hiện tượng cần biết và mức chặn được:

Hiện tượng	Là gì	Chặn được từ mức
Dirty read	Đọc trùng dữ liệu phiên khác CHƯA commit (có thể bị rollback sau đó)	READ COMMITTED
Non-repeatable read	Đọc cùng một dòng hai lần ra hai giá trị (phiên khác sửa + commit xen giữa)	REPEATABLE READ
Phantom read	Đọc cùng điều kiện hai lần ra số dòng khác (phiên khác thêm/xóa dòng)	SERIALIZABLE

MySQL (InnoDB) mặc định **REPEATABLE READ**. Xem mức hiện tại bằng **SELECT @@transaction_isolation;**. Với luồng nhạy cảm (trừ kho, thanh toán), QA nên xác nhận đội dev dùng đúng mức cô lập hoặc khoá phù hợp.

GÓC SOI LỖI CỦA TESTER

- Chỉ test luồng thành công, quên luồng lỗi giữa chừng → bỏ sót bug nguyên tử (cập nhật nửa vời).
- App không bọc nhiều bước trong một transaction → lỗi nửa chừng để lại dữ liệu rác.
- Thiếu khóa hoặc điều kiện nguyên tử → oversell, double-charge khi tải cao — **chỉ lộ lúc nhiều người**, không lộ khi test tay một mình.
- Deadlock dưới tải → giao dịch fail ngẫu nhiên, khó tái hiện nếu không stress test.

Bug đồng thời là loại 'ẩn' nhất: không thấy bằng thao tác tuần tự, chỉ lộ khi hai phiên chạy song song hoặc khi stress test. Sau mỗi lượt thử, chạy lại **ecommerce_test_setup.sql** để đưa dữ liệu về mốc gốc.

Chạy SQL an toàn trên production

Các câu lệnh trong sách đều là **SELECT** — chỉ đọc, không sửa. Nhưng khi chạy trên cơ sở dữ liệu thật đang phục vụ người dùng, ngay cả một câu **SELECT** viết ẩu cũng có thể làm chậm cả hệ thống. Phần này là những nguyên tắc tối thiểu để không gây hại.

Nguyên tắc vàng khi chạm vào dữ liệu thật

- Ưu tiên chạy trên **replica / bản sao chỉ đọc**, không phải database chính đang phục vụ giao dịch.
- Dùng **tài khoản chỉ-đọc** (read-only) để không thể vô tình **UPDATE** hay **DELETE**.
- Luôn thêm **LIMIT** khi thăm dò — đừng **SELECT** toàn bộ một bảng hàng triệu dòng.
- Tránh giờ cao điểm; câu lệnh nặng (**JOIN** lớn, thiếu index) có thể khóa bảng và làm chậm hệ thống.
- Nếu buộc phải sửa: chạy **SELECT** với **đúng WHERE** trước, xem kết quả, rồi mới đổi thành **UPDATE/DELETE**; bọc trong transaction để còn **ROLLBACK** nếu sai.

Đọc EXPLAIN — câu lệnh của bạn có 'quét cả bảng' không?

Đặt **EXPLAIN** trước câu **SELECT**, MySQL sẽ cho biết kế hoạch thực thi mà không thật sự chạy nó — cách kiểm tra một truy vấn có hiệu quả không trước khi chạy trên dữ liệu lớn.

```
EXPLAIN
SELECT email, COUNT(*)
FROM Customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Cột	Ý nghĩa	Cần lo khi nào
type	Cách truy cập bảng	ALL = quét toàn bảng (đáng lo trên bảng lớn). ref / eq_ref / range = đang dùng index (tốt).
key	Index đang được dùng	NULL nghĩa là KHÔNG dùng index nào — thường đi kèm type = ALL .
rows	Số dòng ước tính phải đọc	Càng lớn càng nặng. Đối chiếu với tổng số dòng bảng để biết có quét gần hết không.
Extra	Ghi chú thêm	Using filesort / Using temporary = phải sắp xếp hoặc tạo bảng tạm — dấu hiệu cần tối ưu.

Với bộ dữ liệu mẫu nhỏ trong sách, type = **ALL** hoàn toàn vô hại. Nhưng nếu bảng **Customers** có 10 triệu dòng, câu tìm email trùng cần một index trên cột email — nếu không, mỗi lần chạy là một lần quét 10 triệu dòng.

Khác biệt dialect — khi đổi sang PostgreSQL hay SQL Server

Sách dùng cú pháp MySQL. Phần lớn câu lệnh chạy nguyên vẹn trên hệ khác, nhưng vài chỗ khác nhau:

Việc cần làm	MySQL	PostgreSQL	SQL Server
Giới hạn số dòng	<code>LIMIT 10</code>	<code>LIMIT 10</code>	<code>TOP 10</code>
So khớp biểu thức	<code>x REGEXP '...'</code>	<code>x ~ '...'</code>	<code>LIKE/PATINDEX</code>
Thay NULL	<code>IFNULL(x, 0)</code>	<code>COALESCE(x, 0)</code>	<code>ISNULL(x, 0)</code>
Nối chuỗi	<code>CONCAT(a, b)</code>	<code>a b</code>	<code>a + b</code>

Việc cần làm	MySQL	PostgreSQL	SQL Server
Phân biệt hoa/thường	Thường KHÔNG	CÓ (mặc định)	Theo collation

Window function (ROW_NUMBER, RANK, SUM OVER) và CTE (WITH) đều được hỗ trợ trên cả ba hệ — MySQL từ 8.0, PostgreSQL từ 9.4, SQL Server từ 2012.

PHỤ LỤC — Tra cứu nhanh

Ba bảng tra để dùng cuốn sách như công cụ hằng ngày: từ một triệu chứng nghi ngờ tìm ngay câu lệnh phù hợp, tra nhanh cú pháp lỗi, và bản đồ nổi từng lỗi mẫu với các câu phát hiện nó.

A · Gặp triệu chứng này → dùng câu nào

Triệu chứng / nghi vấn khi kiểm thử	Câu nên dùng
Bản ghi nhân đôi (cùng giá trị hoặc cùng khóa)	Câu 1, 2, 8, 29, 46
Ô bắt buộc bị bỏ trống (NULL hoặc chuỗi rỗng)	Câu 3, 7, 12, 14
Khóa ngoại trỏ vào bản ghi không tồn tại (orphan)	Câu 5
Chuỗi bản: khoảng trắng thừa, ký tự lạ, hoa/thường	Câu 4, 6, 31, 35
Email sai định dạng cơ bản	Câu 32
Giá trị ngoài danh sách cho phép (ENUM)	Câu 9
Tồn kho âm hoặc bán vượt tồn kho	Câu 12, 20, 28
Giá hoặc tiền âm / bằng 0	Câu 14
Ràng buộc DB chưa được enforce (thiếu UNIQUE/FK)	Câu 18
Schema lạ, chưa có tài liệu	Câu 13
Lịch giữa tổng header và tổng chi tiết	Câu 11, 21, 38, 39
Đơn hàng rỗng (không có dòng chi tiết)	Câu 15, 38
Số lượng hoặc ngày tháng bất thường	Câu 30, 33, 34
Giá bán lịch sử khác giá niêm yết hiện tại	Câu 24
Sản phẩm / khách hàng chưa phát sinh giao dịch	Câu 16, 17
Bản ghi bị xóa để lại dấu vết (gap ID, xóa mềm)	Câu 10, 40, 41, 44
Định giá tồn kho / báo cáo tổng hợp / phân bổ	Câu 19, 23, 25, 26, 27, 45
Xếp hạng hoặc đánh số bản ghi theo nhóm	Câu 22, 46, 47, 48, 49
Đơn tồn đọng lâu hoặc dòng thời gian bất thường	Câu 42, 43
Health check nhanh nhiều loại lỗi cùng lúc	Câu 50

B · Cheat sheet cú pháp lỗi

Mục tiêu	Cú pháp lỗi	Câu mẫu
Tìm bản ghi trùng	GROUP BY col HAVING COUNT(*) > 1	1, 2, 8, 29
Tìm mô côi — an toàn với NULL	WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.fk = a.id)	BT 5.1, 5.2

Mục tiêu	Cú pháp lỗi	Câu mẫu
Tìm mờ côi — viết ngắn	LEFT JOIN b ON ... WHERE b.id IS NULL	5, 16, 17
Bẫy cần tránh	NOT IN (subquery có NULL) → luôn rỗng	9
Xử lý NULL khi tính toán	COALESCE(x, 0)	28
Chuẩn hóa chuỗi trước khi so	TRIM(x), LOWER(x)	4, 6, 35
Kiểm tra định dạng chuỗi	x REGEXP 'mau'	31, 32
Đếm có điều kiện	SUM(CASE WHEN ... THEN 1 ELSE 0 END)	7
Tính phần trăm / tỷ lệ	COUNT(*) * 100.0 / (SELECT COUNT(*) ...)	26
Đánh số thứ tự trong nhóm	ROW_NUMBER() OVER (PARTITION BY g ORDER BY c)	46
Xếp hạng (cho phép đồng hạng)	RANK() / DENSE_RANK() OVER (ORDER BY c)	22, 47
Cộng dồn theo thời gian	SUM(x) OVER (ORDER BY ngày)	49
Tái dùng kết quả trung gian	WITH cte AS (...) SELECT ... FROM cte	47, 48
Ghép nhiều báo cáo theo cột dọc	SELECT ... UNION ALL SELECT ...	40, 50
Ngưỡng động tính từ chính bảng	WHERE x > (SELECT AVG(x)*k FROM t)	30, 39, 48

C · Bản đồ 15 lỗi mẫu → câu phát hiện

Mã	Lỗi cài sẵn trong dữ liệu mẫu	Câu phát hiện
Bug-A	ORD_001: total 32M nhưng items cộng 62M (item trùng)	2, 11, 39, 46
Bug-B	ORD_002: total 20M nhưng items cộng 31M	11, 19, 39
Bug-C	PROD_003: stock = -5 (tồn kho âm)	12, 20, 28
Bug-D	C004/C005: trùng email trung_email@email.com	1, 36, 50
Bug-E	C001/C009: trùng email khác hoa/thường	6, 37, 50
Bug-F	C006: email NULL; C007: email rỗng	3, 7, 32
Bug-G	C008: khoảng trắng thừa trong tên	4
Bug-H	C010: membership_tier = VIP (ngoài danh sách)	9, 36
Bug-I	ORD_004: customer_id C999 (orphan) + đơn rỗng	5, 15, 38, 42, 50
Bug-J	ORD_001/PROD_001: trùng (item 1 và 7)	2, 46
Bug-K	item_id = 3 bị thiếu (gap chuỗi ID)	44
Bug-L	PROD_005: trùng tên + giá với PROD_002	8, 35
Bug-M	PROD_006 price NULL; PROD_007 stock NULL	7, 12, 14

Mã	Lỗi cài sẵn trong dữ liệu mẫu	Câu phát hiện
Bug-N	ORD_005: xóa mềm nhưng item 8,9 chưa dọn, đơn vẫn bị tính	10, 40
Bug-O	ORD_003: đã CANCELLED nhưng chưa xóa mềm (deleted_at NULL)	41

D · Đáp án bài tập tự luyện

Lời giải dưới đây chỉ là MỘT cách viết đúng — cách của bạn có thể khác mà vẫn cho cùng kết quả. Đáp án được kiểm chứng trên bộ dữ liệu mẫu nhỏ trong sách.

PHẦN 1 · Toàn vẹn và trùng lặp dữ liệu

Bài 1.1. Có những hạng thành viên (membership_tier) nào đang được gán cho từ 2 khách trở lên? Đếm mỗi hạng có bao nhiêu khách.

```
SELECT membership_tier,
       COUNT(*) AS so_khach
FROM   Customers
GROUP BY membership_tier
HAVING COUNT(*) > 1
ORDER BY so_khach DESC;
```

ĐÁP ÁN

Standard (5), Silver (2), Gold (2). VIP chỉ 1 khách nên bị HAVING loại — nhưng VIP vẫn là giá trị sai (Câu 9). HAVING lọc theo SỐ LƯỢNG, không phải tính hợp lệ của giá trị.

Bài 1.2. Tìm những khách hàng có customer_name chứa khoảng trắng thừa ở đầu hoặc cuối.

```
SELECT customer_id,
       customer_name,
       CHAR_LENGTH(customer_name) AS do_dai
FROM   Customers
WHERE  customer_name <> TRIM(customer_name);
```

ĐÁP ÁN

C008 (' Pham Van D ', dài 14 ký tự — sau TRIM còn 10). Khoảng trắng thừa không nhìn thấy bằng mắt nhưng phá vỡ so khớp, gây trùng ẩn và lỗi tìm kiếm.

Bài 1.3. Sản phẩm nào được mua trong nhiều hơn một đơn hàng khác nhau?

```
SELECT product_id,
       COUNT(DISTINCT order_id) AS so_don
FROM   Order_Items
GROUP BY product_id
HAVING COUNT(DISTINCT order_id) > 1;
```

ĐÁP ÁN

PROD_001, PROD_002, PROD_004 — mỗi sản phẩm nằm trong 2 đơn khác nhau (PROD_001: ORD_001+ORD_002; PROD_002: ORD_001+ORD_005; PROD_004: ORD_002+ORD_005). Lưu ý COUNT(DISTINCT order_id) khác COUNT(*): PROD_001 có 3 dòng item nhưng item 1 và 7 cùng thuộc ORD_001 nên chỉ tính 1 đơn cho lần đó.

PHẦN 2 · Ràng buộc nghiệp vụ

Bài 2.1. Liệt kê các sản phẩm có giá cao hơn giá trung bình của toàn bộ sản phẩm.

```
SELECT product_id,
       product_name,
       price
FROM   Products
WHERE  price > (SELECT AVG(price) FROM Products)
ORDER BY price DESC;
```

ĐÁP ÁN

PROD_001 (30M) và PROD_003 (8M). Trung bình $\approx 7,4M$. Lưu ý: AVG tự bỏ qua PROD_006 (giá NULL) nên ngưỡng chỉ tính trên 6 sản phẩm có giá.

Bài 2.2. Trong các đơn COMPLETED, đơn nào có total_amount KHÁC tổng tiền tính từ Order_Items?

```
SELECT o.order_id,
       o.total_amount,
       SUM(oi.quantity * oi.price) AS tong_items
FROM   Orders o
JOIN   Order_Items oi ON o.order_id = oi.order_id
WHERE  o.status = 'COMPLETED'
GROUP BY o.order_id, o.total_amount
HAVING SUM(oi.quantity * oi.price) <> o.total_amount;
```

ĐÁP ÁN

ORD_001 (32M vs 62M — item trùng) và ORD_002 (20M vs 31M — Bug-B). ORD_003 bị loại vì CANCELLED. Đây là Câu 11 thu hẹp vào đơn đã hoàn tất — nơi sai số gây hậu quả tài chính thật.

Bài 2.3. Sản phẩm nào CHƯA từng được bán nhưng vẫn còn tồn kho lớn hơn 0?

```
SELECT p.product_id,
       p.product_name,
       p.stock
FROM   Products p
WHERE  NOT EXISTS (
        SELECT 1 FROM Order_Items oi
        WHERE  oi.product_id = p.product_id
      )
AND    p.stock > 0;
```

ĐÁP ÁN

PROD_005 (30) và PROD_006 (10). PROD_007 bị loại vì stock = NULL — so sánh 'NULL > 0' cho UNKNOWN, không phải TRUE. Đây là lý do hàng tồn 'tàng hình' dễ bị bỏ sót.

PHẦN 3 · Đối soát và tính toán

Bài 3.1. Tính tổng doanh thu thực (số lượng × giá) của từng đơn hàng, sắp xếp giảm dần.

```
SELECT o.order_id,
       SUM(oi.quantity * oi.price) AS doanh_thu
FROM   Orders o
JOIN   Order_Items oi ON o.order_id = oi.order_id
GROUP BY o.order_id
ORDER BY doanh_thu DESC;
```

ĐÁP ÁN

ORD_001 = 62M · ORD_002 = 31M · ORD_003 = 8M. ORD_001 = 62M (bị item trùng thối phồng) so với total_amount ghi 32M — luôn đối soát hai con số này (Câu 11).

Bài 3.2. Mỗi danh mục (category) có bao nhiêu sản phẩm, kể cả sản phẩm chưa bán?

```
SELECT category,
       COUNT(*) AS so_sp
FROM   Products
GROUP BY category
ORDER BY so_sp DESC;
```

ĐÁP ÁN

Phu kien (6) · Dien thoai (1). Vì đếm trên Products nên bao gồm cả sản phẩm chưa bán — khác Câu 27 (chỉ đếm danh mục có phát sinh đơn qua JOIN).

Bài 3.3. Khách hàng nào có tổng chi tiêu cao hơn mức trung bình của các khách đã từng mua?

```
SELECT c.customer_id,
       c.customer_name,
       SUM(o.total_amount) AS tong
FROM   Customers c
JOIN   Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING SUM(o.total_amount) > (
    SELECT AVG(t) FROM (
        SELECT SUM(o2.total_amount) AS t
        FROM   Orders o2
        JOIN   Customers c2 ON o2.customer_id = c2.customer_id
        GROUP BY o2.customer_id) x);
```

ĐÁP ÁN

C001 (47M). Trung bình ba khách có đơn = 25M ((47+20+8)/3); chỉ C001 vượt. Mẫu 'so với trung bình của chính tập' là nền của phân tích outlier — Câu 48 dùng CTE để viết gọn hơn (lưu ý: Câu 48 chỉ tính đơn COMPLETED nên số khác bài này).

PHẦN 4 · Biên và dữ liệu bất thường

Bài 4.1. Tìm các sản phẩm bị thiếu dữ liệu giá HOẶC tồn kho (giá trị NULL).

```
SELECT product_id,  
       product_name,  
       price,  
       stock  
FROM   Products  
WHERE  price IS NULL  
       OR stock IS NULL;
```

ĐÁP ÁN

PROD_006 (price NULL) và PROD_007 (stock NULL). Phải dùng IS NULL — viết 'price = NULL' luôn cho kết quả rỗng. Hai sản phẩm này phá vỡ mọi phép tính liên quan đến giá/kho.

Bài 4.2. Tìm khách hàng có tên chứa chữ số.

```
SELECT customer_id,  
       customer_name  
FROM   Customers  
WHERE  customer_name REGEXP '[0-9]';
```

ĐÁP ÁN

C009 ('Nguyen Van A (2)'). Tên người thật hiếm khi có chữ số; '(2)' là dấu hiệu dữ liệu được thêm để né ràng buộc trùng tên (Câu 31).

Bài 4.3. Tìm các tên sản phẩm bị trùng sau khi chuẩn hóa (bỏ khoảng trắng + đưa về chữ thường).

```
SELECT LOWER(TRIM(product_name)) AS ten_chuan,  
       COUNT(*) AS so_lan  
FROM   Products  
GROUP BY LOWER(TRIM(product_name))  
HAVING COUNT(*) > 1;
```

ĐÁP ÁN

'ban phim co logitech' xuất hiện 2 lần (PROD_002 + PROD_005). Chuẩn hóa trước khi GROUP giúp bắt trùng mà so khớp thô bỏ sót (Câu 35).

PHẦN 5 · Audit, log và dấu vết

Bài 5.1. Dùng NOT EXISTS để liệt kê sản phẩm chưa bao giờ xuất hiện trong Order_Items.

```
SELECT p.product_id,
       p.product_name
FROM   Products p
WHERE  NOT EXISTS (
        SELECT 1 FROM Order_Items oi
        WHERE oi.product_id = p.product_id);
```

ĐÁP ÁN

PROD_005, PROD_006, PROD_007. Kết quả giống Câu 16 (LEFT JOIN + IS NULL) nhưng NOT EXISTS an toàn hơn khi khóa có thể NULL (xem bẫy NOT IN ở Câu 9).

Bài 5.2. Dùng NOT EXISTS tìm đơn hàng trở tới khách hàng không tồn tại trong Customers.

```
SELECT o.order_id,
       o.customer_id
FROM   Orders o
WHERE  NOT EXISTS (
        SELECT 1 FROM Customers c
        WHERE c.customer_id = o.customer_id);
```

ĐÁP ÁN

ORD_004 / C999 (đơn mồ côi). Cùng kết quả Câu 5 (LEFT JOIN + IS NULL) nhưng viết bằng NOT EXISTS — QA cần đọc được cả hai cách trong dự án thật.

PHẦN 6 · Truy vấn nâng cao cho QA

Bài 6.1. Dùng RANK() xếp hạng sản phẩm theo tổng doanh số bán ra (tính từ Order_Items).

```
SELECT product_id,
       SUM(quantity * price) AS doanh_so,
       RANK() OVER (
         ORDER BY SUM(quantity * price) DESC) AS hang
FROM   Order_Items
GROUP BY product_id;
```

ĐÁP ÁN

PROD_001 90M (hạng 1) · PROD_003 8M (2) · PROD_002 2M (3) · PROD_004 1M (4). Xếp hạng đúng kỹ thuật nhưng 90M của PROD_001 bị thổi phồng do item trùng — dữ liệu nền vẫn bẩn (Câu 47).

Bài 6.2. Dùng UNION ALL tạo báo cáo đếm nhanh: bao nhiêu khách lỗi email, bao nhiêu sản phẩm giá NULL, bao nhiêu đơn mờ côi.

```
SELECT 'Email NULL/rong' AS loai, COUNT(*) AS so_luong
FROM Customers WHERE email IS NULL OR TRIM(email) = ''
UNION ALL
SELECT 'Gia NULL', COUNT(*)
FROM Products WHERE price IS NULL
UNION ALL
SELECT 'Don mo coi', COUNT(*)
FROM Orders o WHERE NOT EXISTS (
    SELECT 1 FROM Customers c
    WHERE c.customer_id = o.customer_id);
```

ĐÁP ÁN

Email NULL/rỗng = 2 (C006, C007) · Giá NULL = 1 (PROD_006) · Đơn mờ côi = 1 (ORD_004). Mẫu health-check: một câu trả ra nhiều loại lỗi (Câu 50), dễ mở rộng bằng cách thêm UNION ALL.

E · Tra cứu theo module nghiệp vụ

Phụ lục A tra theo **triệu chứng**; phụ lục này tra theo **module bạn đang test** — sát với cách một QA thực tế làm việc hằng ngày hơn cách đọc tuần tự 6 phần. Đang test tính năng nào, tra đúng hàng tương ứng để biết ngay nên tham khảo câu nào.

Module / bảng dữ liệu chính	Câu nên tham khảo
Customers (hồ sơ khách hàng)	1, 3, 4, 6, 9, 31, 32, 36, 37
Products (danh mục sản phẩm)	7, 8, 12, 14, 23, 35
Orders (đơn hàng)	10, 26, 29, 30, 34, 41, 42, 43, 49
Order_Items (chi tiết đơn hàng)	2, 21, 33, 44, 46
Customers + Orders (khách hàng & lịch sử mua)	5, 17, 19, 25, 45, 48
Orders + Order_Items (đơn hàng & chi tiết)	11, 15, 38, 39, 40
Products + Order_Items (sản phẩm & doanh số bán)	16, 20, 22, 24, 27, 28, 47
Đa bảng / báo cáo tổng hợp (3 bảng trở lên)	50
Schema hệ thống (information_schema)	13, 18

Cách đọc bảng: mỗi hàng liệt kê đúng bảng (hoặc tổ hợp bảng) mà câu lệnh thực sự JOIN/FROM — lấy trực tiếp từ chính SQL trong sách, không suy đoán theo chủ đề. Một số câu xuất hiện ở nhiều hàng nếu vừa truy vấn 1 bảng riêng vừa có biến thể JOIN sang bảng khác.

Trên dự án thật, schema thường có hàng chục–hàng trăm bảng — bảng tra này chỉ là ví dụ thu nhỏ. Cách áp dụng: khi vào một module mới, dùng Câu 13 (INFORMATION_SCHEMA) để tự liệt kê bảng/cột, sau đó map mỗi bảng nghiệp vụ với khuôn mẫu SQL gần nhất trong 50 câu — đó chính là Bước 3-4 của phương pháp 5 bước áp dụng cho module cụ thể.

Tóm lại

SQL không thay thế tư duy kiểm thử — nó khuếch đại tư duy ấy. Một câu lệnh chỉ mạnh khi đứng sau nó là một câu hỏi tốt: "Điều gì trong hệ thống này lẽ ra phải luôn đúng?". 50 câu lệnh ở đây là 50 câu hỏi như vậy, đã được viết thành truy vấn. Khi gặp một nghiệp vụ mới, hãy tự đặt câu hỏi của riêng bạn rồi dịch nó sang SQL theo đúng các mẫu trong sách.

Ba điều mang theo:

- Mọi truy vấn đối soát đều dựa trên một **bất biến** — tìm cái luôn đúng, rồi tìm cái vi phạm nó.
- Kết quả SQL là **danh sách cần điều tra**, không phải bản án. Luôn xác minh trước khi kết luận bug.
- Cẩn thận với **NULL**, **collation**, **kiểu số thực** và **khác biệt dialect** — đó là nơi bug ẩn nấp ngay trong chính câu lệnh của bạn.

Biên soạn bởi maiqai.com